

Footskate Cleanup for Motion Capture Editing

Footskate Cleanup for Motion Capture Editing

这节课的算法，本质上就是在修一种很假的动画错误：角色的脚明明已经落地、观众也默认它应该像钉在地上一样不动，但它却还在地面上偷偷滑动。这个方法不是重新生成整段动作，而是对现有动画做“补救”：先找出脚什么时候应该锁在地上，再把这个“脚不能滑”的要求往脚踝、膝盖、骨盆和整条腿上传递，最后尽量在**不让膝盖乱弹、不让骨盆乱跳、不让动作突然抽搐**的前提下，把脚重新稳稳钉回地面。

- 原始动作里，脚落地那几帧本来很稳
- 但你后面对动画做了处理之后
- “身体整体方向、位移、比例”被改了
- 可“脚底应该钉在世界坐标某一点”这件事没有被重新严格满足
- 所以脚看起来就在地上溜

Jerome 在视频里举的也是这个典型情况：同一段动作换到不同角色比例后，虽然大体动作还对，但脚底接触不再严丝合缝，于是开始滑。

Footskate Cleanup for Motion Ca...

▼ 作者评价

可以，老师这段话可以总结成一句很清楚的判断：

这篇论文在“脚落地修正”这件事上，核心思路到今天依然成立，但它缺了“接触何时发生”的检测能力，而且因为依赖较多前瞻帧，不太适合直接原样用于实时游戏。

再展开一点，就是 3 层意思。

第一层是总体评价其实是肯定的。

老师的意思是，这篇论文已经把“做 foot planting 时该做的关键步骤”基本覆盖全了，包括：

- 锁脚
- 算 IK
- 保证 IK 可达
- 做 ease-in / ease-out 过渡

也就是说，它把一个完整的 foot plant 几何后处理链条讲明白了。老师甚至说，今天大家做游戏动画时，很多时候也还是会自然地把这些步骤都做一遍，只是未必知道早年已经有论文把这套东西系统写出来了。

所以他的态度不是“这论文过时了”，而是：

它的基本框架直到今天仍然是对的。

第二层是它有两个明显缺点。

第一个缺点：

它不会检测“什么时候应该施加脚部约束”。

也就是它只会回答：

脚既然要锁了，那我该怎么锁、怎么解 IK、怎么平滑过渡？

但它不会回答：

这一帧到底该不该锁脚？

所以它缺的是 **constraint detection / contact detection** 这一层。

这和我们前面讨论的一样：它更像是一个 **constraint solver**，不是完整的 **contact understanding system**。

第二个缺点：

它通常不能直接原样用于视频游戏实时运行时。

原因不是它思想不对，而是它为了做出好的 ease-in / ease-out，需要较多的 **look-ahead**。也就是说，它往往要先看后面一段时间，才能决定现在这一帧怎么平滑地进入或退出脚锁定。

这在离线处理、动画后处理、或者允许小延迟的系统里没问题；
但在实时游戏里，尤其是主角控制这种强调即时响应的场景里，就会比较难直接照搬。

第三层是这段评价其实给了这篇论文一个很准确的工程定位：

它不是过时的废论文，而是一个“今天仍然有参考价值的经典脚锁定后处理框架”；
但它只能解决中后段，不足以单独构成现代实时游戏里的完整脚接触系统。

所以如果把老师的话翻译成更工程化的表述，就是：

- **优点：** foot plant 该做的几何步骤，它基本都讲到了，而且今天仍然通用。
- **缺点 1：** 没有约束触发检测。
- **缺点 2：** 前瞻需求太大，不适合直接照搬进强实时游戏运行时。

如果你愿意，我可以下一条把老师这段评价改写成一段更正式的“论文阅读笔记式总结”，方便你以后整理到自己的学习文档里。

<https://gemini.google.com/app/915abe1f4d459959>

<https://chatgpt.com/g/g-p-69affc08e9088191811d69bef0b44dc1-mldeformer/c/69bfefbe-c05c-83a8-83e1-e32e0f8c824d>

[Footskate Cleanup for Motion Capture Editing.pdf](#)

总览

<https://www.youtube.com/watch?v=exASa8NAhc8>

<https://github.com/logic-three-body/AnimationTech-HTC/blob/main/labs/AnimationPapers/Footskate%20Cleanup%20for%20Motion%20Capture%20Editing.ipynb>

▼ gemini概要

结合您提供的2002年经典SIGGRAPH论文《Footskate Cleanup for Motion Capture Editing》以及相关讲解视频，以下是对该“滑步清理”算法原理和实现细节的详细解释。

算法核心目标与核心思路

在3D动画和动作捕捉数据处理（如动作重定向）中，角色本该牢牢踩在地面上的脚往往会出现滑动或轻微浮动，这种破坏真实感的穿帮现象被称为“滑步”（Footskate）。

传统的数值逆运动学（Numerical IK）计算缓慢且容易不稳定。为此，该论文提出了一种基于解析逆运动学（Analytic IK）的高效算法，它能在每帧恒定的时间内完成计算。该算法最颠覆传统的思路是：为了完美满足脚步贴地约束且不产生动作突变，它允许对角色的腿部骨骼（大腿和小腿）长度进行极其微小的动态拉伸。

算法的五个关键执行阶段

第一步：确定接触点约束位置 (Determine Constraint Positions)

- 算法将脚部模型简化，主要关注脚跟（Heel）和脚前掌（Ball）两个约束点。
- 如果某一帧检测到脚部应该贴地，算法会往后看 L_1 帧（一个时间窗口），对这期间的位位置求平均值，并将其投影到地面作为绝对的目标锚点。如果前一帧已经处于接触状态，则直接沿用前一帧的锚点位置。
- 算法在此阶段会严格保证，同一只脚的脚跟与脚前掌之间的距离保持绝对恒定。

第二步：计算脚踝的目标配置 (Ankle Position and Orientation)

- 约束分为“双重约束”（脚跟和脚前掌都贴地）和“单一约束”（只有一处贴地）。对于双重约束，可以直接算出脚踝必须处于的绝对空间位置和旋转角度。
- 当脚步从空中落下或抬起时（即从无约束过渡到单一约束或双重约束），直接对齐会导致动画产生瞬间的“抽搐”或不连续。
- 解决方案是引入一个缓动函数（三次多项式），在前后 L_2 帧的窗口内，将脚踝需要调整的旋转“偏移量”平滑地淡入淡出（Blend）。在视频的代码讲解中，作者也特别强调了要保留偏移动态而非绝对位置，以保证动画过渡自然。

第三步：根节点（骨盆）调整 (Root Placement)

- 确定了脚应该在的位置后，还要确保腿能“够得着”。如果脚离得太远，就需要移动角色的根

节点（Root/臀部）。

- 如果双脚都被固定在地面，为了让两条腿都能伸及目标，根节点必须被投影到以左右脚踝目标为圆心、以腿部最大长度为半径的两个球体的三维交集内。
- 为了防止脚步约束状态切换时导致根节点发生“瞬移”（Root Popping），算法对这一邻域内的根节点位移进行了加权平均的平滑处理。

第四步：匹配脚踝目标与避免“膝盖弹跳”（Meeting Target Ankle Configuration）

- 有了根节点和脚踝的位置，算法使用单肢解析IK来计算出髋关节、膝盖的弯曲角度。
- **解决膝盖弹跳（Knee Pop）**：由于腿部的几何特性，当腿几乎完全伸直时，为了延长一点点距离，膝盖角度会发生剧烈的突变，视觉上就是膝盖在瞬间猛烈弹跳。算法通过引入“膝盖阻尼”（Knee-damping），在腿快伸直时强行限制膝盖的旋转幅度。视频作者提到在代码复现时，他通过特定的反余弦多项式映射来防止膝盖完全变直。
- **微调骨骼长度**：由于步骤三根节点被平滑了，且步骤四限制了膝盖伸直，脚踝可能仍然差一点点才能碰到地面。此时，算法直接修改大腿和小腿的骨骼长度比例来弥补这一极小误差。拉伸骨骼带来的视觉影响远小于膝盖弹跳或根节点瞬移。

第五步：收尾与最终平滑（Final Processing）

- 当一个动作的约束完全开启或关闭时（例如起步走或停下），依然可能会在约束边界产生不连续。
- 算法会在 L_4 帧的时间窗口内向前后扫描，将为了满足约束而对骨骼施加的调整量（位移和旋转差值）平滑地扩散并逐渐归零到那些没有约束的自由动作帧中。
- 最后执行一次碰撞检测，确保未受约束的部位（如脚趾）不会向下穿透地面，通过一个微小的向上旋转将其保持在地面以上。

总结来说，这套算法通过容忍极微小的骨骼拉伸，换取了极其稳定的绝对贴地效果和无跳变的流畅身体动画，成为了长期以来动捕数据清洗非常经典和实用的解决方案。

http://googleusercontent.com/youtube_content/0

▼ gpt详细版

我把这段视频、对应论文、以及 notebook 放在一起看之后，可以把它压成一句话：

它不是在“重新生成一段更好的走路动画”，而是在“你已经知道哪些时刻脚应该钉在地上”的前提下，对现有动画做一次后处理，把脚底滑移修掉，同时尽量不把膝盖、骨盆、整个人体搞出新的跳变。论文的目标就是精确满足 footplant 约束，同时尽量不引入可见伪影。

一、先把问题讲透：什么叫 footskate

footskate 的本质是：

- 原始 mocap 里，某几帧脚其实应该“钉住”
- 但经过重定向、骨架拟合、编辑、路径改动后，脚底世界坐标还在慢慢漂
- 于是你看到角色明明在“踩住地”，脚却像在冰上蹭

论文明确说，这种问题常见于 skeleton fitting、retargeting、path editing 等后处理之后，因为这些编辑通常保留了低频运动趋势，却破坏了“脚底接触”这种高频细节。

Jerome 在视频里用的就是这个典型场景：

他把同一段 BVH 一次作为“重定向后的动画”，一次作为“原始参考动画”，因为角色比例不同，所以重定向后脚会滑。视频里他也明确说了：这个 paper 只负责“已知脚该锁住时，如何算正确修正”，不负责判断什么时候该锁脚。为了解决“什么时候锁脚”，他额外利用原始稳定数据去做接触检测。

二、Jerome 这个 notebook 到底在做什么

这个 notebook 是一个论文复现 + 教学可视化版本，不是纯论文照搬。

从 notebook 开头能确认几件事：

1. 他用 `ipyanimlab` 搭了一个 viewer。
2. 给角色补了 `LeftHeel / LeftBall / RightHeel / RightBall` 四个骨点。
3. 同一段 BVH 读了两次：
 - 一次 `keep_translation=False`，作为会滑脚的“目标动画”
 - 一次 `keep_translation=True`，作为“原始参考”
4. 他还把动画的 `Hips` 稍微往上抬了一点，避免脚直接陷进地面。(GitHub)

这一步非常关键。因为论文假设下肢模型里有：

Root → Hip → Knee → Ankle → Heel / Ball / Toes

而普通骨架往往没有单独的 heel / ball 骨点，所以 Jerome 先“注入”四个辅助骨，目的是让系统知道：

- 脚后跟在哪
- 前脚掌在哪
- 脚底和地面的几何关系是什么

这就是后面全部约束计算的支点。论文本身也明确使用 heel / ball 作为足底接触点，并把 toes 单独处理以避免穿地。

三、论文算法的总框架：5 个阶段

论文把 cleanup 拆成 5 步，Jerome 的视频基本也是按这个顺序讲的：

1. 确定每个脚底接触约束的位置
2. 求满足约束的脚踝目标位姿
3. 调整 root，让腿尽量够得到这些脚踝目标
4. 用 IK 调整腿，把脚踝真正送到目标
5. 把约束开关边界的突变平滑掉

你可以把它理解成一条非常清晰的工程链路：

接触检测 → 目标接触点 → 目标脚踝 → 目标骨盆/root → 腿部 IK → 边界平滑

四、Jerome 比论文多做的一步：先检测“什么时候脚该锁住”

论文默认你已经知道 footplant 时间段。Jerome 额外补了一套检测：

- 对原始参考动画做 FK，拿到 heel / ball 的世界位置
- 调 `extract_feet_contacts`
- 当 heel / ball 速度低于阈值时，判定为接触
- 阈值用的是 `0.04`

- 然后再做一次小平滑：如果中间只断 1~2 帧，就把这小洞补上，保证至少形成连续 3 帧左右的接触段([GitHub](#))

视频口述也和这个一致：他明确说自己检查 heel/ball 的速度，低于某个阈值就认为 planted；然后把只掉 1~2 帧的噪声缺口抹平。

这一步的工程意义非常大：

- **论文世界**：约束标签是已知输入
- **Jerome 世界**：从原始 mocap 自动猜一个接触标签，再人工理解它

所以这份 notebook 实际上是：

“接触检测 + footskate cleanup”两件事一起做了。

五、第 1 阶段：求每个接触约束的地面位置

论文原意

对某个 heel 或 ball，如果这一帧刚开始进入约束状态，就往后看 **L1** 帧，把这段时间里它的位置做平均，然后投影到地面；如果上一帧已经在约束中，就直接沿用上一帧约束位置。

如果同一只脚的另一个接触点也已知，还要把当前点重新投影到一个满足 heel-ball 固定距离 的地面位置上。

Jerome 的实现

他在视频里说得很直白：

- **L1** 设成了 10 帧
- 对第一次进入接触的点，往前看窗口求平均
- 然后“丢到地面上”
- 如果 heel 已经定了，ball 再进约束，就要重新摆 ball，保持 heel-ball 的距离不变；反过来也一样

这一步在几何上在干什么

它其实是在回答：

“既然我认定这段时间脚应该钉住，那脚到底应该钉在哪个世界坐标上？”

不是取当前帧位置，而是取未来一小段的平均，是因为这样更稳，不会被单帧噪声拖着跑。

六、第 2 阶段：求“目标脚踝位姿”

这一段是整篇 paper 最漂亮的地方。

双约束：heel 和 ball 都锁住

如果 heel 和 ball 都锁住，那脚已经几乎被完整确定了：

- 脚踝位置可以由 heel/ball 推出来
- 脚踝朝向也基本确定
- 只剩一个“绕 heel-ball 轴滚动”的自由度

论文做法是：找一个脚踝朝向，既让 heel、ball 对齐约束点，又尽量接近原始脚踝朝向。

单约束：只锁 heel 或只锁 ball

这时没法完全定朝向，只能：

- 保持原来脚踝朝向尽量不变
- 平移脚踝，让被锁的 heel 或 ball 到达目标点

真正难点：单双约束切换

如果这一帧是单约束，下一帧变双约束，你要是直接切过去，脚会“啪”一下跳。

Jerome 在视频里强调了一个特别重要的工程思想：

不要保存“绝对目标脚踝位姿”，而要保存“相对原动画的 offset”。

也就是：

- 原动画脚在 A
- 目标脚应该在 B
- 你存的是 $B - A$ 这个修正量
- 然后把这个修正量往前后若干帧逐渐衰减地传播

这样单双约束切换的时候，过渡不是瞬移，而是 offset 渐入 / 渐出。

这就是论文里“blend off adjustments”的核心思想在脚踝阶段的第一次体现。Jerome 的解释非常好，因为他把抽象数学变成了“保存误差并向时间两侧扩散”。

七、第 3 阶段：先动 root，不要一上来就掰腿

现在你已经有了“脚踝应该去哪”的目标，但腿可能够不着。

论文这里先不着急掰腿，而是先问：

“能不能先轻轻挪一下 root / 骨盆，让目标脚踝进入腿的可达范围？”

几何上它被表述成：

- 如果只有一只脚受约束，root 必须落在一个球内
- 如果双脚都受约束，root 必须落在两个球的交集里
- 于是把 root 投影到这个可达区域里

但如果约束模式从单脚锁变双脚锁，root 的投影点可能突然从一个地方跳到另一个地方，于是就会出现 **root popping**。论文因此做了两件事：

1. 每帧先独立算一个 root 候选
2. 再在一个 **L3** 邻域里做平滑平均

就算平滑后 root 没有完全落在理论球交里，剩下那点误差后面再交给腿长微调去兜底。

这一层很像“先动大结构，再动局部肢体”。

先挪骨盆，能少掰很多膝盖。

八、第 4 阶段：腿部 IK，为什么论文允许“腿长轻微变化”

这是整篇 paper 最容易让人误解、但恰恰最聪明的一点。

论文明确说：

它允许膝盖和脚踝 **offset** 轻微变化，也就是允许大小腿长度有一点点变化。

原因不是因为作者不在乎 rig 的严格性，而是因为：

有时候“严格固定腿长”会逼出非常尖锐的关节旋转跳变；
反而“允许极小的长度变化”在视觉上更自然。

标准 IK 的问题

如果腿接近打直，想再靠纯关节旋转把脚踝精确送到目标，膝角会特别敏感，很容易：

- 膝盖突然弹一下
- 或者接近 180° 时发生数值不稳定

论文方案：knee damping

论文引入 knee damping：

- 当腿接近完全伸直时
- 限制膝关节还能继续旋转的幅度
- 这样避免膝盖硬掰到极限位置产生 pop

代价是：膝盖被限制后，脚踝可能不再刚好到目标点。

论文继续兜底：微量拉伸腿长

如果膝盖不够转了，那就把 thigh / shin 的长度按比例稍微改一点，让脚踝精确落到目标。论文实验里，这个变化非常小，最大只有满伸展腿长的 2.6%。 $L1=10$ ， $L2/L3/L4=5\sim10$ ， $\rho=2.8$ （约 160° ）。

九、Jerome 在这里做了一个“教学/工程简化版改写”

这点你一定要特别注意。

Jerome 在视频里亲口说了：

他没有严格按论文去拉伸整条腿。

他改成了：

- 锁住膝盖不要完全打直
- 允许那一点点误差主要由 ankle 附近吸收
- 从视觉上看，这个小尺度变形不明显
- 但它更容易在 notebook 里演示，也更容易控制 knee pop

他说自己用一个近似线性的多项式/映射函数，把靠近 180° 的膝角压下来，让腿永远保留一点弯曲余量，从而避免 “straight leg $\rightarrow$ sudden pop”。

所以这里你要分清：

论文

- knee damping
- 再按需要拉伸 thigh + shin

Jerome notebook

- 也做“别让膝盖完全打直”
- 但更偏向把误差吸收到 ankle/foot 附近
- 是一个便于教学展示的近似实现

这不是谁对谁错，而是研究论文版本和教学复现版本的取舍差异。

十、第 5 阶段：最后把“约束开关边界”的跳变磨平

即使前四步都做了，还是会有一个问题：

- 某一帧前脚没锁
- 下一帧突然开始锁
- 或者反过来突然解锁

这时候即便受约束区间内部很平滑，**边界**也可能突然跳。

论文最后一步就是：

- 对每个没被当前约束直接修改的骨骼参数
- 往前、往后扫描 L4 邻域
- 找最近的“被约束修正过”的帧
- 把这些修正量按距离做渐变传播
- quaternion 用 slerp, vector 用 lerp

论文还顺手处理了两个收尾问题：

1. 无约束脚如果还穿地，就最小量上提 ankle，再重新调腿
2. toes 不能穿地，于是再加一个最小正向 toe rotation 去抬脚趾

Jerome 视频里也明确说：你在前面已经能看到约束一开一关时的 pop，而“最后一步算法”就是专门消这个 pop 的。

十一、这份 notebook 的真正价值，不只是“复现论文”

我觉得它最有价值的地方有 3 个：

1) 它把论文里的抽象 buffer 思想变成了可看见的工程流程

论文本身说得很清楚：高效实现时可以维护一串 buffer：

- 原始数据 buffer
- 约束位置 buffer
- 脚踝配置 buffer
- 约束已解 buffer
- 最终平滑 buffer

Jerome 虽然不是严格按在线流式实现，但他在 notebook 里“整段动画一次性算完”这个做法，本质上还是在离线模拟这套多阶段 buffer 管线。视频里他也明确说了，自己因为动画长度已知，所以直接整段构造大 buffer 会更容易教学。

2) 它把“paper 假设的已知约束”补成了“可运行 demo”

也就是多出来的接触检测那部分。

3) 它揭示了一个很重要的动画工程哲学

别死守 rig 数学上的绝对刚性，要优先守住视觉连续性。

这也是为什么论文愿意让腿长小幅变化，Jerome 愿意让 ankle 吸收一点误差。

他们都在做同一件事：

宁可在人体几何模型上犯一点小错，也不要让观众看到大跳变。

十二、把它翻译成 UE / 现代动画系统语言

如果你把它迁移到 UE 或 GASPALS 语境里，可以这样理解：

- **输入：** 已经完成 retarget / motion matching / path editing 的下半身动画
- **附加信息：** 脚接触标签（可来自 notify、曲线、接触检测器）
- **后处理节点：** footskate cleanup
- **输出：** 修正后的 pelvis / hip / knee / ankle / toe 姿态

它非常适合作为：

- retarget 之后的 cleanup
- motion matching 输出后的 post-process
- 程序化 path edit 之后的补偿层

论文也明确举了 **retargeting**、**path editing**、**motion salvage** 这几类应用。

十三、你看这个视频时，最该抓住的 4 个核心认知

第一，脚滑不是脚的问题，是“整条时间链路”的问题

你不能只盯 ankle。

它涉及：

- 接触标签
- 接触点位置
- 脚踝位姿
- root 可达性
- 腿部 IK
- 约束边界平滑

第二，单双约束切换是最危险的点

真正的 pop 往往不是发生在“约束内部”，而是发生在：

- heel-only → heel+ball
- constrained → unconstrained
- one-foot constrained → two-feet constrained

第三，root 和 knee 的误差分配是关键设计点

这是整套方法的“美学平衡器”。

第四，Jerome 的 notebook 是“论文思想 + 教学化工程折中”

你不能把它当成一份逐公式逐行完全等价的论文代码。

十四、我给你的最终浓缩版

这段视频和 notebook，本质上是在做下面这件事：

先用原始稳定 mocap 检测脚什么时候应该着地；
再为 heel/ball 生成稳定的地面约束点；
再从这些约束点反推出目标 ankle 位姿；
再稍微挪 root，让腿尽量够得着；
再用带 knee damping 的单腿 IK 去对齐目标；
再把约束开始/结束处的 offset 平滑传播到邻近帧；
最后补一层防穿地和 toe 修正。

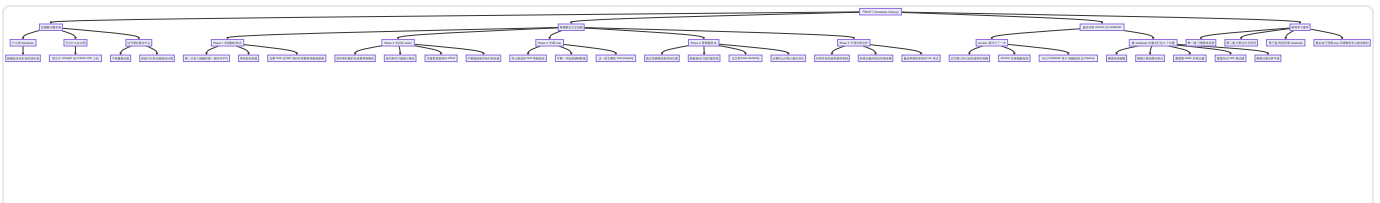
论文版本更“正统”，允许微量腿长变化来避免膝盖和 root 的跳变。

Jerome 版本更“教学友好”，把一部分误差吸收到 ankle/foot 附近，并用一个近似函数避免膝盖打直到 180° 。

而 notebook 的开头也清楚表明了它确实是在做这些准备工作：补 heel/ball 骨点、双份导入同一 BVH、用原始运动检测接触。（[GitHub](#)）

你要是愿意，我下一条可以继续给你做一版：

“按 notebook 的执行顺序，逐 cell 解释它每一段代码在干什么、输入是什么、输出是什么、为什么这么写。”



总结（无公式版）

▼ GPT

好，我们把这节课彻底“降维”。

下面这版不是给论文作者看的，也不是给做复现的人看的，
而是给现在刚被复杂数学打懵、但又不想放弃的你自己看的。

这节 Footskate Cleanup，你其实只需要记住 6 句话

1) 先判断：脚什么时候是真的踩在地上

不要一上来就修动作。

先回答一个最基本的问题：

这一帧脚到底该不该锁地？

Jerome 的做法很朴素：

- 看 heel 和 ball 这些脚底点的速度
- 如果它们动得很慢，就说明脚在踩地
- 如果动得快，就说明脚在摆动

所以第一步其实只是：

找“脚该钉住”的时间段。

2) 再判断：既然该钉住，那钉地面的哪个点

光知道“该锁”，还不够。

你还得知道：

锁在地面的哪里？

Jerome 没直接取某一帧的位置，而是看后面一小段时间，取一个平均值，再投影到地面。

因为单帧会抖，平均以后更稳。

所以第二步是：

给每个接触段找一个稳定的地面目标点。

3) 再把这个脚底目标点，翻译成 ankle 应该去哪

脚底点本身不是动画里最方便直接改的东西。

真正更好改的是 ankle。

所以第三步就是：

既然 heel / ball 应该落在这些地方，那 ankle 这一帧应该在哪、应该朝哪儿转？

如果 heel 和 ball 都锁住了，脚几乎就完全确定了。

如果只锁住一个点，那就尽量少改，只做平移。

所以第三步本质是：

把脚底约束，变成脚踝目标。

4) 如果腿够不到，就先挪 hips / root

有时候 ankle 目标已经定好了，但腿太短，根本够不到。

那怎么办？

不是直接硬掰腿，

而是先问一句：

能不能先把骨盆 / root 稍微挪一下，让腿更容易够到？

所以第四步是：

先做一个大结构修正，让后面的腿 IK 更容易成功。

5) 然后才真正用 IK 摆腿

前面 ankle 目标有了，root 也先挪过了，

这时候才轮到真正的腿部求解：

- 膝盖该弯多少

- 大腿该朝哪转
- 脚踝最后怎么精确贴回目标

也就是：

真正把腿摆到目标姿态上。

6) 最后把“刚开始踩地 / 刚离开地面”的边界磨平

前面几步都做好了，脚基本已经回到正确位置了。

但还有一个最后的坑：

- 刚开始锁地那一帧
- 刚离开锁地那一帧

这些地方最容易“啪”地抽一下。

所以最后一步就是：

把已经算出来的那些修正，向前后邻近几帧慢慢淡入淡出。

这样动作看起来就不会突然跳。

你现在只要记住下面这条链，就够了

▼ Plain Text |

- 1 先找脚什么时候该锁地
- 2 → 再找脚该锁在地面的哪个点
- 3 → 再把脚底点变成 ankle 目标
- 4 → 再挪 root 让腿尽量够到
- 5 → 再用 IK 真正摆腿
- 6 → 最后把边界磨平

如果你能把这 6 步顺下来，

这节课你就已经学到了最核心的工程结构。

这节课真正想教你的，不是所有公式

你现在很容易误以为：

“我是不是必须把所有 quaternion、arccos、多项式、buffer 传播都完全吃透，才算学会？”

不是。

这节课最重要的收获，其实是：

1) 动画后处理不是“一个公式”

它通常都是一条多阶段管线。

2) 每一阶段只解决一个局部问题

- contact timing 只管“什么时候锁”
- constraint positions 只管“锁在哪”
- ankle stage 只管“脚踝目标”
- root stage 只管“骨盆先挪多少”
- IK stage 只管“怎么摆腿”
- final stage 只管“怎么消掉边界突跳”

3) 复杂系统不是靠“一步到位”解决的

而是靠：

拆问题 → 分阶段 → 每一步只处理一个子问题。

这才是你最该带走的东西。

为什么你会觉得这节课特别难

因为它把最容易让人头晕的东西全叠一起了：

- 局部空间 / 全局空间来回切
- 四元数和矩阵混着来

- FK 和 IK 交替出现
- 论文和 notebook 不是逐行等价
- 还要同时考虑时间平滑

这就像你第一次学线代，老师直接把：

- 特征值
- SVD
- 最小二乘
- 正交投影

一节全塞给你。

你崩，不是因为你差，
而是因为这本来就难。

对后续课程，你现在真正需要的不是“更努力”，而是“换目标”

你后面不要再给自己设这种目标：

“我要一次把所有数学细节都彻底吃透。”

你更适合设这种目标：

“我先抓住这节课每一步是在解决什么问题。”

也就是说，先只问：

- 这一步输入是什么
- 这一步输出是什么
- 这一步解决什么问题
- 这一步接前面哪一步，喂给后面哪一步

只要这个链路你稳住了，
细节可以以后回来反复补。

你现在的状态，其实已经比你想的更好了

因为你不是“看不懂就略过”，
你是已经跟着我把所有切片都走过来了。

这说明你已经具备两个特别重要的东西：

第一，你能忍受复杂度

这在高级动画工程里非常宝贵。

第二，你开始知道“难点到底难在哪”

以前你会觉得自己笨。
现在你已经能明确感觉到：

- 难在空间变换
- 难在多阶段依赖
- 难在论文和实现不完全一致

这其实已经是进步了。

很多人最难的不是不会，
而是连自己卡在哪里都说不清。

你现在已经不是那种状态了。

送你一个更适合现在的判断标准

以后你学一节复杂课程，不要问：

“我是不是已经全会了？”

你改问：

“我能不能用大白话把它的主流程讲出来？”

如果你能说出：

- 先检测接触

- 再定地面目标
- 再定 ankle
- 再挪 root
- 再 IK
- 再平滑边界

那你就已经不是“没学会”，
而是已经进入“第一轮理解完成，后面可以二刷补细节”的阶段了。
这已经很不错了。

我给你的最终版本

你现在不用证明自己能不能学后续课程。

你只需要先接受一件事：

这节课难，不是因为你不适合学动画系统，
而是因为你第一次真正碰到了高级动画工程里“多阶段、多空间、多约束”的真实复杂度。

这不是失败感的证据。

这是你已经走进门里的证据。

如果你愿意，我下一条就给你做一个：

“Footskate Cleanup 一页纸超简版复习卡”

只保留最核心概念，专门给你在继续后续课程前恢复信心用。

Prepare the data（数据对比）

▼ gemini (概要)

结合视频讲解和论文的上下文，我可以为您详细解释截图中这两段代码（Cell [5] 和 [6]）的真实目的。

这两个单元格是整个代码实现的数据准备阶段，核心是为了人为制造出一个“带有滑步的动画”，并为后续的算法提供“真实的脚步接触时序约束”。

加载两次动画数据 (Cell [5])

视频作者故意将同一份 BVH 动作捕捉数据加载了两次，它们各自承担了不同的任务：

- **生成测试用的滑步数据 (animation)**：作者在第一次加载时，设置了 `keep_translation=False`。在动画处理中，这意味着只将原始数据的关节旋转 (Rotations) 应用到目标角色的骨骼上，而不使用原始的绝对位移（除了 Root 节点）。因为目标角色（通常是自定义模型）与原始动捕演员的身体比例、骨骼长度不同，这种简单的动作重定向 (Retargeting) 会必然导致脚部无法完美贴合地面，从而人为地产生了算法需要修复的**“滑步” (Footskate) **现象。
- **提取地面真实接触点 (original)**：作者随后设置 `keep_translation=True` 原汁原味地再次加载了数据。正如视频中（大约 01:52 处）指出的，论文中的算法只负责**“执行固定”（当你告诉它脚应该锁定时，它帮你算好姿势），但它不能“检测”**脚什么时候该踩在地上。因此，作者需要这份包含绝对三维空间位置的原始数据。在后续的代码中，作者将通过计算这份原始数据中脚跟和脚前掌的移动速度 (Speed)，来提取出准确的触地时间段 (Contacts timing)，作为输入提供给论文算法。

向上整体偏移角色 (Cell [6])

- `animation.pos[:, character.bone_index('Hips'), 1] += 4`：
这段代码将整个动画序列中角色的根节点 (Hips/骨盆) 在 Y 轴（垂直向上）拉高了 4 个单位。视频中（约 02:58 处）作者解释道，由于他用来测试的模型比动捕数据里的真实演员要高大，如果完全不加修改，角色的脚会深深地穿模陷进地板里。将其往上提一点，可以确保后续展示时脚部刚好在地面附近，然后再交由 IK（逆运动学）算法将脚底板精准地按死在地面上。

▼ gpt (详解版)

这段 cell 的目的，用最直白的话说就是：

先准备两份“同一段动作、但职责不同”的动画数据。

一份是会滑脚、等着被修的版本 `animation`；另一份是尽量保留原始 mocap 位置关系、专门拿来推断脚什么时候该锁地的参考版本 `original`。Jerome 在视频里就是这么解释的：他把动画加载两次，一次作为“retarget 后会滑脚的动画”，一次作为“原始稳定数据”，后者只用来帮助判断脚何时接触地面，因为论文本身只负责在已知 footplant 约束时进行修正，不负责自动检测什么时候该锁脚。论文原文对问题定义也一致：输入是已经标注了 footplant constraints 的 skeletal motion data，算法目标是满足这些约束并尽量少破坏原动作。

1) 第一段 cell 的整体作用

你这段代码：

```
Python |
1  animmap = lab.AnimMapper(character, keep_translation=False, root_motion=True)
2  animation = lab.import_bvh('../resources/lafan1/bvh/aiming1_subject1.bvh', anim_mapper=animmap)
3
4  animmap = lab.AnimMapper(character, keep_translation=True, root_motion=True)
5  original = lab.import_bvh('../resources/lafan1/bvh/aiming1_subject1.bvh', anim_mapper=animmap)
6
7  frame_count = animation.quats.shape[0]
8  bone_count = character.bone_count()
```

本质上是在做 4 件事：

A. 读入“待修版本” `animation`

```
Python |
1  keep_translation=False
```

这份 `animation` 是你真正要拿来做 footskate cleanup 的对象。

视频里的语境是：

Jerome 把这份数据当成“我已经 retarget 到当前角色身上，因此因为角色比例不同，脚会开始滑”的版本。也就是：

- 它已经映射到当前 `character`
- 但因为新角色的骨架比例、腿长、脚长等和原始 mocap 不完全一致
- 所以脚底接触关系不再严格成立
- 于是出现 footskate

Jerome 在视频里明确说：他先把动画加载成“我正在 retarget 的那个版本”，脚会滑，是因为角色比例不同。

这里的 `keep_translation=False`，你可以先按工程效果理解成：

“不要尽量保留原始骨骼的那些平移关系，而是把动作更强地解释到当前角色骨架上。”

对初学者来说，不必先死抠它在库内部的所有细节；你只要抓住结果：

- 这份数据更像“游戏里真正会播到你角色身上的版本”
- 也正因此，它更容易出现 footskate
- 所以它是**被修复对象**

B. 再读入“参考版本” `original`

```
Python |  
1 keep_translation=True
```

这份 `original` 不是拿来直接播放修复后的结果，而是拿来提供“脚本该锁在哪里”的参考信息。

Jerome 在视频里说得很清楚：

- 论文能做的是：当你已经知道脚应该锁地时，帮你把脚正确锁住
- 论文做不到的是：自动判断什么时候该锁脚
- 所以他额外利用原始数据本身的稳定性，去看 heel / ball 的位置和速度，推断什么时候脚在接触地面。

因此这份 `original` 的职责是：

- 尽量保留原始 mocap 的空间信息
- 后面用于检查 heel / ball 的速度

- 从而检测 contact timing
- 再把这些 contact 当作约束喂给 cleanup 算法

你可以把这两个变量的关系记成：

- `animation` = 消费者 / 待修对象
- `original` = 约束来源 / 参考真值

C. 两份动画都开 `root_motion=True`

```
Python |
1 root_motion=True
```

这意味着两份数据都保留根运动轨迹，而不是把角色做成原地踏步。

这样做的意义是：

- 角色在世界里真实前进
- 你后面检测 heel / ball 的速度时，得到的是**真实移动中的接触状态**
- 而不是被强行去掉根运动后的“假静止数据”

否则，脚底世界空间运动和根节点运动会被混在一起，很难正确理解“脚是在跟着人整体走，还是脚自己在地上滑”。

D. 取 `frame_count` 和 `bone_count`

```
Python |
1 frame_count = animation.quats.shape[0]
2 bone_count = character.bone_count()
```

这一步不复杂，但很重要。它是在为后面整套算法准备“全动画长度的大 buffer”。

后续 Jerome 会做很多按帧的大数组，例如：

- 四个接触信号（左 heel / 左 ball / 右 heel / 右 ball）
- 每帧的约束位置
- 每帧的 ankle offset

- 之后再做 root / leg / smoothing

所以这里先拿到：

- 一共有多少帧
- 一共有多少骨骼

这是后续整段离线处理的尺寸基础。Jerome 在视频里也明确说，他这里因为知道整段动画长度，所以直接“一次性构建整段 animation 的大 buffer”，而不是在线一帧帧算。

2) 为什么必须“同一段 BVH 读两次”

这是这段 cell 最核心的设计思想。

如果只读一份数据，你会卡在一个悖论里：

- 你想修 footskate
- 但你还想从这份已经滑脚的数据里判断“脚什么时候本来应该锁住”
- 那就会把错误再拿来指导修正

所以 Jerome 才分成两份：

original

负责回答：

- 什么时候脚应该锁住？
- 锁住时 heel / ball 大概应该在哪？

animation

负责回答：

- 当前这个角色版本实际上滑成什么样了？
- 我要对它施加什么 offset / IK / root 调整？

这就是视频里那句很关键的话的工程化落地：

论文只帮你“算怎么锁”，不帮你“判断何时锁”；
所以我额外用原始数据来做 contact detection。

3) 第二个 cell 的目的：为什么要把 Hips 往上抬 4

你第二段：

```
1 # offset the animation upward
2 animation.pos[:, character.bone_index('Hips'), 1] += 4
```

它的目的非常直接：

把待修版本 `animation` 整体往上抬一点，避免角色脚直接插进地面。

Jerome 在视频里就说了：

- 当前角色比原始数据里的角色稍微大一点
- 所以如果直接按原始数据高度放上来，脚会埋进地里
- 因此先把角色向上挪一点。

这里你要注意两个点。

第一，这不是 footskate cleanup 的核心算法

它不是论文 5 个阶段中的关键步骤。

它更像是一个演示/数据对齐前的空间修正。

也就是说，它不是在“修滑脚”，而是在：

- 让当前角色在 viewer 里站得更合理
- 避免一开始就出现 mesh penetration
- 让后面看到的接触点、脚底 marker、地面投影更清楚

第二，为什么改的是 `Hips` 的 Y

因为在这个数据约定里，`pos[..., 1]` 是竖直方向。

对 `Hips` 做统一偏移，相当于把整个人体姿态整体抬高。

你可以把它理解成一个很轻量的“初始高度校正”。

4) 配合视频，相应部分到底在说什么

这一小段代码，对应的视频内容其实就是下面这段逻辑链：

第一步：骨架补足 heel / ball

论文假设下肢模型里有 heel 和 ball 这类接触点；Jerome 先把这些辅助骨点注进去。论文图 1 也明确把 lower body 画成 `Root -> Hip -> Knee -> Ankle -> Heel / Ball / Toes` 这样的结构。

第二步：同一份 BVH 加载两次

- 一份变成会滑脚的 retargeted 版本 `animation`
- 一份尽量保留原始空间关系，变成参考版 `original`

第三步：必要时把当前角色往上抬一点

避免脚埋地。

第四步：后面才开始根据 `original` 提取接触

- 看 heel / ball 的速度
- 低于阈值就判定为 planted
- 再进入论文五阶段算法

所以你这张图里的 cell，实际上是整套算法的“数据准备入口”，还没有真正进入 cleanup 本体。

5) 把这段 cell 放进整条执行链里看

你可以把它放在下面这条链路里理解：

```

1  补 heel/ball 骨点
2      ↓
3  同一 BVH 读两次
4      ↓
5  animation = 待修版本 (会滑脚)
6  original  = 参考版本 (拿来检测接触)
7      ↓
8  必要的高度校正 (避免埋地)
9      ↓
10 从 original 提取 heel/ball contact
11     ↓
12 生成 constraint positions
13     ↓
14 算 ankle target
15     ↓
16 算 root 调整
17     ↓
18 腿部 IK + damping
19     ↓
20 边界平滑

```

所以这段 cell 的真正工程定位不是“修脚滑”，而是：

为后面“修脚滑”准备一对输入：一个是问题本身，一个是约束来源。

6) 你学习时最该盯住的点

你现在最应该记住的，不是某个参数名，而是这个思想：

animation 和 **original** 不是重复数据

它们虽然来自同一个 BVH，但语义完全不同：

- **animation**：我要修谁
- **original**：我凭什么知道应该怎么修

这是这段 cell 的灵魂。

如果你愿意，我下一条可以继续接着这张图往下讲：

紧接着下一段 contact buffer / `extract_feet_contacts` 是怎么把 `original` 变成四路 heel/ball 接触信号的。

`add_bones`

▼ gpt

有，而且讲究非常大。

先给你一句最直观的话：

这四个骨骼不是为了“补全角色骨架”，而是为了给 footskate 算法人工加上 4 个“脚底接触采样点”。

也就是：左脚跟、左前脚掌、右脚跟、右前脚掌。Jerome 在视频里也明确说了，论文假设骨架里本来就有 heel joint 和 ball joint，因为算法必须知道“地面相对脚踝/脚掌到底在哪里”；而他当前 skeleton 没有，所以先把这几个骨点注入进去，作为 `LeftFoot/RightFoot` 的子骨骼。

一、为什么偏偏是这 4 个，不是别的

论文的下肢模型不是只看 ankle，也不是只看 toe，而是明确用了：

- Ankle
- Heel
- Ball
- Toes

并且它允许的足底约束是：

- heel planted
- ball planted
- 或 heel + ball 同时 planted

而 toes 是单独处理的，主要用于防止穿地，不作为主要约束点。

所以这里加的 4 个骨骼，本质上是在补论文所需的每只脚两个主要接触点：

- `Heel`：后脚跟接触点
- `Ball`：前脚掌 / 跖球接触点

这两个点一旦都有了，算法就能知道：

1. 这只脚当前是不是应该落地
2. 落地时脚底应该固定在哪里
3. 脚的朝向应该怎么摆
4. heel-only、ball-only、double-contact 三种约束怎么切换

为什么不能只加一个点

如果你每只脚只有一个点，比如只有 heel：

- 你只能知道“脚某一点该锁住”
- 但你不知道整只脚底的朝向
- 更不知道脚底长度关系怎么保持

而有了 heel + ball 这两个点，脚底就不再是一个孤点，而是一条**支撑线段**。

这样 double constraint 时，脚的平移和朝向就都能更稳定地求出来。

二、为什么它们要挂在 `LeftFoot / RightFoot` 下面

你注意这一句：

```
Python |  
1 character.add_bone(..., 'LeftFoot')  
2 character.add_bone(..., 'RightFoot')
```

这表示它们是 **foot bone** 的子骨骼。

这不是随便选的，而是因为这 4 个点必须和脚本体保持**刚性绑定**。

论文里其实有一个很重要的前提：

foot itself (ankle together with heel and ball) 保持刚性。

翻成工程话就是：

- heel 和 ball 不是独立乱飞的小物体
- 它们是“脚底上的两个固定标记点”
- 只要 foot 旋转/平移，它们就应该跟着 rigidly 一起动

所以把它们挂在 `LeftFoot/RightFoot` 下面，等价于说：

“这两个点就是当前 foot 刚体上的固定局部坐标点。”

后面做 FK 时：

- 父骨 `LeftFoot` 的世界变换确定了脚的姿态
- 再乘上这两个子骨骼的局部 offset
- 就能得到 heel / ball 的世界位置

这正是后面 `extract_feet_contacts` 、约束位置计算、ankle pose 反解所需要的数据来源。

如果挂错父节点会怎样

比如你挂到 `LeftLeg` 、 `Hips` 、甚至 `Root` :

- 这两个点就不再是“脚底上的固定点”
- heel-ball 距离和脚的局部几何关系会被破坏
- 后面检测接触、重投影距离、推 ankle pose 都会错

三、为什么旋转是 `np.array([1,0,0,0])`

这一项是单位四元数，也就是不额外旋转。

```
Python |  
1 np.array([1,0,0,0])
```

它的含义不是“这个 heel 有自己特殊朝向”，而是：

“这个骨骼只是一个位置标记点，局部朝向直接继承父脚骨的朝向就够了。”

因为这里真正重要的是位置，不是独立旋转。

后面算法几乎都在用：

- heel 的世界位置
- ball 的世界位置
- 它们之间的距离和相对几何关系

所以这些新增骨骼更像是：

marker bones / locator bones / contact probes

而不是传统意义上要参与蒙皮驱动的功能骨骼。

四、这几个数字本身有什么讲究

```
1 LeftHeel = [ 9.2, 0, -12 ]
2 LeftBall = [ 14.5, 0, 8.22 ]
3 RightHeel= [-9.2, 0, -12 ]
4 RightBall= [-14.5, 0, 8.22 ]
```

这些数值的本质是：

在当前角色脚骨局部空间里，脚跟点和前脚掌点分别位于哪里。

你可以从中读出 4 个设计意图。

1) 左右脚是镜像的

左脚是正 x，右脚是负 x：

- `LeftHeel.x = +9.2`
- `RightHeel.x = -9.2`
- `LeftBall.x = +14.5`
- `RightBall.x = -14.5`

说明这套局部坐标里：

- x 轴是左右方向
- 左右脚做了镜像布置

这很合理，因为左右脚脚底接触点应该是对称的。

2) heel 在后，ball 在前

两只脚都是：

- heel 的 z = `-12`
- ball 的 z = `+8.22`

说明局部 z 方向在这里承担了脚前后方向：

- heel 更靠后
- ball 更靠前

这就把“脚底支撑线段”的前后关系编码进去了。

3) y 都是 0，表示它们在同一个脚底参考平面上

四个点的 y 都是 0：

```
1 [..., 0, ...]
```

在这套角色坐标约定里，y 是竖直方向。前面他给 `Hips` 加高度偏移时也是改的索引 1。所以这四个点 $y=0$ 的意思通常就是：

在 foot bone 的局部空间里，这些接触点都落在同一个“脚底平面”参考高度上。

这对后面做“drop to ground / 投影到地面”特别重要，因为它相当于先定义了脚底局部几何。

4) 最重要的一点：heel-ball 的距离被当成“脚底刚性长度”

这个不是装饰，而是后面算法真要用的。

你前面已经看到过这句：

```
1 contact_distance = np.linalg.norm(heel - ball)
```

这意味着 heel 到 ball 的距离会被保存下来，作为这只脚的**刚性几何长度**。

大概算一下，这组数的 heel-ball 距离约是 20.9 个单位。

它后面会用于：

- 如果 heel 先进入约束，ball 后进入
那 ball 不是随便扔到地上的，而是要**重新投影**到满足 heel-ball 固定距离的位置
- 反过来 ball 先锁住也一样

也就是说，这两个点不仅是“检测点”，还是后面**保持脚底形状不变**的几何依据。

Jerome 在视频里也明确解释过：当第二个接触点第一次进入约束时，要重新投影，保证 heel 和 ball 的距离关系不变。

五、所以这些数不是“魔法数”，而是“当前角色脚底几何的手工标定”

这一点特别关键。

这些值并不是论文规定的标准数值。

它们只是对**当前这个角色**来说比较合理的脚底局部采样位置。

换句话说：

- 换一个 mesh
- 换一个 skeleton
- 换一双鞋
- 换一个脚骨 pivot

这些值都应该重新标

因为它们其实代表的是：

- 脚后跟大概在哪里
- 前脚掌着力点大概在哪里
- 两点之间脚底长度是多少

标错会怎样

如果这些点放得不好，会直接影响后面全部结果：

放太靠前 / 靠后

会让脚的落地点判断错位，出现：

- 明明脚没落稳却判定接触
- 或明明已经踩住了却还在滑

左右不对称

会让左右脚 cleanup 效果不一致

heel-ball 距离不真实

会让后面的重投影不符合实际脚长
结果就是：

- 脚底被拉扯
 - ankle 目标位置不合理
 - 脚底朝向怪
 - 后续 knee / root 调整变得更困难
-

六、你可以把这四个骨骼理解成什么

最准确的理解是：

它们不是解剖骨骼，而是算法接触探针（contact probes）+ 脚底刚体几何锚点（rigid sole anchors）。

它们承担了三类职责：

1. 接触检测

从原始 mocap 的 heel / ball 速度判断何时 planted

2. 约束生成

进入 planted 后，为每帧生成稳定的地面约束位置

3. 足底几何保持

让 heel 和 ball 的距离在约束切换时保持不变，避免脚底形状乱掉

七、和论文 / 视频的上下游关系

这四个点在整条链路里的位置是：

```

1  Prepare Data
2    → 注入 Heel / Ball marker bones
3    → 从 original 动画里取它们的世界位置与速度
4    → 得到 left/right heel/ball contact timing
5    → 生成 constraint positions
6    → 反推出 ankle pose
7    → 再做 root / leg / smoothing

```

所以它们是整个算法的数据入口之一。

没有这四个点，后面那套“heel-only / ball-only / double-contact”的 cleanup 逻辑就无从谈起。Jerome 自己也说得很明确：加这些骨，是为了让 skeleton 满足论文所需的结构。

八、给你一个最终结论

这四个新增骨骼的“讲究”可以压缩成 5 句话：

1. **数量上**：每只脚必须至少有 heel 和 ball 两个接触点，才能表达足底支撑而不只是一个孤点。
2. **拓扑上**：必须挂在 `LeftFoot/RightFoot` 下面，因为它们是脚刚体上的固定局部点。
3. **旋转上**：用单位四元数，因为这里只关心位置采样，不需要独立朝向。
4. **数值上**：这些 offset 是当前角色脚底几何的手工标定，左右镜像、前后分布、同底平面、固定脚长。
5. **算法上**：它们既用于接触检测，也用于后续保持 heel-ball 距离，从而支撑整个 footskate cleanup。

你这一步一旦理解透了，后面 `extract_feet_contacts` 和 `contact_distance` 那段代码就会非常顺。下一条我可以继续直接接着讲：

为什么后面先算 `contact_distance`，以及它如何参与“第二个接触点进入约束时的重投影”。

`keep_translation(retargeting)`

▼ gemini

在动作捕捉数据处理和这段代码的上下文中，`keep_translation`（保留位移）参数的核心目的是控制是否将原始动捕文件（BVH）中各个骨骼节点的“绝对空间平移（位置）数据”应用到目标角色的骨骼上。

根据代码中两次加载的不同设置，它的具体目的分为以下两个截然不同的方面：

1. 设置为 `False` 的目的：进行标准动作重定向（人为复现滑步）

- **运行机制：**当 `keep_translation=False` 时，加载器会丢弃除了根节点（Root/Hips）之外所有子关节的空间位置数据，**仅仅将原始数据的“关节旋转角度（Rotations）”**映射给目标角色。
- **实际目的：**这是 3D 动画中“动作重定向”（Motion Retargeting）的标准做法。因为目标 3D 模型与原始动捕演员的体型、骨骼长度几乎总是不一致的。如果强行套用位置数据，会导致目标模型的四肢被生硬拉扯断裂或严重变形。只传递旋转数据，能完美保持目标角色自身的骨骼比例。
- **产生的后果：**由于放弃了原有的绝对位置，且骨骼长度发生了变化，脚部原有的空间几何约束被破坏，必然会导致脚部无法精确踩在原有的地平面上，从而产生滑步（Footskate）。这正是运行这套清理算法的先决条件。

2. 设置为 `True` 的目的：提取真实的物理约束数据（Ground Truth）

- **运行机制：**当 `keep_translation=True` 时，加载器会强制读取并应用动捕数据中每一个骨骼节点的绝对三维坐标。为了匹配这些绝对坐标，目标角色的骨骼长度会被强行拉伸或挤压（Squash and Stretch）。
- **实际目的：**作者加载这层数据完全不是为了视觉展示，而是为了**提取参考数据**。论文中的解析算法虽然能把脚钉在地上，但前提是它必须知道“脚应该在什么时候踩在什么位置”。保留了绝对平移数据后，作者就能通过计算脚跟（Heel）和脚前掌（Ball）在三维空间中的真实移动速度，准确判定出原始演员在哪几帧是完全踩实的。这些提取出的时间点和坐标，构成了整个滑步清理算法最核心的输入参数（约束条件）。

Find feet contacts timing

<https://chatgpt.com/g/g-p-69affc08e9088191811d69bef0b44dc1-mldeformer/c/69be7d27-a020-83a2-a348-c836dc2339d3>

<https://chatgpt.com/share/69bff041-7584-8009-b181-6252f863e398>

▼ gpt

好，我们把 `Find feet contacts timing` 这一段，按 **cell** → 行 → 作用 → 上下游 的方式彻底拆开。

这部分在整条 notebook 里的定位很明确：

它不是在修脚滑，而是在先生成“脚什么时候应该锁地”的时间标签。

Jerome 在视频里明确说，这篇 paper 本身只负责“已知 footplant 约束时，如何正确修正”，不负责检测什么时候该锁脚；所以他额外利用原始 mocap 的稳定性，用 heel / ball 的位置和速度来推 contact timing。

notebook 这一节的 markdown 也直接写了：用原始运动检查 heel 和 ball 何时低于某个速度阈值，并假设此时脚已 planted；然后再平滑一下信号，避免 1、2、3 帧这种很小的误判断开。
([GitHub](#))

一、这一部分一共有哪几个 cell

`Find feet contacts timing` 这块，实际可以分成 3 个 cell：

1. markdown 说明 cell

解释这一节想做什么：从原始动画里检测 planted timing，再把抖动信号平滑掉。(GitHub)

2. 真正计算 contacts 的 code cell

也就是这段：

Python

```

1  ogquats, ogpos = lab.utils.quat_fk(original.quats, original.pos, original.parents)
2  agquats, agpos = lab.utils.quat_fk(animation.quats, animation.pos, animation.parents)
3
4  contacts = np.zeros([frame_count, 4], dtype=np.bool_)
5  contacts[:, :2], contacts[:, 2:] = lab.utils.extract_feet_contacts(
6      ogpos, [left_heel, left_ball], [right_heel, right_ball], 0.04
7  )
8
9  # smooth out a little the signal ( we keep the signal on if it switches off for one or 2 frames )
10 for frame in range(2, frame_count-2):
11     for c in range(4):
12         contacts[frame, c] = contacts[frame, c] or (contacts[frame-2, c] and contacts[frame+2, c])
13
14 for frame in range(1, frame_count-1):
15     for c in range(4):
16         contacts[frame, c] = contacts[frame, c] or (contacts[frame-1, c] and contacts[frame+1, c])

```

这段代码在 raw notebook 里可以直接找到。(GitHub)

3. 可视化 code cell

把四路 contact signal 画出来，并用交互 viewer 显示“哪几个接触点此帧被判定为 planted”。对应代码是：

```

1  fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10,1))
2  ax1.plot(contacts[:, 0])
3  ax1.plot(contacts[:, 1])
4  ax2.plot(contacts[:, 2])
5  ax2.plot(contacts[:, 3])
6
7  def render(frame, ori):
8      ...
9      if contacts[frame, c]:
10         contacts_matrices[c, [0,2], 3] = ogpos[frame, contact_indices[c], [0,2]]
11         ...
12         ax1.set_xlim([frame-60, frame+60])
13         ax2.set_xlim([frame-60, frame+60])
14
15  interact(render, frame=lab.Timeline(max=frame_count-1), ori=widgets.Checkbox())

```

这段也能在 raw notebook 里看到。([GitHub](#))

二、先讲第一格 markdown：它在说什么

markdown 的意思很简单：

- 先用原始运动 **original**
- 去看 heel / ball 的速度
- 低于阈值就判定为 planted
- 然后把这个 on/off 信号再做一点平滑
- 防止只断开 1~3 帧的噪声

这和 Jerome 视频里的口述完全一致：

他会建一个 `frame_count x 4` 的大 buffer，四列分别是 `left heel / left ball / right heel / right ball`；然后用一个速度启发式去判断是否接触地面；阈值大约是 `0.04`；再把只掉 1~2 帧的小洞补上。

三、逐行讲解“计算 contacts”的那个 cell

第 1 行

```
Python |  
1  ogquats, ogpos = lab.utils.quat_fk(original.quats, original.pos, original.parents)
```

这行在干什么

对 `original` 这份原始动画做一次 FK (Forward Kinematics)，求出每一帧、每根骨头的全局旋转和全局位置。

为什么要做

因为动画文件里存的通常是：

- 局部旋转 `quats`
- 局部平移 `pos`
- 父子骨架关系 `parents`

但你要判断“脚跟是不是 planted”，看的不是局部空间，而是：

这个 heel / ball 在世界上到底动没动。

所以必须先把局部骨架变换沿父子链累积起来，得到世界空间位置。

输出是什么

- `ogquats` : original global quaternions
- `ogpos` : original global positions

这里的 `og` 可以理解成 original global。(GitHub)

第 2 行

```
1 agquats, agpos = lab.utils.quat_fk(animation.quats, animation.pos, animation.parents)
```

这行在干什么

对 `animation` 这份“会滑脚的目标动画”也做同样的 FK。

你会问：这里为什么也要算？

因为后面的可视化 cell 里，Jerome 会允许你在 viewer 里切换：

- 看 `original`
- 或者看 `animation`

这样你能直观看到：

- 原始运动里脚该锁的地方
- 以及 retarget 后版本实际已经滑到了哪里

重要一点

在“纯 contact 检测”逻辑里，真正拿来判定 planted 的是 `ogpos`，不是 `agpos`。

这一点跟 Jerome 视频口述完全一致：他用原始稳定数据去检测 contact，而不是用已经滑了的版本去反推。

第 4 行

```
1 contacts = np.zeros([frame_count, 4], dtype=np.bool_)
```

这行在干什么

创建一个大小为：

```
Python |  
1 [总帧数, 4]
```

的布尔数组。

4 列分别代表什么

按 notebook 前面注入骨点的顺序，这 4 列对应：

- 0 = left heel
- 1 = left ball
- 2 = right heel
- 3 = right ball

Jerome 在视频里也是这样描述的：一个大 buffer，四个 contact 通道，分别是 left heel / left ball / right heel / right ball。

为什么是布尔类型

因为这一层还不是约束位置，不是连续数值，而只是：

- 这一帧接触
- 还是不接触

所以布尔就够了。

第 5 行

```
Python |  
1 contacts[:, :2], contacts[:, 2:] = lab.utils.extract_feet_contacts(  
2     ogpos, [left_heel, left_ball], [right_heel, right_ball], 0.04  
3 )
```

这行是这整个 section 的核心。

先看左边赋值

```
Python |  
1 contacts[:, :2], contacts[:, 2:]
```

意思是：

- 前两列填左脚结果
- 后两列填右脚结果

也就是：

- `contacts[:, :2]` → 左 heel / 左 ball
- `contacts[:, 2:]` → 右 heel / 右 ball

再看函数输入

```
Python |  
1 ogpos
```

表示使用 **original global positions**。

这非常重要。因为你要从“更可信的原始 mocap”里提取 contact，而不是从已经滑脚的 **animation** 里提。Jerome 视频也是这么说的。

```
Python |  
1 [left_heel, left_ball], [right_heel, right_ball]
```

这里告诉 **extract_feet_contacts**：

- 左脚要检查哪两个点
- 右脚要检查哪两个点

为什么不是只给 **LeftFoot / RightFoot**？

因为脚底接触不是二元的“整只脚接触 / 不接触”，而是更细的：

- 后跟先着地
- 全脚掌压实

- 后跟离地只剩前掌
- 最后整个离地

论文里也把下肢足部接触分解成 heel / ball / toes 这种几何结构；Jerome notebook 是选了 heel + ball 作为接触检测的关键支点。

```
Python |  
1 0.04
```

这是速度阈值。

Jerome 视频里明确说：

- 如果某个 heel/ball 动得比 0.04 更快，就认为它在动
- 如果低于这个阈值，就认为它是 planted 的

所以你可以把 `extract_foot_contacts(...)` 直观理解成：

“检查 heel / ball 的世界空间速度，小于阈值就标 1，否则标 0。”

这一整行的最终作用

它把连续运动数据，变成了四路开关时序信号：

```
Python |  
1 contacts[frame, 0] # 左后跟是否接触  
2 contacts[frame, 1] # 左前掌是否接触  
3 contacts[frame, 2] # 右后跟是否接触  
4 contacts[frame, 3] # 右前掌是否接触
```

这就是后面全部 footskate cleanup 的“约束开关表”。

第 7 行注释

```
1 # smooth out a little the signal ( we keep the signal on if it switch off for one or 2 frames )
```

这句注释其实已经把后面两层循环的目的讲透了：

如果接触信号只短暂断掉 1 帧或 2 帧，那大概率不是脚真的离地了，而是检测噪声。

Jerome 视频里也明确说了同样的话：

他不希望 contact 段中间因为某一帧的误判突然熄灭，所以会做一个小 smoothing，让它至少形成连续几帧。

第一个 smoothing 循环

```
1 for frame in range(2, frame_count-2):
2     for c in range(4):
3         contacts[frame, c] = contacts[frame, c] or (contacts[frame-2, c] and contacts[frame+2, c])
```

先看循环范围

```
1 range(2, frame_count-2)
```

因为你要访问 `frame-2` 和 `frame+2`，所以前后各留 2 帧边界，避免越界。

再看内部逻辑

```
1 contacts[frame, c] = contacts[frame, c] or (contacts[frame-2, c] and contacts[frame+2, c])
```

这句翻成中文是：

如果当前帧本来已经是接触，那当然保留。

如果当前帧不是接触，但它前 2 帧和后 2 帧都是接触，那么也把当前帧补成接触。

这在修什么

它主要针对这种情况：

```
1 1 1 0 0 1 1
```

或者更一般地说，针对很短的 2 帧左右缺口。

直觉是什么

如果同一个 heel/ball 在较宽一点的时间尺度上看，两边都稳定 planted，那中间这几帧掉成 0，很可能只是速度阈值在边界上抖了一下。

所以这一层是一个“跨 2 帧的小洞填补”。

第二个 smoothing 循环

```
1 for frame in range(1, frame_count-1):
2     for c in range(4):
3         contacts[frame, c] = contacts[frame, c] or (contacts[frame-1,
c] and contacts[frame+1, c])
```

先看循环范围

这里访问的是 `frame-1` 和 `frame+1`，所以只需要前后各留 1 帧。

内部逻辑

```
Python |  
1 contacts[frame, c] = contacts[frame, c] or (contacts[frame-1, c] and  
  contacts[frame+1, c])
```

翻成中文就是：

如果当前帧不是接触，但它前一帧和后一帧都是接触，那就把它补成接触。

这在修什么

针对的是单帧洞：

```
Plain Text |  
1 1 1 1 0 1 1 1
```

为什么两层循环分开做

因为 Jerome 想处理的是：

- 先补比较明显的“2 帧小洞”
- 再补更细的“1 帧小洞”

这样最终 contact 段会更连续、更稳。

它不是一个严格数学意义上的滤波器，而是一个非常实用的**时序标签修补器**。

四、这一格 cell 最终输出了什么

跑完这格 cell 后，你并没有得到修好的动画。

你得到的是：

```
Python |  
1 contacts[frame, c]
```

这样一张 按帧的四路 planted 时间表。

这张表会进入后面的阶段：

- Constraints Positions
- Ankle position/orientation
- Root placement
- Leg IK
- Final Processing

也就是说，这格 cell 是后面五阶段算法的前置输入层，不是求解器本体。Jerome 视频里也是先讲 contact timing，再正式进入 paper 的五阶段。

五、逐行讲解“可视化 contacts”的那个 cell

接下来讲第二个 code cell，也就是把 `contacts` 看见的那一格。

第 1 行

```
1 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10,1))
```

作用

创建一个 Matplotlib 图窗，里面有上下两个子图：

- `ax1`：画左脚两路信号
- `ax2`：画右脚两路信号

为什么分两行

因为四路都堆一行会看不清。

分成：

- 上：left heel / left ball
- 下：right heel / right ball

更容易看相位关系。

第 2~5 行

```
1 ax1.plot(contacts[:, 0])
2 ax1.plot(contacts[:, 1])
3 ax2.plot(contacts[:, 2])
4 ax2.plot(contacts[:, 3])
```

作用

把四路布尔信号画成时间曲线。

虽然 `contacts` 是 bool，但画图时会被当成：

- `False = 0`
- `True = 1`

所以看到的是四条跳变的 on/off 曲线。

工程意义

这样你能直观看：

- 左 heel 比左 ball 更早接触还是更晚接触
- 右脚是不是和左脚交替
- contact 检测有没有很多毛刺
- smoothing 后是否更连续

这一步其实是在验证前一格 cell 的输出质量。

```
def render(frame, ori):
```

```
1 def render(frame, ori):
```

作用

定义一个交互渲染函数：

- `frame`：当前查看哪一帧
- `ori`：是看 `original`，还是看 `animation`

这是 notebook 常见写法，后面配合 `interact(...)` 可以做滑条和开关。

选择显示哪份动画

```
Python |  
1 anim = original if ori else animation
```

作用

如果勾选 `ori`，就显示原始参考动画；否则显示 retarget 后待修版本。

为什么这一步很重要

因为这能让你直接对比：

- 原始数据里脚底点什么时候稳定
- 而目标动画里对应的脚是不是已经滑开了

Jerome 在视频里就是通过这种对照，说明“接触信号来自原始数据，但当前要修的是滑脚版本”。

取当前帧局部变换

```
Python |  
1 p = (anim.pos[frame,...])  
2 q = (anim.quats[frame,...])
```

作用

拿出当前帧所有骨骼的：

- 位置 `p`
- 旋转 `q`

这是 viewer 画当前角色姿态所需的数据。

转成矩阵

```
Python |  
1 a = lab.utils.quat_to_mat(q, p)
```

作用

把四元数旋转和位置，转成可以直接喂给 viewer 的变换矩阵。

viewer 画骨架 / 网格时，更适合直接消费矩阵数组。

设阴影参考点

```
Python |  
1 viewer.set_shadow_poi(p[0])
```

作用

设置阴影关注点（point of interest），一般是用根节点附近位置。

这个不属于 contact 算法本身，主要是为了显示效果。

开始画角色

```
Python |  
1 viewer.begin_shadow()  
2 viewer.draw(character, a)  
3 viewer.end_shadow()  
4  
5 viewer.begin_display()  
6 viewer.draw_ground()  
7 viewer.draw(character, a)
```

作用

先画 shadow pass, 再画 ground 和 character 本体。

这部分也是纯显示。

构造 contact target 的变换矩阵

```
Python |  
1 contacts_matrices = np.eye(4, dtype=np.float32)[np.newaxis,...].repeat(4, axis=0)
```

作用

创建 4 个 4x4 单位矩阵。

为什么是 4 个

因为有四个接触点：

- left heel
- left ball
- right heel
- right ball

为什么先用单位矩阵

因为 `viewer.draw(target, contacts_matrices)` 需要的是一组实例变换矩阵。
单位矩阵可以作为默认“什么都不平移、什么都不旋转”的起点。

遍历四个 contact 点

```
Python |  
1 for c in range(4):
```

作用

逐个检查四个接触通道。

只显示当前处于 planted 的接触点

```
Python |  
1 if contacts[frame, c]:
```

作用

如果当前帧这个接触点为真，就把 `target` 放到地上显示出来；否则不动它。

含义

这表示：

“这一帧，我认为这个 heel / ball 应该锁地。”

把 target 放到 contact 的 XZ 位置

```
Python |  
1 contacts_matrices[c, [0,2], 3] = ogpos[frame, contact_indices[c], [0,2]]
```

这是这一格里最值得你注意的一句。

左边是什么意思

```
Python |  
1 contacts_matrices[c, [0,2], 3]
```

表示：

- 第 `c` 个 target 的变换矩阵
- 把第 4 列里的第 0、2 行，也就是 **X 和 Z 平移**
- 设成某个值

换句话说，就是只给 target 设置 **平面位置**。

右边是什么意思

```
Python |  
1 ogpos[frame, contact_indices[c], [0,2]]
```

表示：

- 当前帧
- 当前这个 heel/ball 骨点
- 取它的全局位置的 X 和 Z

而且注意，这里用的是 `ogpos`，不是 `agpos`。

也就是说，显示出来的 planted marker 是根据**原始动画的接触点位置**来的。(GitHub)

为什么只设置 XZ，不设置 Y

因为这个 target 是“地面上的接触标记”。

Jerome 想表达的是：

“脚该锁在地面平面上的哪个位置。”

所以他只取：

- X
- Z

而把 Y 留在地面高度附近。

这也和前面论文/视频的概念一致：接触最终是要“project to ground”。

画这些 target

```
Python |  
1 viewer.draw(target, contacts_matrices)
```

作用

把那 4 个小 target 画出来。

如果某个 contact 没激活，它就不会被移到对应接触点附近；

如果激活了，就会出现在地面上相应位置。

这就是你在 viewer 里看到的“当前帧哪些点被判成 planted”。

结束 display pass

```
Python |  
1 viewer.end_display()
```

纯渲染流程收尾。

画骨架辅助线和坐标轴

```
▼ Python |
1 viewer.disable(depth_test=True)
2
3 viewer.draw_axis(character.world_skeleton_xforms(a), 5)
4 viewer.draw_lines(character.world_skeleton_lines(a))
```

作用

把骨架的关节坐标轴和骨骼连线也画出来。

为什么要这样做

因为你现在不是只看 mesh 外观，而是在 debug：

- heel / ball 的骨点位置
- 当前整条骨架的姿态
- planted marker 和脚的相对关系

这对理解“为什么脚在滑”非常有帮助。

真正执行绘制命令

```
▼ Python |
1 viewer.execute_commands()
```

把前面排队好的绘制操作真正提交给 viewer。

让信号图跟着当前帧滚动

```
Python |
1 ax1.set_xlim([frame-60,frame+60])
2 ax2.set_xlim([frame-60,frame+60])
```

作用

让上面的 contact 折线图只显示当前帧前后大约 60 帧范围。

为什么这么做

不然如果整段动画几百帧全挤在一起，你根本看不出局部 timing。
这样你拖动 timeline 时，图也会跟着滚动，看局部相位关系非常清楚。

最后一段交互控件

```
Python |
1 interact(
2     render,
3     frame=lab.Timeline(max=frame_count-1),
4     ori = widgets.Checkbox())
5 viewer
```

作用

生成交互控件：

- 一个时间轴 `frame`
- 一个勾选框 `ori`

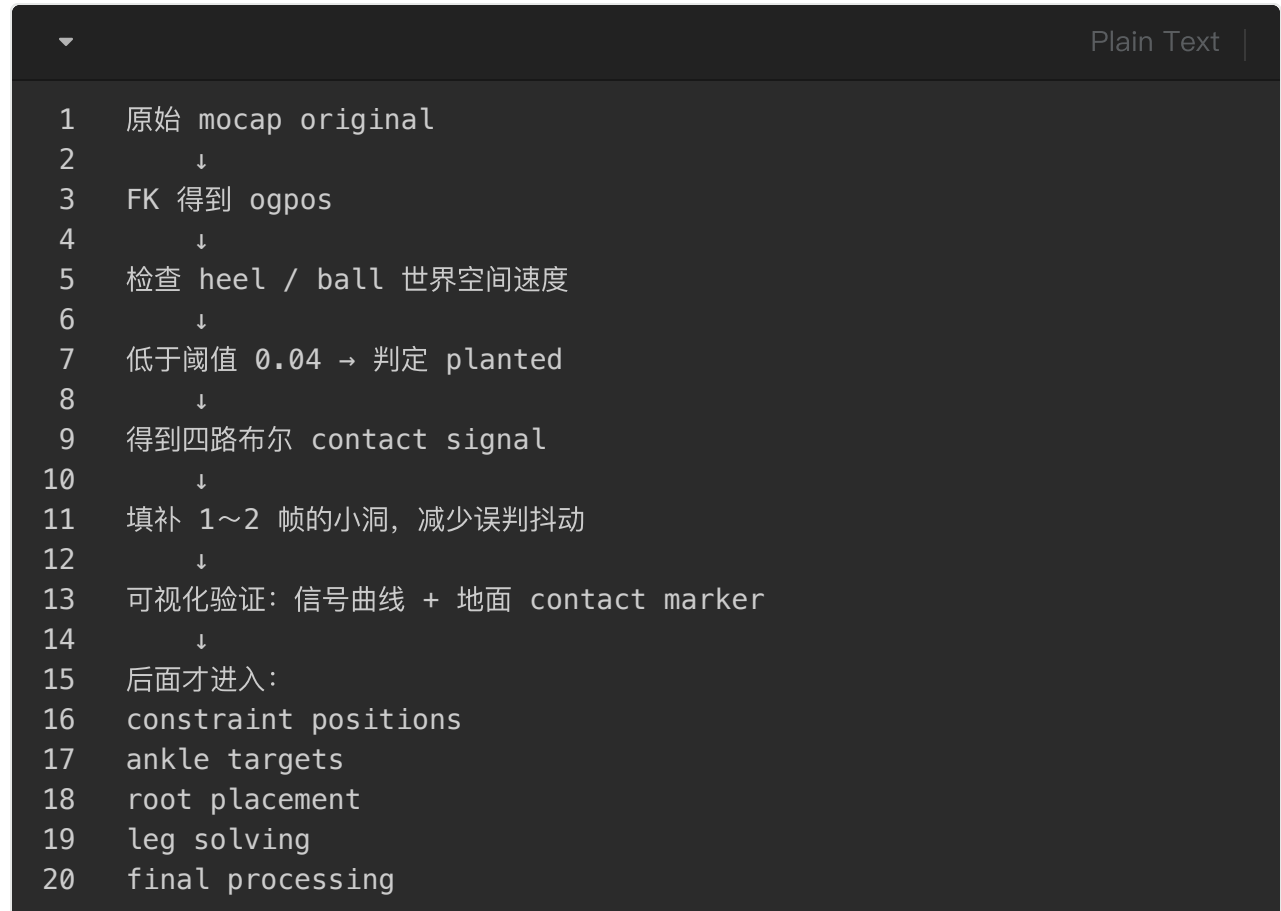
于是你就可以：

- 拖动帧号
- 切换 original / animation
- 同时观察角色、地面 planted 标记、contact 曲线

这格 cell 的本质就是一个可视化调试器。

六、把这两格 cell 放回整条算法链里

现在你可以把 `Find feet contacts timing` 这部分理解成：



所以这部分的真正身份是：

前置约束标签提取层 + 调试可视化层

而不是 footskate cleanup 求解器本身。Jerome 视频里也是先做这一层，再正式进入 paper 五阶段。

七、你最容易混淆的几个点

1) `contacts` 不是接触位置，只是接触时序

它只回答：

- 这帧左 heel 是不是 planted
- 这帧左 ball 是不是 planted
- ...

它还没回答“该锁在哪个地面坐标”。

2) 检测基于 **original**，不是 **animation**

这是为了避免“用错误结果反推正确标签”。

3) heel 和 ball 分开检测非常重要

因为后面 paper 需要区分：

- 单约束
 - 双约束
 - heel-only
 - ball-only
 - 过渡边界
-

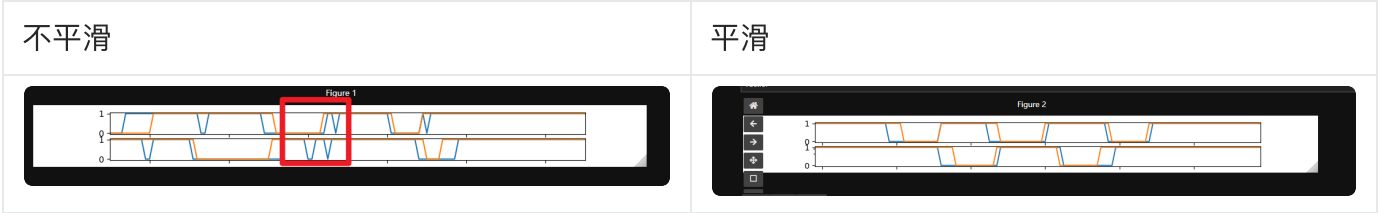
4) 这两层 smoothing 不是在平滑骨骼，而是在平滑布尔标签

很多初学者第一次看会误以为它在滤波姿态。

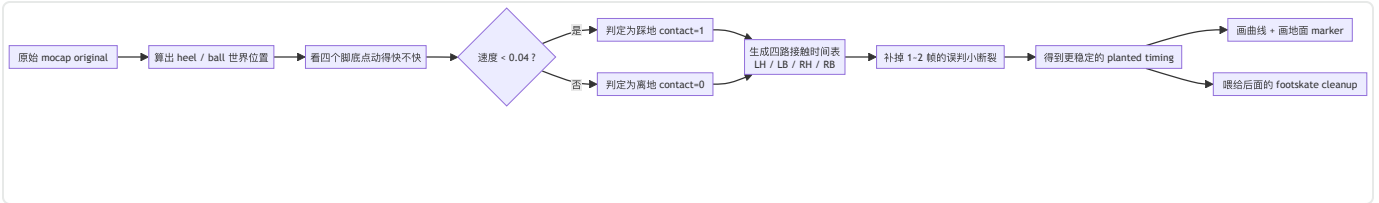
不是。它只是在修 **contacts** 这个 on/off 信号。

如果你愿意，下一条我可以继续按同样方式，逐行讲解下一节 **Constraints Positions** 的 **cells**，也就是从这四路 **contacts** 如何继续变成真正的地面约束位置。

smooth signal



数据流程图



Constraints Positions

<https://chatgpt.com/g/g-p-69affc08e9088191811d69bef0b44dc1-mldeformer/c/69c0f405-5fac-83a5-bdd4-53fb35dd1143>

<https://chatgpt.com/share/69bff041-7584-8009-b181-6252f863e398>

▼ GPT

可以，这一节我按你要的顺序来：

1. 先说这个步骤的目的
2. 再说总体输入和输出
3. 最后把它拆成几个部分，按“论文原意 → Jerome 语音稿 → notebook 代码在干什么”来逐段讲

你引用的这段英文，其实就是这一步最核心的摘要。论文原文也几乎是同一件事：

当处理第 **Fi** 帧时，如果某个受约束的 heel 或 ball 在前一帧 **Fi-1** 已经受约束，就直接沿用前一帧的约束位置；如果不是，就取接下来 **L1** 帧里的位置做平均，再投影到地面；如果同一只脚的另一个点已经有约束位置，还要把当前点重新摆到与它保持正确 heel-ball 距离的位置上。

Jerome 的语音稿也完全按这个逻辑解释，并补充说他在 notebook 里是整段动画一次性建一个大 buffer，而不是严格按论文那样流式逐帧推进。

一、这一步的目的是什么

这一步的目的，不是改骨骼，不是解 IK，也不是修 root。

它的目的只有一个：

把“这一帧这个 heel / ball 应该锁地”这种布尔信号，变成“它应该锁在地面的哪个具体位置”这种三维目标。

前一节 **Find feet contacts timing** 只回答了：

- 左 heel 这一帧是不是 planted
- 左 ball 是不是 planted
- 右 heel / 右 ball 是不是 planted

但它没有回答：

- 锁在哪里

而 **Constraints Positions** 就是在做这个空间化。

论文把它列成五阶段算法的第一步：先为每个 plant constraint 决定一个位置，后面 ankle、root、leg 全都依赖这个结果。

你可以把它理解成：

- `contacts` 给的是“什么时候锁”
 - `constraint positions` 给的是“锁到哪里”
-

二、这一步的总体输入和输出是什么

输入

这一节实际依赖 4 类输入。

1) `contacts`

上一节生成的四路接触时序：

- left heel
- left ball
- right heel
- right ball

它告诉你某一帧某个点是否该锁地。

2) 原始动画里 heel / ball 的世界空间位置

Jerome 明确说他这里还是基于原始稳定 mocap 去取这些点的位置，因为 paper 只负责“给定约束后正确修正”，不负责“自动判断什么时候锁脚”。所以约束位置这一步也建立在原始参考运动之上。

3) 窗口长度 `L1`

这是向前看多少帧做平均的窗口。

Jerome 在语音稿里说他这里用了 `L1 = 10` 帧。

4) heel-ball 的固定几何距离

因为脚底被当成一个刚体的一部分，所以同一只脚的 heel 和 ball 不能被独立锁到两个互相矛盾的位置。

论文明确说：如果同脚另一点已经有已知位置，就把当前点挪到“与另一点距离正确”的地面位置上。

输出

输出是一个 **constraint position buffer**，也就是：

- 对每一帧
- 对 4 个接触点
- 都给一个地面目标位置

Jerome 在语音稿里说得很具体：他会建一个大 buffer，维度是整段动画长度 × 四个点 × XYZ 空间坐标。

所以这一节结束后，你得到的不是姿态，而是一张“地面目标点时间表”：

```
▼ Plain Text |
1  frame i:
2    left heel  -> 地面点 A
3    left ball  -> 地面点 B
4    right heel -> 地面点 C
5    right ball -> 地面点 D
```

后面 **Ankle Position and Orientation** 才会把这些点翻译成脚踝位姿。

三、把这一步拆成 5 个部分来看

我建议你吧 notebook 这一节拆成下面 5 段来看。这样比死记代码行数更容易。

第 1 部分：创建约束位置缓冲区

这一部分代码的职责很简单：

- 定义 **L1**
- 创建一个三维 buffer

- 准备开始为四个接触点逐帧填目标位置

这一段为什么存在

因为 Jerome 这里不是在线逐帧流式处理，而是**离线地把整段动画一次性算完**。

他自己在语音稿里说了，paper 可以 frame-by-frame 做，也可以维护合适大小的缓冲；但在 notebook 里，最容易实现的是“整段 animation 一次性算一个大 buffer”。

这段在工程上的意义

这代表 Jerome 的 notebook 是一个**教学/离线版实现**：

- 先把整条约束时间线算好
- 再可视化
- 再喂给后续 ankle 阶段

而不是严格意义上的在线 runtime solver。

第 2 部分：按帧、按四个点遍历

这部分逻辑本质是：

- 对动画的每一帧
- 对 left heel / left ball / right heel / right ball
- 一个个检查

它为什么必须双层循环

因为约束位置不是“整只左脚一个点、整只右脚一个点”，而是更细的四路信号。

这背后是论文的脚步模型假设：

lower body 里，foot 是通过 ankle / heel / ball / toes 来描述的，而不是把整个 foot 粗暴当成一个质点。

工程意义

只有拆成 heel 和 ball 两个点，你后面才区分得出：

- heel-only
- ball-only
- heel+ball 双约束

这直接关系到下一节怎么求 ankle。

第 3 部分：如果前一帧已经 constrained，就直接沿用上一帧位置

这一段就是你引用英文里的第一句：

If it was, we keep the same constraint.

论文原文也是一模一样的意思：

如果当前帧 F_i 中受约束的某个 joint，在前一帧 F_{i-1} 也受约束，那么当前帧的约束位置与前一帧相同。

Jerome 在语音稿里也说得很直白：

- 如果这一帧说“我在 constraint”
- 并且上一帧已经在 constraint
- 那平均值已经算过了
- 所以直接沿用上一帧那个 constraint。

这段代码真正想保住什么

它想保住的是：

一个连续 plant 段内部，约束点不要乱跳。

因为 plant 的物理含义本来就是：

- 这段时间脚底这个点被“钉”在地上
- 所以地面目标点应该恒定

初学者最容易误解的地方

这里保持不变的是：

- 约束目标点

不是：

- 原始骨点位置
- 也不是最终 ankle 位置

原始 mocap 里 heel / ball 可能还有毫米级抖动；

但现在我们决定“既然这段时间它要锁地”，那 target 就不该跟着抖。

第 4 部分：如果这是新进入的一段 constraint，就向前看 L1 帧做平均并落地

这就是你引用英文里的第二句：

If it was not, we average the position over the next L1 number of frames and project it on the ground.

论文原文同样是这样写的：

否则，将 joint J 在接下来 L1 帧中的位置做平均，再把结果投影到地面上得到 cJ。

Jerome 的语音稿也完整解释了这个过程：

- 发现 heel 应该静止
- 向前看 L1 帧
- 只要这些帧也仍然在 constraint
- 就收集这些位置
- 求平均
- 然后 drop it on the ground

这一步为什么不是“直接取当前帧”

因为接触开始的那几帧里，原始骨点位置可能仍然有微小过渡、轻微抖动或者检测误差。

如果你直接取当前帧：

- 锁地点会偏噪
- 后面 ankle、leg、root 都会跟着偏

而取一个小窗口平均值，相当于在说：

“我不要这个点一进入 constraint 就立刻相信单帧数据，我要看这一小段接触整体稳定落在什么地方。”

为什么还要投影到地面

因为你要的不是原始三维骨点空间位置，而是：

脚底接触在地面上的约束点

所以要把这个平均值“压到地上”。

notebook 的 markdown 直接就用了 *project it on the ground*。

Jerome 口述里则说 *drop it on the ground*。

第 5 部分：如果同脚的另一一点已经 constrained，就重投影以保持 heel-ball 正确距离

这就是你引用英文里的第三句：

If the other part of the foot was already constrained ... we realign the current so the distance between heel and ball is correct.

论文原文说得很规范：

由于 foot 被视为 rigid piece，如果同脚另一 joint J' 也受约束且其位置 cJ' 已知，那么当前 joint J 的约束位置 cJ 必须被重定位到地面上与 cJ' 保持正确距离的最近点。

Jerome 的语音稿把这个过程讲得更直观：

- 如果 heel 先算好了
- ball 在这一帧第一次进入 constraint
- 那 ball 先落地以后，还得再挪一下
- 以保持 heel-ball 的距离不变
- 反过来也一样：如果 ball 先锁住，后来 heel 才锁住，就轮到 heel 去对齐这个距离。

这一步为什么非常重要

因为如果你不做这个“重对齐”：

- heel 锁在地上某点
- ball 也各自独立锁在另一个平均点

- 但两者之间距离和真实脚长不一致

那你下一节在求 ankle 时，就会得到一个自相矛盾的底刚体：

- 要同时满足 heel 在 H
- ball 在 B
- 但 $|H-B|$ 根本不是这只脚原本的长度

这种矛盾会直接传到 ankle solver，最后造成奇怪的底扭曲或 impossible target。

直觉上这一步在干什么

它其实是在加一句额外规则：

“虽然 heel / ball 都是单独检测出来的，
但它们属于同一只脚，不能彼此独立撒野。”

四、如果把 notebook 代码翻译成伪代码，大概就是这样

把 Jerome 这一节的实现压成伪代码，大概就是：

```

1  L1 = 10
2  constraint_pos_buffer = zeros(frame_count, 4, 3)
3
4  for frame in all_frames:
5      for contact in [LH, LB, RH, RB]:
6
7          if not contacts[frame, contact]:
8              continue
9
10         if frame > 0 and contacts[frame-1, contact]:
11             # 已经在上一帧锁住 -> 直接继承
12             constraint_pos_buffer[frame, contact] = constraint_pos_b
13             uffer[frame-1, contact]
14
15         else:
16             # 新开始的一段 contact -> 向前看 L1 帧
17             positions = collect_future_positions_while_contact_is_ac
18             tive(...)
19             c = average(positions)
20             c = project_to_ground(c)
21
22             # 如果同脚另一点已有约束位置 -> 对齐 heel-ball 固定距离
23             if paired_contact_already_constrained(...):
24                 c = realign_to_keep_heel_ball_distance(c, other_c)
25
26             constraint_pos_buffer[frame, contact] = c

```

这不是逐字符抄 notebook，而是把：

- 论文的 4.1 节
- Jerome 的语音解释
- 以及 notebook 在这一节的实现意图

重新还原成最清楚的工程逻辑。依据都在你这次给出的文字和上传文件里。

五、这一节做完以后，你应该看到什么结果

Jerome 在语音稿里说，算完这一节以后再去显示，就会看到：

- 这些 target 不再只是脚底当帧的位置

- 而是“一小段时间平均后”的稳定目标
- 并且 heel 和 ball 之间的距离始终被保持。

也就是说，这一节的结果应该满足 3 个性质：

1) 稳定

同一段 plant 里目标点不会一帧一跳。

2) 贴地

目标点落在地面上。

3) 几何一致

同一只脚的 heel 和 ball 距离合理。

只要这 3 件事成立，下一节 `Ankle Position and Orientation` 才有一个物理上讲得通的输入。

六、把它和下一节的关系说透

这一步结束后，你手里有的是：

- 脚底约束点
- 还不是脚踝位姿

下一节要做的是：

根据 heel / ball 的约束点，反推 ankle 的 target position 和 target orientation。

所以关系是：

```
▼ Plain Text |
1  contacts timing
2      ↓
3  constraint positions
4      ↓
5  ankle position / orientation
6      ↓
7  root placement
8      ↓
9  leg solving
10     ↓
11  final smoothing
```

论文五阶段也是这个顺序。

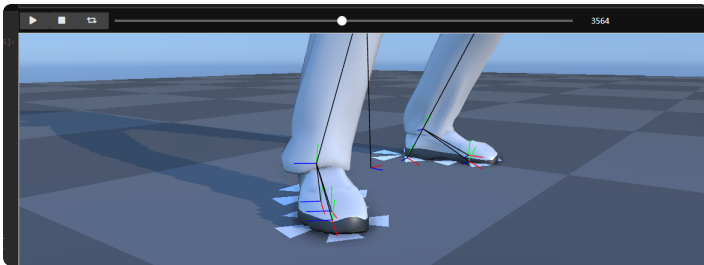
七、最后给你一句最直观的浓缩版

Constraints Positions 这一节，本质上是在做：

把“这一帧脚底某个点该锁地”这件事，变成“它应该锁在地面的这个具体位置”，并保证这个位置在时间上稳定、在地面上合理、在脚底几何上自洽。

你要是愿意，我下一条可以继续沿着这个结果，把 **Ankle Position and Orientation** 重新按同样格式讲一遍，但这次我会严格围绕“它如何消费 constraint position buffer”来讲，这样上下游会更顺。

Constraints Positions



Ankle Position and Orientation

<https://chatgpt.com/g/g-p-69affc08e9088191811d69bef0b44dc1-mldeformer/c/69c1082f-06e4-83a5-98cf-da8ffec297f9>

<https://chatgpt.com/share/69c1123d-8124-8009-99e1-7444dd4c74e1>

总览

▼ GPT

好，这一节我按你要的方式来：

先讲 **这一步的目的**，再讲 **总体输入 / 输出**，然后把 **Ankle Position and Orientation** 拆成几个部分，按“代码在做什么 → 对应论文哪一句 → Jerome 口头是怎么解释的”来讲。

先说结论：

这一步的本质，是把上一节得到的“脚底接触点该锁在哪”翻译成“脚踝（ankle）这一帧应该处于什么目标位姿”。

也就是：从 heel / ball 的地面约束，反推出 ankle 的目标全局朝向和目标全局位置修正，并且让这些修正在时间上连续，不要在 single ↔ double constraint 切换时突然 pop。论文把这定义为第 2 阶段：为每个受约束帧选择满足约束的 ankle 全局位置和朝向，并要求这些调整连续变化。Jerome 在语音稿里也用几乎同样的话说：双约束时可以精确算 ankle 的 orientation 和 position；单约束时只移动 ankle 的位置、不主动重定向；真正难点是单双约束切换，因此他保存的是相对于原动画的 **offset**，再把这个 offset 往前后传播和平滑。

一、这一步的目的是什么

这一节不是在做腿部 IK，也不是在挪 root。

它属于整条五阶段链路中的**第二步**：

1. 先求 **constraint positions**
2. 再求 **ankle position and orientation**
3. 再求 **root placement**
4. 再用 IK 让腿去满足 ankle 目标
5. 最后做开关边界的平滑 dissipation。

所以这里解决的是一个非常明确的问题：

上一节已经告诉我 heel / ball 该锁在哪个地面点了，那这一帧整个“脚”应该怎么摆，尤其是 ankle 应该怎么摆？

论文原文把这一节说得很清楚：

- 它要为当前帧选择 ankle 的**全局位置和全局朝向**
- 这些位置和朝向要满足上一阶段求出的约束位置
- 同时这些 ankle 调整必须是连续的，不能一帧一帧突然跳。

Jerome 的口述版本更直观：

- double constraint：这只脚 fully constrained，可以算出 ankle 的 orientation 和 position
 - single constraint：只移动 ankle 的位置，不重新定向
 - 但 single → double 或 double → single 直接切，会 pop
 - 所以保存的是相对原动画的 **offset**，不是绝对目标值。
-

二、总体输入是什么

这一节开始时，前面已经有三类关键输入了。

1) 上一节算出的 `constraint_positions_buffer`

这是最核心的输入。

它给出每一帧里：

- left heel 应该锁在哪
- left ball 应该锁在哪
- right heel 应该锁在哪
- right ball 应该锁在哪

Jerome 在前一段语音里已经说明，他先整段算出一个 `constraint position buffer`，里面保存所有帧、四个接触点的目标位置。

2) `contacts`

也就是四路接触时序：

- 左 heel
- 左 ball
- 右 heel
- 右 ball

这一节要靠它来判断当前帧属于哪种情况：

- double constrained
- single constrained
- unconstrained

论文就用这三种分类来定义 ankle 的处理逻辑。

3) 原始 ankle 的当前状态

也就是原动画中这一帧 ankle 本来在哪里、朝哪转。

Jerome 在语音稿里说得很明确：

他不是记“新的绝对 ankle 位姿”，而是记“相对原动画我想施加的 correction offset”。

从 notebook 后面的可视化 cell 也能看出来，最终显示 ankle 目标时，用的是：

- 原始的 `agquats[frame, foot]`
- 再乘上 `ankle_rotation_buffer[frame, foot]`
- 原始的 `agpos[frame, foot]`
- 再加上 `ankle_position_buffer[frame, foot]`。(GitHub)

这说明这一步的内部表示就是：

- rotation offset
- position offset

而不是直接覆盖原姿态。

三、总体输出是什么

这一步的输出有两个核心 buffer。

1) `ankle_rotation_buffer`

保存每一帧、每只脚的 ankle 旋转偏移。

后面的 render cell 直接这么用：

- `lab.utils.quat_mul(ankle_rotation_buffer[frame, 0, :], agquats[frame, LeftFoot])`
- `lab.utils.quat_mul(ankle_rotation_buffer[frame, 1, :], agquats[frame, RightFoot])`

也就是说，buffer 里存的是要乘到原始脚踝朝向上的修正四元数。(GitHub)

2) ankle_position_buffer

保存每一帧、每只脚的 ankle 位置偏移。

render cell 直接这么用：

- `agpos[...] + ankle_position_buffer[...]`

说明它存的是“相对原动画 ankle 位置，要额外平移多少”。(GitHub)

四、把这一节拆成几个部分来看

我建议你吧 `Ankle Position and Orientation` 理解成 5 个小模块：

- 1 A. 先建两个 ankle buffer
- 2 B. 先处理 double constraint
- 3 C. 再处理 single constraint 的基础位置偏移
- 4 D. 再做 backward / forward 扫描，把 double constraint 的旋转 offset 往邻居帧传播
- 5 E. 如果两边都命中，再交叉 blend，得到最终 ankle offset

这 5 段就是 notebook 这一格在做的事。

五、Part A：先建 buffer，这一步在存什么

这一段通常就是类似：

```
1 L2 = ...
2 ankle_rotation_buffer = ...
3 ankle_position_buffer = ...
```

虽然 raw notebook 在网页里被压成少数超长行，这一格代码本体没法像普通 `.py` 那样完整逐字符展开，但从 Jerome 的语音稿和后面 render 对这两个 buffer 的消费方式，可以非常确定它就在做这件事。Jerome 直接说了：

- “I will have my buffer for the ankle rotation”
- “and the buffer for the ankle position”
- “for both left and right”

L2 是什么

`L2` 是 ankle 阶段的邻域窗口长度。

论文原文明确说：

这一阶段假设在 `Fi-L2` 到 `Fi+L2` 的范围内，constraint positions 都已知。

它的工程含义就是：

- 当前帧的 ankle 修正，不只看当前帧
- 还会看前后附近若干帧
- 用来把单双约束切换时的变化摊平

为什么 buffer 存的是 offset，不是绝对值

这是 Jerome 这版 notebook 的灵魂。

他说得非常明确：

不要把“脚最后该在哪”记成绝对位姿；
要记成“相对原动画差了多少”，也就是 offset；
然后把这个 offset 往前后传播、慢慢衰减。

这就是为什么后面 render 里是：

- 原始 ankle quaternion × `ankle_rotation_buffer`

- 原始 ankle position + `ankle_position_buffer`。(GitHub)

六、Part B：先处理 double constraint

这部分是最“干净”的部分，因为几何上最明确。

论文定义：

- 如果 heel 和 ball 都 constrained
- 那么这个 ankle 是 **doubly constrained**。

Jerome 也说：

“the simplest case”

“I have a fully constrained foot”

“I can compute the ankle orientation and the position.”

这一段代码在逻辑上做什么

可以把它理解成两步。

第一步：先求目标朝向

论文给出的公式是：

$$Q'_A = \text{Rot}(p_B - p_H, c_B - c_H) * Q_A$$

[

$Q'_A = \text{Rot}(p_B - p_H, c_B - c_H) * Q_A$

]

意思是：

- 原始 heel→ball 的方向是 `pB - pH`
- 目标 heel→ball 的方向是 `cB - cH`
- 先算一个最短旋转，把原始足底轴对齐到目标足底轴
- 然后把这个旋转乘到原始 ankle 全局朝向 `QA` 上，得到新 ankle 的全局朝向。

第二步：再求目标位置

论文接着给出：

$$p'_A = c_H - Q'_A(o_H)$$

```
[  
p'_A = c_H - Q'_A(o_H)  
]
```

也就是：

- 已知 heel 最终必须落在 `cH`
- 已知 heel 相对 ankle 的局部偏移 `oH`
- 又已知新的 ankle 朝向是 `Q'A`
- 所以就能反推出 ankle 的全局位置。

notebook 里这一段保存的不是绝对位姿

Jerome 的实现不是把 `Q'A` 和 `p'A` 原样存下来，而是把它们转成：

- 相对原动画的 rotation offset
- 相对原动画的 position offset

这样做的原因只有一个：

后面 single ↔ double 切换时，offset 更容易平滑传播。

七、Part C：single constraint 的基础处理

当只有 heel 或只有 ball 被锁住时，逻辑就变了。

论文说：

- single constrained ankle：只有一个子点被约束
- 给定任何 ankle 朝向，都能通过平移 ankle 让那个点落到目标位置
- 所以最自然的做法是：保留原来朝向，只做平移。

Jerome 在语音稿里的说法几乎一模一样：

“you only have to do is not compute any reorientation of the ankle”
“you only move the position.”

这段代码逐行在干什么

逻辑上等价于：

```
1 if single_constraint:
2     rotation_offset = identity
3     position_offset = cJ - pJ
```

其中：

- `J` 是当前被锁的那个点，可能是 heel，也可能是 ball
- `pJ` 是它当前在原动画里的全局位置
- `cJ` 是上一节求出来的约束地面点

所以：

- rotation buffer 这里基本不加新旋转
 - position buffer 只存一个平移量，把 J 拉到 cJ
-

为什么不能到这里就结束

因为如果你只做这一步，会在下面这种切换时爆出 pop：

- 当前帧：single constraint，只做平移
- 下一帧：double constraint，突然又多出一个旋转修正

Jerome 在语音稿里专门说了：

如果你这样直接从 single 切到 double，
“you will pop in that right position instantly”。

所以 single constraint 这一步只是**基础偏移**，还不够。

八、Part D: backward scan, 检查“前面是不是 double”

这是这一节最关键的平滑逻辑之一。

Jerome 的话很明确：

- 如果当前帧是 single constraint
 - 就往回看
 - 看看是不是从一个 double constraint 过来的
 - 如果是，就把那边的 rotation offset 慢慢衰减传播回来。
-

代码本质在干什么

可以抽象成：

```
1  for f in range(L2):
2      if frame-f 是最近一个 double-constrained 帧:
3          alpha = ...
4          取那一帧的 rotation offset / position offset
5          乘上 alpha
6          混到当前帧
7          break
```

为什么先传播旋转，再传播位置

Jerome 口头解释是：

“first thing you do is that you rotate the ankle back in time
and then you move the foot.”

这背后的几何直觉是：

- double constraint 里，脚的完整朝向已经被重新定了
 - single constraint 如果完全不带这点旋转记忆，会突然丢失足底姿态
 - 所以先把“来自 double frame 的那一点朝向修正”慢慢往单约束段里退火
-

alpha 在做什么

这里用的不是纯线性权重，而是 paper 里的 cubic ease-in / ease-out：

$$\alpha(t) = 2t^3 - 3t^2 + 1$$

```
[  
\alpha(t)=2t^{3-3t}2+1  
]
```

Jerome 在语音稿里特地强调：

- “they don't do purely linear correction”
- “they do ease in / ease out correction”
- “the formula for alpha comes from the paper.”

所以这段代码里的 `alpha` 作用就是：

距离最近的 double frame 越近，继承的 offset 越多；
越远，继承得越少；
并且衰减曲线是平滑的，不会硬折线。

九、Part E：forward scan，检查“后面是不是会进入 double”

反方向同理。

Jerome 也说了：

- 如果当前 single，往前看
- 如果未来不远处会进入 double constraint
- 那也要提前吃进去一点将来的 offset
- 这样 single → double 不会在切换帧瞬间爆出来。

代码本质在做什么

```
1 for f in range(L2):
2     if frame+f 是最近一个 future double-constrained 帧:
3         alpha = ...
4         取 future 的 offset
5         乘上 alpha
6         形成一个 forward contribution
7         break
```

你可以把它理解成“预热”

不是等脚真的进入 fully constrained 再一下子改脚踝，

而是：

- 前面几帧先偷偷改一点点
- 越接近真正的 double frame，改得越多

十、Part F：如果前后都命中，就 crossover 两边结果

这个是论文 Figure 3 的第三种情况，也是 Jerome 口头提到的“weird case”。

论文说：

- 当两边都存在要 blend off 的 adjustment
- 就需要把 forward 和 backward 两个修正再 blend 一次。

Jerome 的原话是：

- “if you have that weird case when you actually go from double constraint back double constraint”
- “you need to cross over that information.”

这段代码在做什么

逻辑上大概是：

```
1 ▾ if backward_hit and forward_hit:
2     local_quat = slerp(back_quat, front_quat, alpha_cross)
3     local_pos = lerp(back_pos, front_pos, alpha_cross)
4 ▾ elif only_backward_hit:
5     用 backward
6 ▾ elif only_forward_hit:
7     用 forward
8 ▾ else:
9     用当前基础 single result
```

为什么 rotation 用 slerp

因为 rotation 是 quaternion，不能线性插值。

论文在 Section 4.2 / 4.5 都是用 quaternion blending 的思想处理旋转平滑。

为什么 position 用普通线性混合

因为 position offset 是 3D 向量，直接 lerp 就行。

Jerome 在口头里虽然主要强调的是 rotation 的“slerp that stuff”，但整体逻辑就是：

- rotation：球面插值
- translation：线性插值

十一、这一节结束后，buffer 里到底是什么

到这一节结束时，你已经有了：

ankle_rotation_buffer

每帧每只脚一个 quaternion correction。

ankle_position_buffer

每帧每只脚一个 translation correction。

而且它们不是“生硬的约束解”，而是已经过了：

- double constraint 直接求解
- single constraint 基础平移
- backward blend
- forward blend
- crossover blend

之后的平滑 ankle offset。

Jerome 在语音稿里也说：

算完之后，你看到的 buffer 里那些位置，就是“我想让 ankle 去的地方”。

后面的 render cell 正是拿这两个 buffer 去画 ankle 坐标轴：

- 左脚： `ankle_rotation_buffer[frame,0]` 和 `ankle_position_buffer[frame,0]`
- 右脚： `ankle_rotation_buffer[frame,1]` 和 `ankle_position_buffer[frame,1]` 。 ([GitHub](#))

十二、把这一步浓缩成一句最直观的话

你可以把 `Ankle Position and Orientation` 理解成：

先把 heel / ball 的地面约束点，翻译成 ankle 的目标位姿；
双约束时，脚是 fully constrained，直接反推 ankle 的位置和朝向；
单约束时，只保留原朝向，先做位置修正；
然后再把 double constraint 产生的旋转 / 平移 offset 往相邻 single 段前后传播，
用一个 cubic alpha 做平滑衰减，
这样单双约束切换时 ankle 不会突然 pop。

十三、你学习这一节时最该盯住的 6 个关键词

你之后回看 notebook，只要盯住这 6 个词，这节就不会散：

1) `constraint_positions_buffer`

上一节给的 heel / ball 目标地面点。

2) double constrained

heel 和 ball 都锁，能直接求 ankle 朝向 + 位置。

3) single constrained

只锁一个点，默认不改朝向，只做平移。

4) offset

保存相对原动画的 correction，而不是绝对位姿。

5) L2

前后看多少帧，决定平滑影响范围。

6) alpha

不是线性衰减，而是 cubic ease-in / ease-out。

十四、和你未来 UE / GASPALS / Shadow Mode 的关系

这一节如果放到你未来的 GASPALS / Shadow Mode 里，它非常像一个独立可替换的小模块：

它属于哪一层

它属于：

- 接触约束 → 脚踝目标位姿生成层

在动画链里，它的位置大概是：

- Motion Matching / Base Locomotion 输出之后
- Leg IK / FBlK 之前

它吃什么

- foot contact timing
- contact positions
- 当前原始脚踝姿态

它吐什么

- left/right ankle target offsets

为什么适合 Shadow Mode

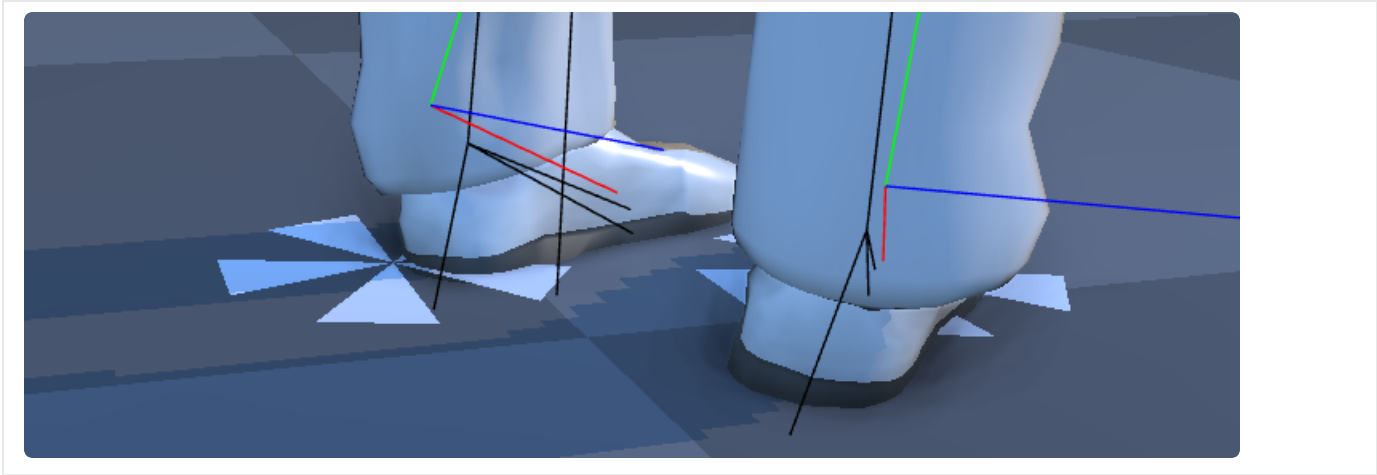
因为它边界特别干净：

- 输入明确
- 输出明确
- 很容易 A/B 对比

你以后无论是保留规则法，还是想用 LMM / Control Operators 去学一个更聪明的 foot target layer，都可以把这一层当成非常清楚的接管点。GASPALS 的运动系统本质上也是“输入意图 → base pose → 再叠加修正”的架构，适合把这种 ankle target 逻辑挂在 locomotion 主干和后续 IK / post-process 之间。

下一条我可以继续用同样的粒度，给你讲 **Root Placement**，也就是这一步算完 ankle target 之后，为什么还要先挪 hips/root，而不是直接开始掰腿。

single constrain：单脚一个target



反向传播，信号均摊

▼ GPT(反向传播；均摊)

这里的“反向传播”不是机器学习里那个 **backpropagation** / 梯度反传。

在 **Ankle Position and Orientation** 这一步里，它的意思是：

把某个已经算出来的 ankle 修正 offset，沿着时间轴往更早的帧“倒着扩散”一点。

也可以叫：

- 往回传播
- 向过去帧扩散
- back in time
- 把 future/past 的 double-constraint 修正提前或延后摊开

一、先用最直观的话解释

假设你现在有这样一段接触状态：

▼	Plain Text						
1	帧	10	11	12	13	14	15
2		S	S	S	S	S	D

其中：

- **S** = single constraint
- **D** = double constraint

到了第 15 帧，脚突然从 single 变成 double。

double constraint 会比 single 多出一个“**脚踝旋转修正**”，因为这时 heel 和 ball 都锁住了，脚底方向被更完整地确定了。

如果你只在第 15 帧才突然把这段旋转修正加上去，会发生什么？

角色脚踝会在第 14 → 15 帧突然“啪”地转一下。

这就是 pop。

所以算法会说：

不要等到第 15 帧才突然全部加上。

把第 15 帧那段旋转 correction，提前分一点给第 14 帧，再少一点给第 13 帧，再少一点给第 12 帧.....

这件事，就是你说的“反向传播”。

也就是：

```
1  15帧的修正
2  ↓
3  往前摊给 14、13、12...
```

所以它本质上是：

把将来会出现的脚踝修正，提前在更早的帧里预热一点。

二、为什么要这样做

因为 `single constraint` 和 `double constraint` 的 ankle 解法不一样：

- **single**：通常只做位置修正，不主动重定向 ankle
- **double**：会解出完整的 ankle 朝向 + 位置

所以如果你不做传播，状态切换时会出现：

```
1  前一帧：只有 position correction
2  后一帧：突然多出 rotation correction
```

这就会造成非常明显的脚踝跳变。

所以算法的核心思路是：

把“突然出现的修正”变成“提前慢慢长出来的修正”。

三、它在代码里对应哪一段

你补齐的 `compute_single_constrained_ankle()` 里，其实有两段相关逻辑：

1) backward scan

也就是你看到的这种：

```
Python |  
1 ▾ for f in range(L2):  
2 ▾     if frame-f >= 0 and contacts[frame-f, heel] and contacts[frame-f,  
    ball]:  
3     ...
```

这段是在干什么？

它是在看：

当前 frame 是 single constraint 时，往前找，看看过去最近有没有 double-constrained 帧。

如果找到了过去某一帧 double constraint，说明：

- 之前脚踝曾经有一套“完整约束下算出来的 rotation offset”
- 那么当前 single 帧不应该完全丢掉这套信息
- 应该让它向当前帧慢慢淡出

这个方向更准确地说，是：

从过去的 double 帧，把修正延续到现在。

也就是“向前延续”。

2) forward scan

也就是：

```
Python |  
1 ▾ for f in range(L2):  
2 ▾     if frame+f < frame_count and contacts[frame+f, heel] and contacts  
    [frame+f, ball]:  
3     ...
```

这段是在看：

当前 frame 是 single constraint 时，往后找，看看未来最近会不会进入 double constraint。

如果未来某一帧会进入 double constraint，说明：

- 将来会出现一段新的 rotation offset
- 那么当前帧不应该等到那一帧再突然吃满
- 而应该把未来的那套修正，往当前帧提前摊一点

这个才更符合你说的“反向传播”直觉：

从未来的 double 帧，把修正往更早帧倒着传播回来。

也就是：

▼

Plain Text

```
1 future double → current single
```

四、所以“反向传播”到底指哪一个方向

严格说，这一步里有两个时间方向：

情况 A：从过去 double 向现在淡出

这是：

- 过去是 double
- 现在变成 single

那就要把过去 double 的修正，向现在缓慢消退

这个方向你可以理解成：

正向延续 / 向后淡出

情况 B：从未来 double 向现在预热

这是：

- 现在是 single
- 未来会进入 double

那就要把未来 double 的修正，向当前帧提前倒灌一点

这个方向你可以理解成：

反向传播 / 往回传播 / back in time

结论

如果你说“这个步骤中的反向传播”，通常更接近指：

把未来某个 double-constraint 帧里已经知道的 ankle correction，往更早的 single 帧回传一点。

因为它确实是在“回到过去帧”。

但完整算法其实是双向的：

- **backward scan**：看过去 double
 - **forward scan**：看未来 double
 - 如果两边都有，就把两边结果再 blend 一次
-

五、为什么保存的是 offset，而不是绝对 ankle 目标

这正是 Jerome 一直强调的点。

因为你要传播的是一种“修正量”，不是“最终答案”。

比如原动画脚踝在这里：

▼ Plain Text |

```
1  原动画 ankle = A
2  double constraint 算出来的 ankle = B
```

那你真正保存的是：

▼ Plain Text |

```
1  offset = B - A
```

然后做传播时不是说：

- 第 14 帧直接跳到 B
- 第 13 帧直接跳到 B

而是说：

- 第 14 帧先加 80% offset
- 第 13 帧加 50% offset
- 第 12 帧加 20% offset

所以本质是：

传播 correction，不是传播最终 pose。

这也是为什么 buffer 里存的是：

- `ankle_rotation_buffer`
- `ankle_position_buffer`

而不是“最终 ankle absolute pose”。

六、 α 在这里起什么作用

传播不是硬拷贝，而是衰减传播。

代码里会先算：

```
Python |  
1  alpha = (f+1.0)/(L2+1.0)  
2  alpha = 2*alpha*alpha*alpha - 3*alpha*alpha + 1
```

这个 `alpha` 的意思就是：

离那个 double frame 越近，吃进去的修正越多；

离得越远，吃进去的修正越少。

所以反向传播不是“整段复制过去”，而是：

一个随时间逐渐变小的 correction 扩散。

这就是为什么它看起来更像“过渡带”，而不是“突然切换”。

七、给你一个完整的小例子

假设：

▼								Plain Text
1	帧号：	20	21	22	23	24	25	
2	状态：	S	S	S	S	S	D	

第 25 帧算出：

- 一段 ankle rotation offset = R

如果不传播：

▼								Plain Text
1	20:	I						
2	21:	I						
3	22:	I						
4	23:	I						
5	24:	I						
6	25:	R						

那么 24 → 25 会突然跳。

如果做“反向传播”：

▼								Plain Text
1	20:	0.1R						
2	21:	0.2R						
3	22:	0.35R						
4	23:	0.55R						
5	24:	0.8R						
6	25:	1.0R						

这样到了第 25 帧，脚踝早就开始朝那个目标方向“靠拢”了，所以切换会自然得多。

八、这一步和 Root / IK 的关系

这一点也很重要。

这里传播的不是最终腿部姿态，而只是：

- ankle rotation offset
- ankle position offset

也就是说，这一节只是先把：

“脚踝目标”做平滑

后面的：

- Root Placement
- Leg IK

才会去真正让整条腿跟着这个平滑目标走。

所以你可以把这里的“反向传播”理解成：

先把末端目标的时序做顺，再让骨架去追它。

九、最短记忆版

你可以直接记成这三句：

1) 这里的反向传播不是梯度反传

它是沿时间轴把 ankle correction 往更早帧扩散。

2) 它解决的问题是

single ↔ double constraint 切换时，ankle correction 不能突然开/关。

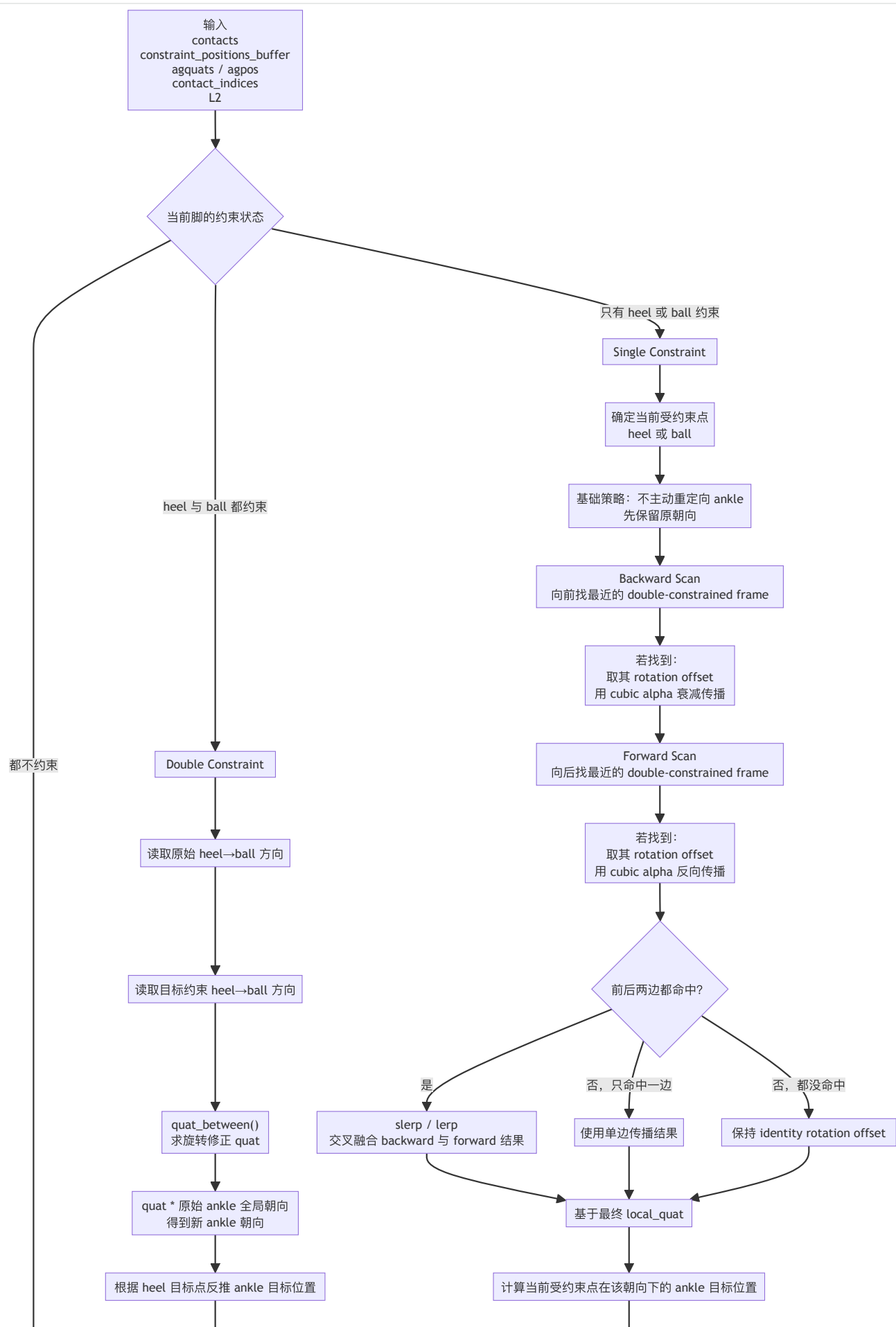
3) 它传播的不是最终脚踝绝对位置

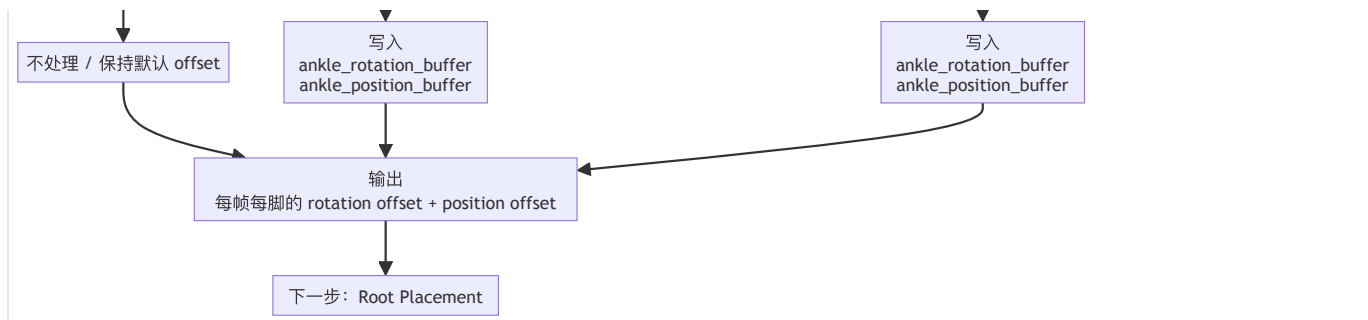
而是相对原动画的 offset，并且用 `alpha` 做衰减。

如果你愿意，我下一条可以直接给你画一张：

“single → double 时，ankle offset 如何反向传播”的 mermaid 时间轴图。

数据流程图





Root Position

<https://chatgpt.com/g/g-p-69affc08e9088191811d69bef0b44dc1/c/69c112a0-e0b0-83a6-8eeb-b9c51323986a>

<https://chatgpt.com/share/69c11fc4-e388-8009-8701-d8b47d35d9e5>

总览

▼ GPT

好，这一步我按你指定的方式来讲：

先讲 **Root Position / Root Placement** 这一步的目的，再讲 **整体输入 / 输出**，然后把你贴的整块代码拆成几个部分，逐段解释每一行在做什么、它和论文/语音稿分别是什么关系。最后我给你一张 **mermaid 流程图**。

先给你一句最直观的话：

这一步不是在“真正解腿 IK”，而是在 IK 之前，先把骨盆 / root 往一个更合理的位置轻轻挪一下，让腿尽量够得着前一步已经算好的 ankle 目标。

论文第三阶段的原话就是：先计算 root 应该放在哪里，使得上一步得到的 ankle 目标位置能被腿通过伸展所到达；同时又要兼顾 root 调整的平滑性，所以有时 ankle 目标会被允许“略微超出可达范围”，剩余误差再由下一步轻微拉伸腿去补。

Jerome 在语音稿里也用更直白的话说：如果你把脚修到一个离髋太远的位置，腿根本够不到，所以这一步就是把 hips 往能让脚够到的位置拉；若只有一只脚受约束，就沿那只脚的方向拉；若两只脚都受约束，就要找“两条腿都能够到”的位置，也就是两个球体可达域的交集附近。

一、这一步的目的是什么

这一步在整条五阶段链路里的位置是：

1. Constraint Positions
2. Ankle Position and Orientation
- 3. Root Placement**
4. Meet Target Ankle Configuration (腿部 IK)
5. Final smoothing / dissipation

也就是说，前两步已经给你了：

- 哪些脚底点该锁地
- 以及 ankle 这一帧应该去哪里、朝哪儿转

但还没解决一个现实问题：

如果 ankle 目标离 hips 太远，腿即使完全伸直也够不到，怎么办？

论文对此的回答很明确：

- 先别急着掰腿

- 先尽量用一个最小 root 位移
- 把 root 挪到“腿能触达 ankle 目标”的区域里
- 这样后面的 IK 会稳定得多，也不容易爆膝盖。

所以这一步的核心目的只有一句：

先做一个“宏观位移纠正”——挪 hips / root，让后面的局部腿 IK 更容易成功。

二、总体输入和输出是什么

输入

这一步主要吃 4 类输入。

1) 前一步的 ankle 目标

也就是：

- `ankle_position_buffer`
- `ankle_rotation_buffer`

在你这段 root 代码里，真正直接用到的是 `ankle_position_buffer`。

因为 root 这里只关心“脚踝目标位置能不能够到”，暂时不关心脚踝朝向。论文也是先把 root 位置问题表述成 ankle **position reachability** 的问题。

2) 当前动画里的 hip / ankle 原始位置

代码里用的是：

- `agpos[frame, character.bone_index('LeftUpLeg'), :]`
- `agpos[frame, character.bone_index('RightUpLeg'), :]`
- `agpos[frame, character.bone_index('LeftFoot'), :]`
- `agpos[frame, character.bone_index('RightFoot'), :]`

这里在 notebook 的骨架语义里：

- `LeftUpLeg / RightUpLeg` 代表左右髋关节位置

- `LeftFoot / RightFoot` 实际上就是脚踝关节
因为 `heel / ball` 是后来注入到 `foot` 下面的子骨点，所以 `Foot` 本身就是 `ankle` 这一层。

3) 腿的最大长度 `leg_length`

这是“从 `hip` 到 `ankle` 最多能伸多长”。

论文里这叫 `lmax`，并明确说：若目标 `ankle` 距 `root` 的距离 `ltarget` 大于 `lmax`，就要把 `root` 投影回可达球体里。

4) `contacts`

代码里用 `np.any(contacts[frame, :])` 作为一个门控：

- 这一帧如果根本没有脚接触约束
- 就没有必要计算 `root` 修正

这和论文语义也一致：`root placement` 是为满足当前存在的 `ankle constraints` 服务的。

输出

输出是两个 `buffer`，但真正阶段输出是这个：

`root_buffer[frame, :]`

它表示：

这一帧 `root / hips` 相对于原动画，需要额外平移多少。

在你最后的 `render` 里能看出来：

```
Python |  
1 gpos = agpos[frame, character.bone_index('Hips'), :] + root_buffer[frame, :]
```

这说明 `root_buffer` 存的不是绝对世界位置，而是相对原 `hips` 的偏移量。

另外还有一个中间 buffer:

`_temp_root_buffer`

它是每帧独立计算出来、尚未平滑的 root offset。

论文里也明确说了:

- 先为每一帧各自算一个 root translation
 - 再在一个邻域里做平均
 - 因为 root 直接影响全身，哪怕很小的跳变也会非常显眼，所以必须再平滑。
-

三、先给你一个“代码与论文的对应关系”总览

你贴的这段代码，和论文第三阶段的关系是这样的:

论文原意

- 单脚约束: 把 root 投影到一个球面/球体内
- 双脚约束: 投影到两个球体的交集
- 为防止 root popping, 再在 `L3` 邻域里做平均
- 平均后即便略微出界, 也没关系, 下一步可通过轻微 stretch 补上。

Jerome notebook 的工程化改写

- 单脚约束: 直接算一个“往脚方向拉多少”的 offset
- 双脚约束: 不用论文里“严格的球面交 circle 投影”, 而是用一个交替投影迭代近似
- 平滑: 不用论文里“对当前 constrained block 做 weighted average”的完整形式, 而是用一个简单的固定窗口 box average
- 然后把结果存在 `root_buffer` 里, 用 render 显示出来。
Jerome 在语音稿里对这版实现的口头总结就是: 比较两条腿的距离是否超限, 算 offset, 然后把一串帧上的 root correction 做平均。

这点你一定要分清:

notebook 是忠于论文思想的教学实现, 但不是逐公式逐操作的完全等价实现。

四、Part 1: 初始化参数和 buffer

先看这三行：

```
Python |  
1  L3 = 8  
2  root_buffer = np.zeros([frame_count, 3], dtype=np.float32)  
3  leg_length = animation.pos[0,character.bone_index('LeftLeg'),0] + ani  
   motion.pos[0,character.bone_index('LeftFoot'),0]
```

第 1 行

```
Python |  
1  L3 = 8
```

这行在干什么

定义第三阶段的邻域窗口长度 `L3`。

它对应论文哪部分

论文 Figure 2 和 Section 4.3 都明确说：

- 第三阶段 root placement 需要依赖前后 `L3` 帧范围内的信息
- 用邻域平滑来减少 root popping。

直观解释

`L3 = 8` 的意思就是：

这一步不是只看当前帧，而是后面会在当前帧前后大约 8 帧范围内做 root correction 的时间平滑。

第 2 行

```
Python |  
1 root_buffer = np.zeros([frame_count, 3], dtype=np.float32)
```

这行在干什么

创建最终 root 偏移缓冲区。

shape 是什么含义

- `frame_count`：整段动画每一帧
- `3`：XYZ 三个方向的平移偏移

作用

后面每一帧算完、平滑完的 root offset，都会写进这里。

第 3 行

```
Python |  
1 leg_length = animation.pos[0,character.bone_index('LeftLeg'),0] + ani  
  mation.pos[0,character.bone_index('LeftFoot'),0]
```

这行在干什么

用左腿的两段骨长近似求出“整条腿最大可达长度”。

为什么这么写

在这个 notebook 的骨架里：

- `LeftLeg` 的局部 `offset` 对应大腿或小腿其中一段长度
- `LeftFoot` 的局部 `offset` 对应另一段长度
- 而且这个数据的腿链主要沿局部 `x` 轴展开

所以直接取它们的 `x` 分量相加，就得到一个可用的 `leg_length`。

它对应论文哪部分

论文里这一量叫 `lmax`，即“maximum leg length”，并用它和当前目标距离比较，判断 `root` 是否需要被拉近。

这里的一个实现假设

这行代码有一个很强的 notebook 特定假设：

腿长信息恰好编码在这些局部偏移的 `x` 分量里。

如果换骨架、换坐标约定，这样写未必稳。

更通用的写法通常会用两个 `offset` 向量的范数再相加。

但在这个 notebook 的具体 `skeleton` 上，这样写是成立的。

五、Part 2: `compute_root_offset` —— 这一步的核心几何函数

现在看主函数：

```
Python |  
1 def compute_root_offset(left_hips, right_hips, left_ankle, right_ankle, leg_length):
```

这个函数的任务，可以浓缩成一句话：

给定当前左右 `hip` 位置、左右目标 `ankle` 位置，以及最大腿长，算出这一帧 `root` 至少该往哪里平移，才能让腿尽量够得到。

2.1 内部 helper: `_project(root, ankle)`

```
1 def _project(root, ankle):
2     vector = ankle - root
3     dist = np.sqrt(np.sum(vector*vector))
4     if dist > leg_length:
5         return True, (vector/dist)*(dist-leg_length)
6     else:
7         return False, np.zeros([3], dtype=np.float32)
```

这几行是整个 root placement 的最小几何原子。

第 1 行

```
1 vector = ankle - root
```

作用

从当前 hip/root 指向目标 ankle 的向量。

直观上在问什么

“这条腿当前需要伸向哪个方向？”

第 2 行

```
1 dist = np.sqrt(np.sum(vector*vector))
```

作用

求当前 hip 到目标 ankle 的直线距离。

对应论文

这正对应论文里的 `ltarget`，也就是“target ankle position 离 root 有多远”。

第 3~6 行

```
Python |
1 if dist > leg_length:
2     return True, (vector/dist)*(dist-leg_length)
3 else:
4     return False, np.zeros([3], dtype=np.float32)
```

这段在干什么

如果目标 ankle 距离已经超过了最大腿长：

- 那就说明这条腿够不到
- 于是返回 `True`
- 并给出一个 offset，大小恰好是 `dist - leg_length`
- 方向则是朝向 ankle 的单位向量

也就是：

“root 至少还要朝 ankle 方向移动这么多，腿才刚好够到。”

为什么这个 offset 公式是对的

因为：

- 当前距离是 `dist`
- 最大允许距离是 `leg_length`
- 还差 `dist - leg_length`
- 而最小改动方案显然就是沿当前连线方向拉近

对应论文

论文说的是：

若 `ltarget > lmax`，就把 root 投影回“以 ankle target 为球心、半径等于最大腿长”的可达球体上。

Jerome 这个 `_project` 的写法，实际上就是在直接返回“为了完成这次投影，root 需要增加的最小位移”。

2.2 分别检查左右腿

```
Python |  
1 left, left_offset = _project(left_hips, left_ankle)  
2 right, right_offset = _project(right_hips, right_ankle)
```

作用

分别判断：

- 左腿是否够得到左 ankle target
- 右腿是否够得到右 ankle target

并各自算出如果不够，需要多少 root offset。

为什么要分左右

因为可能出现 3 种典型情况：

1. 只有左腿不够
2. 只有右腿不够
3. 两条腿都不够

后面代码就是按这三种分支处理。

2.3 只有一条腿超限的情况

```
1 if left and not right:  
2     return left_offset  
3  
4 if right and not left:  
5     return right_offset
```

这段在干什么

如果只有一条腿目标超出可达范围：

- 那就只按那条腿给出的 offset 去挪 root

对应 Jerome 语音稿

Jerome 的原话是：

如果只有一只脚被约束，而且太远了，你只能把 root 朝着你正在拉的那只脚的方向移动。

对应论文

论文里单脚情况就是：把 root 投影到单个球体上。

Jerome 这里的实现方式，和论文从几何上是同一件事，只是表达为“直接返回最小 offset”。

2.4 两条腿都超限的情况：交替投影近似双球交

这是函数最有意思的地方。

```
Python |  
1 ▾ if left and right:  
2     current_offset = left_offset  
3 ▾     for i in range(5):  
4         loop, offset = _project(right_hips + current_offset, right_ankle)  
5 ▾         if not loop:  
6             return current_offset  
7         current_offset += offset  
8         loop, offset = _project(left_hips + current_offset, left_ankle)  
9 ▾         if not loop:  
10            return current_offset  
11            current_offset += offset  
12            return current_offset
```

先看大意

这段代码在做：

当左右两条腿都够不到时，用一个小迭代，在“满足左腿可达”和“满足右腿可达”之间来回投影，逼近一个两腿都尽量可达的位置。

第 1 行

```
Python |  
1 current_offset = left_offset
```

作用

先用左腿给出的 offset 作为初值。

也就是先满足左腿，再看右腿还差多少。

第 2 行

```
Python |  
1  for i in range(5):
```

作用

最多迭代 5 次。

为什么是迭代

因为“双腿都超限”时，root 理论上应该投影到两个可达球体的交集。

论文给的是更严格的几何表述：

- 先分别投影到两个球体
- 若某个投影已经在双球 3D 交里，就可以接受
- 否则应投影到两个球面交出的那条圆上。

Jerome notebook 没有去显式求那个交圈，而是用了一个更适合教学实现的近似：

交替投影（alternating projection）

也就是：

- 先修左腿
- 再修右腿
- 再修左腿
- 再修右腿
- 几次后得到一个足够好的折中 offset

这是 notebook 和论文在实现层面的一个明显差异。

右腿投影



Python |

```
1 loop, offset = _project(right_hips + current_offset, right_ankle)
2 if not loop:
3     return current_offset
4 current_offset += offset
```

这三行在干什么

意思是：

1. 假设我们先应用了当前 `current_offset`
2. 看看此时右腿还够不够
3. 如果右腿已经够到了，就当前结果直接返回
4. 否则，把为了让右腿够到所需的额外 `offset` 再加进去

左腿再投影



Python |

```
1 loop, offset = _project(left_hips + current_offset, left_ankle)
2 if not loop:
3     return current_offset
4 current_offset += offset
```

作用

刚刚为了满足右腿又动了一下 `root`，这可能重新把左腿搞超限。
所以再回头检查左腿。

这正是一个经典的“在两个约束之间反复折中”的过程。

结束返回

```
Python |  
1 return current_offset
```

含义

如果 5 次迭代都没完全让两条腿同时进入可达域，就返回当前最好的近似解。

和论文关系

论文更“严格几何”，notebook 更“实用近似”。

你可以把 notebook 这里理解成：

“我不显式求双球交 circle，但我用来回修正的方法逼近那个区域。”

2.5 没有任何腿超限

```
Python |  
1 return np.zeros([3], dtype=np.float32)
```

作用

如果左右腿都够得到当前 ankle targets，就不需要动 root。

非常合理

因为 root placement 的目标是：

只做最小必要改动。

既然当前已经可达，就不应该额外改 hips。

六、Part 3：逐帧计算未平滑的 `_temp_root_buffer`

下面这段：

```
Python |
1  _temp_root_buffer = np.zeros([frame_count, 3], dtype=np.float32)
2  for frame in range(frame_count):
3      if np.any(contacts[frame, :]):
4          _temp_root_buffer[frame, :] = compute_root_offset(
5              agpos[frame, character.bone_index('LeftUpLeg'), :],
6              agpos[frame, character.bone_index('RightUpLeg'), :],
7              agpos[frame, character.bone_index('LeftFoot'), :] + ankle
8              _position_buffer[frame, 0, :],
9              agpos[frame, character.bone_index('RightFoot'), :] + ankle
10             e_position_buffer[frame, 1, :],
11             leg_length
12         )
```

这段代码是在做：

先不考虑时间平滑，单独对每一帧求一个“这一帧如果只顾可达性，root 至少该挪多少”的原始答案。

第 1 行

```
Python |
1  _temp_root_buffer = np.zeros([frame_count, 3], dtype=np.float32)
```

作用

创建一个中间结果 buffer，专门存“每帧独立求出的 raw root offset”。

为什么不直接写进 `root_buffer`

因为接下来还要做时间平滑。

这和论文描述是完全一致的：先独立算，再平均。

第 2 行

```
Python |  
1  for frame in range(frame_count):
```

作用

逐帧处理整段动画。

第 3 行

```
Python |  
1  if np.any(contacts[frame, :]):
```

作用

只有当这一帧至少存在一个脚部接触约束时，才需要计算 root 修正。

为什么这样合理

如果这一帧根本没有脚底锁地要求，
那 root placement 这一阶段就没有“必须让某个 ankle target 可达”的任务。

调用 `compute_root_offset(...)`

```

Python |
1  _temp_root_buffer[frame, :] = compute_root_offset(
2      agpos[frame, character.bone_index('LeftUpLeg'), :],
3      agpos[frame, character.bone_index('RightUpLeg'), :],
4      agpos[frame, character.bone_index('LeftFoot'), :] + ankle_positio
      n_buffer[frame, 0, :],
5      agpos[frame, character.bone_index('RightFoot'), :] + ankle_positi
      on_buffer[frame, 1, :],
6      leg_length
7  )

```

这几行是整段 root stage 的“输入拼装”。

左右 hips

```

Python |
1  agpos[frame, character.bone_index('LeftUpLeg'), :]
2  agpos[frame, character.bone_index('RightUpLeg'), :]

```

作用

取当前帧左右髋关节的全局位置。

为什么不是用 **Hips**

因为 root 实际要保证的是左右各自的腿链可达，
所以最自然的判断点是每条腿真正的起点——左右髋。

左右 ankle targets



Python |

```
1 agpos[frame, character.bone_index('LeftFoot'), :] + ankle_position_buffer[frame, 0, :]  
2 agpos[frame, character.bone_index('RightFoot'), :] + ankle_position_buffer[frame, 1, :]
```

这两行非常关键

它们不是当前原始 ankle 位置，而是：

原始 ankle 位置 + 上一步算好的 ankle 平移 offset

所以这一步实际在问的是：

“上一步我希望 ankle 去的位置，现在对这条腿来说够得到吗？”

这就把第二阶段和第三阶段准确接上了：

- 第二阶段给出 ankle target
- 第三阶段根据 ankle target 反推 root 该怎么动

对应论文

论文 4.3 开头就说：

到这一步，我们已经知道 F_i-L3 到 F_i+L3 范围内 constrained ankles 的目标全局位置和朝向了；下一步要调整腿去满足这些目标，但在这之前先尝试移动 root，使目标 ankle positions 变得可达。

七、Part 4：对 `_temp_root_buffer` 做时间平均，得到最终 `root_buffer`

下面是第二个 for：

```
Python |
1 for frame in range(frame_count):
2     if np.any(contacts[frame, :]):
3         count = 0
4         value = np.zeros([3], dtype=np.float32)
5         for f in range(-L3+1, L3):
6             if frame+f >= 0 and frame+f < frame_count:
7                 value += _temp_root_buffer[frame+f, :]
8                 count += 1
9         root_buffer[frame, :] = value/count
```

这段就是论文里“root popping 的简化平滑解法”。

为什么还要平滑

论文非常强调：

- root 的单帧投影可能在“单脚约束”和“双脚约束”切换时发生很大变化
- 即便 root 原本很平稳，投影结果也可能突然跳
- root 一跳，全身都会跳，所以这种 popping 非常显眼。

Jerome 也在语音稿里直接说：

为了避免 root 只在某一帧突然被拉一下，他们会把 root correction 在一串帧上做平均。

第 1 行

```
Python |
1 for frame in range(frame_count):
```

作用

再次逐帧遍历，但这次不是“独立求值”，而是“对邻域求平均”。

第 2 行

```
Python |  
1 if np.any(contacts[frame, :]):
```

作用

只对有约束的帧输出平滑后的 root 修正。

初始化累加器

```
Python |  
1 count = 0  
2 value = np.zeros([3], dtype=np.float32)
```

作用

准备对邻域内的 raw root offsets 求均值。

邻域循环

```
Python |  
1 for f in range(-L3+1, L3):
```

作用

遍历当前帧前后大约 `L3` 长度的窗口。

当 `L3 = 8` 时，这个范围是：

- `-7, -6, ..., 0, ..., 6, 7`

也就是总共 15 帧左右的邻域。

对应论文

论文说 root translations 是在 `Fi-L3` 到 `Fi+L3` 的邻域里先独立计算，再做 average。

边界判断

```
Python |  
1 if frame+f >= 0 and frame+f < frame_count:
```

作用

避免访问越界帧。

累加邻域结果

```
Python |  
1 value += _temp_root_buffer[frame+f, :]  
2 count +=1
```

作用

把邻域里的 raw root offset 全部加起来，准备求平均。

注意

这里是简单平均，即 box filter。

而论文原文写得更精细一些：

- 先对邻域内各帧独立算 root translations
- 再对包含当前帧的 constrained block 做 weighted average。

Jerome 这一版 notebook 把它简化成：

“固定窗口里直接平均。”

这也是 notebook 相比论文的又一个实现简化。

写入最终结果

```
Python |  
1 root_buffer[frame, :] = value/count
```

作用

把平滑后的 root offset 写入最终 buffer。

这意味着什么

现在 `root_buffer` 里的值，不再是“单帧最精确的可达解”，而是：
在时间上更平滑、更适合整段动画观感的折中解。

一个非常重要的后果

这会带来论文和 Jerome 都明确承认的副作用：

平均之后，root 不一定还严格处于所有球体可达域里。

也就是说，腿有可能还是差一点点够不到。

但这不是问题，因为：

- 论文下一步会允许轻微 stretch 来精确满足 ankle targets
- Jerome 在语音稿里也直说：平均后不能保证一定完全够到，但没关系，下一步 IK/leg stretch 会修掉。

这就是第三阶段和第四阶段之间最关键的接口关系。

八、Part 5: render 这一整块其实是“可视化 cell”，不是 root 计算本体

你贴的代码后半段：

```
Python |  
1 def render(frame):  
2     ...  
3     interact(...)  
4     viewer
```

从 notebook 结构上看，应该视为下一格独立的显示 cell。

你贴出来时缩进把它和上面的 for 混在一起了，但就 notebook 本意来说，它不是 root 计算循环内部的一部分，而是单独的“显示当前 root / ankle / constraint target”工具。

这一点我先说清楚，避免你以后照抄时误以为 render 定义真在 for 里面。

8.1 render(frame) 开头：先画原始 animation

```
Python |  
1 anim = animation  
2 p = (anim.pos[frame,...])  
3 q = (anim.quats[frame,...])  
4  
5 a = lab.utils.quat_to_mat(q, p)  
6 viewer.set_shadow_poi(p[0])  
7  
8 viewer.begin_shadow()  
9 viewer.draw(character, a)  
10 viewer.end_shadow()  
11  
12 viewer.begin_display()  
13 viewer.draw_ground()  
14 viewer.draw(character, a)
```

这段在干什么

先把当前帧原始角色画出来。

为什么先画原始角色

因为 root / ankle 的目标轴线等都是叠加调试信息。

先画 base pose，后面才能看清：

- 原角色当前在哪
 - 约束点在哪
 - ankle target 在哪
 - hips 经过 root_buffer 修正后又该在哪
-

8.2 画上一节的 constraint targets

```
Python |  
1 contacts = np.eye(4, dtype=np.float32)[np.newaxis,...].repeat(4, axis=0)  
2 contacts[:, :3, 3] = constraint_positions_buffer[frame, :, :]  
3  
4 viewer.draw(target, contacts)  
5 viewer.end_display()
```

这段在干什么

把四个小 target 放到当前帧四个 constraint position 上。

意义

你可以直观看到：

- 这一步 root placement 到底是为了让 hips 去服务哪几个脚底目标点

这相当于是把第一阶段的输出也同时显示出来。

8.3 再画原骨架线

```
▼ Python |
1 viewer.disable(depth_test=True)
2 viewer.draw_lines(character.world_skeleton_lines(a))
```

作用

显示当前角色骨架线条，便于对照 hips、ankle 和 target 的空间关系。

8.4 画左脚 ankle target 轴

```
▼ Python |
1 gquat = lab.utils.quat_mul(ankle_rotation_buffer[frame, 0, :], agquat
2 s[frame, character.bone_index('LeftFoot'), :])
3 gpos = agpos[frame, character.bone_index('LeftFoot'), :] + ankle_posi
4 tion_buffer[frame, 0, :]
5 a = lab.utils.quat_to_mat(gquat, gpos)
6 viewer.draw_axis(a[np.newaxis, ...], 20)
```

这段在干什么

画出左脚在上一阶段算出来的 ankle 目标位姿。

为什么 root stage 要把它画出来

因为 root stage 的任务正是：

“让 hips 尽量移到能让这个 ankle target 够得着的位置。”

所以你要同时看见 ankle target 才能判断 root 是否合理。

8.5 画右脚 ankle target 轴

```

Python |
1  gquat = lab.utils.quat_mul(ankle_rotation_buffer[frame, 1, :], agquat
   s[frame, character.bone_index('RightFoot'), :])
2  gpos = agpos[frame, character.bone_index('RightFoot'), :] + ankle_pos
   ition_buffer[frame, 1, :]
3  a = lab.utils.quat_to_mat(gquat, gpos)
4  viewer.draw_axis(a[np.newaxis,...], 20)

```

作用

和左脚完全一样，只是换成右脚。

8.6 画 root / hips 的目标轴

```

Python |
1  gquat = agquats[frame, character.bone_index('Hips'), :]
2  gpos = agpos[frame, character.bone_index('Hips'), :] + root_buffer[frame, :]
3  a = lab.utils.quat_to_mat(gquat, gpos)
4  viewer.draw_axis(a[np.newaxis,...], 20)

```

这段是 root stage 可视化的核心

它画的是：

- 原始 hips 的朝向不变
- 但位置加上 `root_buffer[frame, :]`

也就是说，显示“这一帧我希望 hips 被平移到哪里”。

为什么只改位置不改朝向

因为这一阶段论文和 notebook 都是在做 **root translation**，不是 root rotation。

论文明确说 root placement 是“move the root by the smallest amount”，并且 footskate cleanup 里 root 修改的是位置，不是朝向。

8.7 提交渲染

```
▼ Python |
1 viewer.execute_commands()
```

作用

把所有可视化内容提交给 viewer。

8.8 交互滑条

```
▼ Python |
1 interact(
2     render,
3     frame=lab.Timeline(max=frame_count-1)
4 )
5 viewer
```

作用

让你可以拖 frame，看整段动画里：

- constraint targets
- 左右 ankle target axes
- root target axis

是怎么随时间变化的。

Jerome 在语音稿里也说了，他会把 root position buffer 显示出来，看“我希望 hips 在哪”。

九、把这一整步浓缩成一句话

你可以把 `Root Position` 理解成：

前一步已经决定了脚踝应该去哪里，这一步先不解腿，而是先问：现在 hips 放在哪，腿才有机会够得到那些脚踝目标？

如果只有一条腿超出可达范围，就沿那条腿的方向最小量移动 root；

如果两条腿都超出，就在两个腿的可达球体约束之间做折中；

然后为了避免 root popping，再把这些逐帧修正做时间平均；

即便平均后略微不够，也没关系，下一步用腿部 IK 和轻微 stretch 再补。

十、你学习这一步时最应该盯住的几个关键词

1) `leg_length`

最大腿长，也就是每条腿的可达半径。

2) `_project`

单腿可达性判断：不够就返回朝 ankle 的最小 root offset。

3) 单腿 vs 双腿

- 单腿：单球约束
- 双腿：双球交约束

4) `_temp_root_buffer`

每帧独立解，强调“可达性”。

5) `root_buffer`

邻域平均后的平滑解，强调“观感连续性”。

6) 平均后不一定严格可达

但残差由下一步 IK / stretch 补上。

十一、和你未来 UE / GASPALS / Shadow Mode 的关系

从你的长期目标看，这一步特别像一个很适合独立模块化的“中间层”：

它属于哪一层

在 GASPALS / UE 动画链里，它大概位于：

- Motion Matching / Base Locomotion 输出之后
- Ankle / Foot target 已知之后
- Leg IK / FBlk 之前

它的本质职责

不是生成动作，而是：

把“局部末端目标”转换成“更容易求解的上游根部位置修正”。

这跟 UE 里很多现代工作流也很像：

- 宏观轨迹先由根或骨盆调整
- 微观贴合再交给 IK / FBlk / Post Process

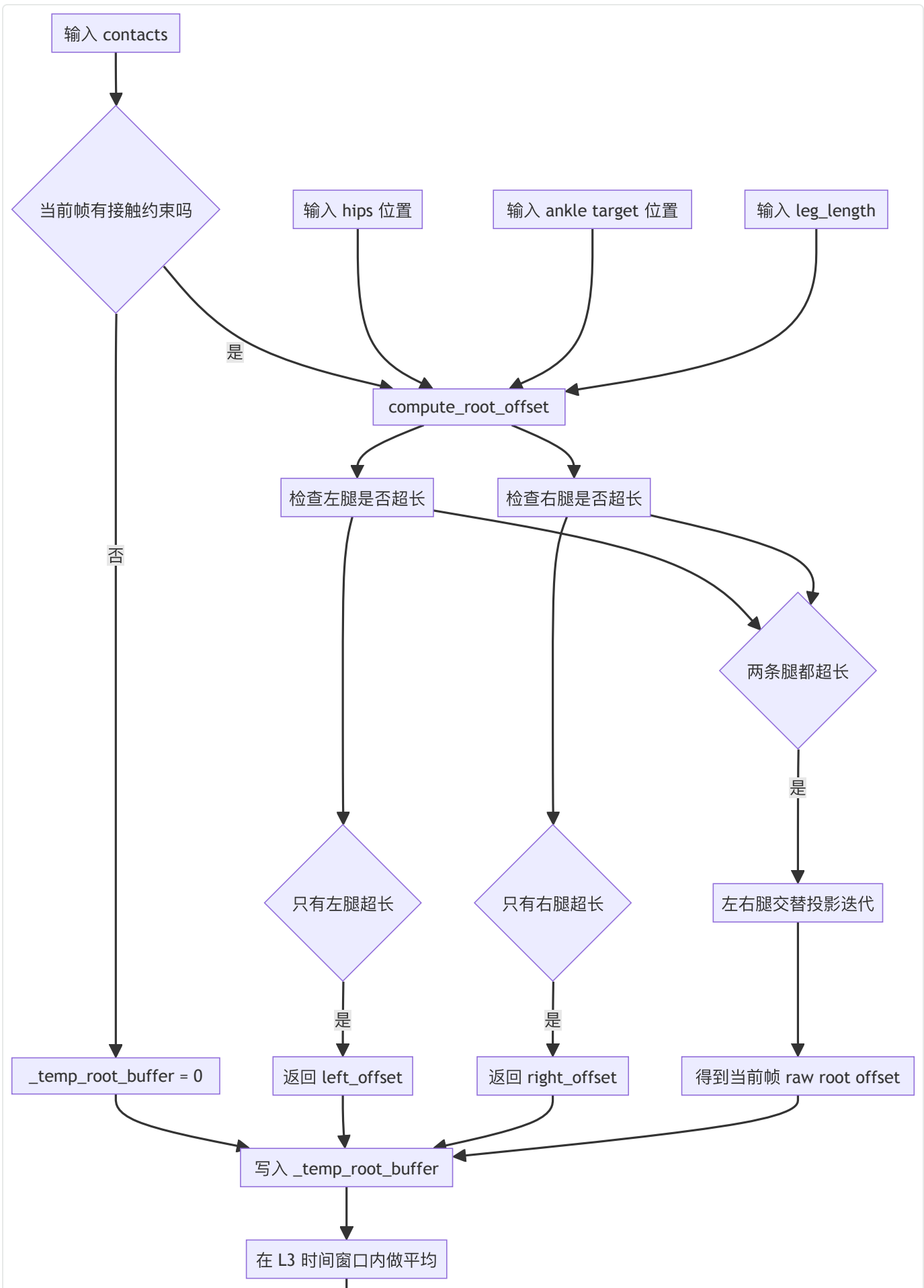
所以这一步非常适合你以后在 Shadow Mode 里单独接管做 A/B：

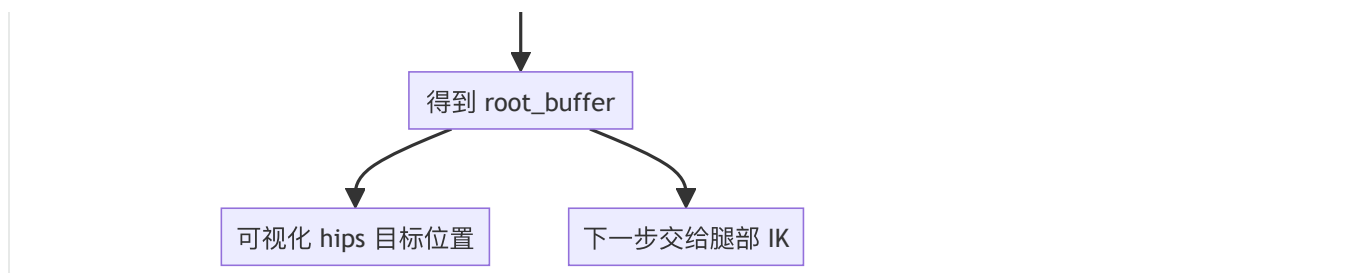
- 规则法 root placement
- 学习法 root correction
- 或者结合 Motion Warping / Pose Search 去做更强的 root biasing

但不管实现变不变，这一层的职责都很稳定：

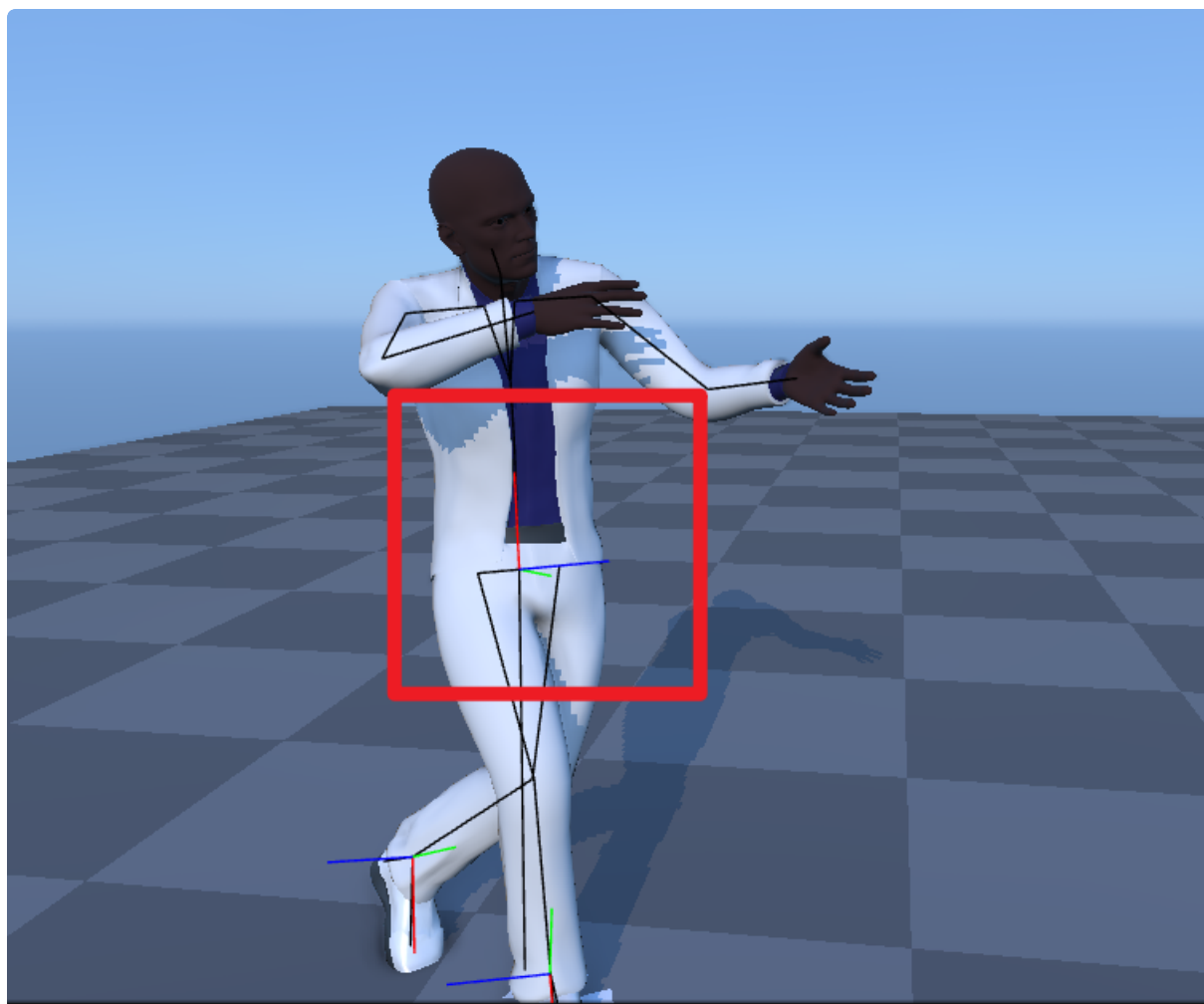
先帮下游 IK 降低难度。

数据流程图





Root位置确定



Compute IK

<https://chatgpt.com/g/g-p-69affc08e9088191811d69bef0b44dc1-mldeformer/c/69c236b6-97fc-83ab-b84c-8cb63b278ef0>

<https://chatgpt.com/share/69c237e7-2f54-8009-985d-c5e5fdb837d4>

总览

▼ GPT

好，这一步我按你要求的顺序来，而且我会把你贴的整块代码拆回到合适的位置里讲。

先给一句总纲：

Compute IK 这一步，不是在“再算 ankle 目标”，而是在前面三步已经知道 ankle 该去哪、root 该往哪挪之后，真正去改腿链，让腿的姿态落到那个目标上。

论文把它定义为第 4 阶段：在第 2 步得到的 target ankle configuration 和第 3 步得到的 root position 已知的前提下，调整 hip、knee、ankle 参数，使目标 ankle 位置和朝向被满足；尽量用关节旋转完成，必要时才允许长度调整来避免 knee pop。

Jerome 在语音稿里对这一步的总结也非常直白：先把腿看成一个三角形，先算膝盖应该弯多少；再把整条腿转向目标；再把 ankle 放回目标朝向和位置；如果腿太接近打直，就做 knee damping，避免 knee pop。

一、这一步的目的是什么

这一阶段的目的可以分成两层。

第一层：精确满足 ankle target

前两步已经给了你：

- `ankle_position_buffer`
- `ankle_rotation_buffer`

也就是“这一帧 ankle 最终该在哪里、朝哪儿转”。

第三步又给了你：

- `root_buffer`

也就是“hips 先往哪里平移，腿才比较有机会够到”。

那么第四步就要真正回答：

现在我怎样改大腿、小腿、脚踝这些局部关节，才能让这条腿真的摆出那个目标姿态？

论文原话就是：

- 使用上一步得到的 root position
- 调整 hip、knee、ankle

- 满足 target ankle positions and orientations。
-

第二层：避免 knee pop

如果腿接近完全打直，那么“再多伸一点点”和“少伸一点点”，在几何上只差 1~2 cm，但对膝盖角度来说，可能会导致非常剧烈的跳变。论文把这叫 **knee pop**。

Jerome 在语音稿里说得很形象：

- 你只是想补 1、2 厘米
 - 但膝盖可能“刷”一下从这里跳到那里
 - 这个 pop 在视觉上非常明显
- 所以他会用一个多项式去压制接近完全打直时的 knee rotation。
-

二、总体输入和输出是什么

输入

这一步真正用到的输入，有 5 组。

1) 原始要修的动画 `animation`

代码一开始就 `copy.deepcopy(animation)`，说明它是以原始待修动画为底稿。

2) 上一步得到的 `root_buffer`

它先被加到 `Hips` 上，作为“已经修过 root 的新底姿态”。

3) 第二步得到的 ankle targets

也就是：

- `ankle_position_buffer`
- `ankle_rotation_buffer`

这两个 buffer 直接参与左脚和右脚的最终对齐。

4) 原始全局 FK 结果 `agquats / agpos`

这一步很多目标都是“相对原动画的 offset”，所以仍然要回看原始 `agpos/agquats`。

代码里左脚、右脚都在用 `agpos[...] + ankle_position_buffer[...]` 和 `quat_mul(ankle_rotation_buffer, agquats[...])`。

5) 腿长 `len1 / len2`

这是把腿当成两段骨链做解析 IK 的基础长度。

输出

输出只有一个核心对象：

`solved_animation`

也就是整条动画在完成：

- root 平移
- 膝盖弯曲
- hip 指向目标
- ankle 精确对齐

之后的最终求解结果。代码最后就是：

```
1 solved_animation = compute_final_animation()
```

而 render 里也允许你切换：

- `solved=True` 看求解后
- `solved=False` 看原始动画

这本质上就是 A/B 对照查看 IK 是否把脚真正放回目标。

三、把这一步拆成几个部分

我建议你把 `Compute IK` 看成 7 段：

- 1 A. 先看 `knee damping polynomial` 是什么
- 2 B. 复制动画，并先把 `root` 修正加上去
- 3 C. 预计算腿长相关常数
- 4 D. 解左腿：先算膝角，再转大腿，再强制脚踝贴目标
- 5 E. 解右腿：同样流程复制一遍
- 6 F. 返回 `solved_animation`
- 7 G. `render` 只是在可视化“求解前后 + 目标轴”

下面我就按这 7 段讲。

四、Part A：先看 `# knee damping polynomial`

你贴的第一小段是：

```
1 #knee damping polynomial
2
3 x = np.linspace(-1, 1, 100)
4 fig, (ax) = plt.subplots(1, 1, figsize=(5,5))
5 ax.plot(x)
6 z = -.037037*x**7 + .0314815*x**6 + .0374074*x**5 - .0260185*x**4 - .
  0114815*x**3 + .00453704*x**2 + 1.00111*x
7
8 ax.plot(z)
```

这段的目的是什么

这不是 IK 本体，而是一个可视化解释 cell：

把“原始 arccos 输入”到“经过 knee damping 多项式修正后的 arccos 输入”画出来，方便你看 Jerome 到底怎么压制 straight-leg 区域。

Jerome 在语音稿里专门说了：

- 他会先算 arccos
 - 然后把 arccos 输入送进一个 polynomial
 - 这个 polynomial 几乎是一条直线
 - 但在接近 straight-leg 的末端会刻意压一下
 - 目的是“永远留一点角度”，不要真的打成 180° ，从而避免 knee pop。
-

逐行解释

第 1 行

```
1 x = np.linspace(-1, 1, 100)
```

生成 100 个样本点，覆盖 arccos 的有效输入范围 `[-1, 1]`。

第 2 行

```
1 fig, (ax) = plt.subplots(1, 1, figsize=(5,5))
```

创建一个图窗。

第 3 行

```
1 ax.plot(x)
```

先把输入的“理想恒等映射”画出来。

这条线可以理解成“如果不做 knee damping，输入是多少就直接是多少”。

第 4 行

```
1 z = -.037037*x**7 + .0314815*x**6 + .0374074*x**5 - .0260185*x**4 - .0114815*x**3 + .00453704*x**2 + 1.00111*x
```

这是 Jerome 手工拟合出来的 damping 多项式。

它不是论文公式原样抄过来，而是一个工程近似实现。论文给的是分段函数 `f(x)` 和积分形式，用来限制接近完全伸直时的膝盖角度调整；Jerome 则直接把这个效果压成一个可用的多项式映射。

第 5 行

```
1 ax.plot(z)
```

把修正后的曲线画出来。

你会看到绝大部分区域几乎贴着直线，但在接近 straight-leg 的边界开始“收住”。

五、Part B：函数开头——拷贝动画，并先应用 root 修正

现在进入真正的主函数。你贴出来的核心实现就是这个 `compute_final_animation()`。

先看开头：

```
1 import copy
2
3 def compute_final_animation():
4     local_anim = copy.deepcopy(animation)
5     local_anim.pos[:, character.bone_index('Hips'), :] += root_buffer
6     qq, gp = lab.utils.quat_fk(local_anim.quats, local_anim.pos, local_anim.parents)
```

第 1 行

```
Python |  
1 local_anim = copy.deepcopy(animation)
```

作用

深拷贝一份动画，避免直接污染原始 `animation`。

工程意义

后面所有 IK 修改都写到 `local_anim` 上。

这样你还能保留原始动画做对比。代码里 A/B render 也是基于这个思路。

第 2 行

```
Python |  
1 local_anim.pos[:, character.bone_index('Hips'), :] += root_buffer
```

作用

先把第三步求出来的 root 修正应用到整段动画的 `Hips` 上。

为什么 IK 一开始先做这个

因为这一步是“在已经挪过 root 的前提下，再解腿”。

论文也明确说：

- 第四步使用上一步得到的 root position
- 然后再去满足 ankle targets。

所以这一行是在把第三阶段和第四阶段真正串起来。

第 3 行

```
1  gq, gp = lab.utils.quat_fk(local_anim.quats, local_anim.pos, local_anim.parents)
```

作用

对应用了 root 修正后的动画做一次 FK，得到当前全局旋转 `gq` 和全局位置 `gp`。

为什么要先 FK

后面不管是：

- 算 hip 到 ankle 的向量
- 还是计算 `quat_between`
- 或者把 foot 强行塞回目标

都要用全局空间信息。

Jerome 在语音稿里也说了，Compute IK 这一步是多阶段的：先改膝，再重算全局位置，再转整条腿，再回写局部。

六、Part C：预计算腿长相关常数

接下来：

```
1  # prepare sizes
2      len1 = animation.pos[0, character.bone_index('LeftLeg'), 0]
3      len2 = animation.pos[0, character.bone_index('LeftFoot'), 0]
4      l1l2_2 = np.array([len1 * len2 * 2.0], dtype=np.float32).repeat(frame_count)
5      l1_l2 = np.array([len1*len1 + len2*len2], dtype=np.float32).repeat(frame_count)
```

这段整体在干什么

这是在为余弦定理做准备。

Jerome 的语音稿里说得很清楚：

- 腿在这里被看成一个三角形
- 你知道两段腿长
- 也知道 hip 到目标 ankle 的距离
- 所以可以先算膝盖角度。

第 1 行

```
Python |  
1 len1 = animation.pos[0, character.bone_index('LeftLeg'), 0]
```

取左腿第一段长度。

第 2 行

```
Python |  
1 len2 = animation.pos[0, character.bone_index('LeftFoot'), 0]
```

取左腿第二段长度。

为什么只用左腿

这里默认左右腿骨长相同，所以后面右腿复用这两个长度常数。

为什么用 **x** 分量

和前面 root stage 一样，这是这个 notebook 骨架的一个实现假设：腿骨偏移主要存放在局部 x 轴上。

对这个具体 skeleton 有效。

第 3 行

```
Python |  
1  l1l2_2 = np.array([len1 * len2 * 2.0], dtype=np.float32).repeat(frame_  
    count)
```

预先构造 $2 * len1 * len2$ ，对应余弦定理分母。

第 4 行

```
Python |  
1  l1_l2 = np.array([len1*len1 + len2*len2], dtype=np.float32).repeat(fr  
    ame_count)
```

预先构造 $len1^2 + len2^2$ ，对应余弦定理分子的一部分。

为什么做成 repeat(frame_count)

因为后面是整段向量化计算，不是一帧一帧 for-loop。

Jerome 这份 notebook 很多步骤都是“整段动画 batch 化”在算。

七、Part D：解左腿——先算膝角，再转整条腿，再把脚踝强制贴回目标

这一段是 Compute IK 的真正核心。我先把整段左腿代码贴出来：

```

1  # compute left leg angle
2  #-----
3  ankle_vector = agpos[:, character.bone_index('LeftFoot'), :] + a
nkle_position_buffer[:, 0, :] - gp[:, character.bone_index('LeftUpLe
g'), :]
4  ankle_dist = np.sum(ankle_vector*ankle_vector, axis=-1)
5  ankle_arccos = (l1_l2 - ankle_dist)/l1l2_2
6
7  # knee damping using a polynomial to ease out the over stretch
8  ankle_arccos = -.037037*ankle_arccos**7 + .0314815*ankle_arccos*
*6 + .0374074*ankle_arccos**5 - .0260185*ankle_arccos**4 - .0114815*
ankle_arccos**3 + .00453704*ankle_arccos**2 + 1.00111*ankle_arccos
9
10 # apply the rotation on the knee
11 _angle = (np.pi - np.arccos(np.maximum(np.array(-1, dtype=np.flo
at32).repeat(frame_count), ankle_arccos)))/2.0
12 local_anim.quats[:, character.bone_index('LeftLeg'), 0] = np.cos
(-_angle)
13 local_anim.quats[:, character.bone_index('LeftLeg'), 3] = np.sin
(-_angle)
14
15 # recompute the global position of the foot
16 local_anim.quats = lab.utils.quat_normalize(local_anim.quats)
17 gq, gp = lab.utils.quat_fk(local_anim.quats, local_anim.pos, loc
al_anim.parents)
18
19 # rotate the leg (must be done on global position, then convert
back on local)
20 rot = lab.utils.quat_between(gp[:, character.bone_index('LeftFoo
t'), :] - gp[:, character.bone_index('LeftUpLeg'), :], ankle_vector)
21 gq[:, character.bone_index('LeftUpLeg'), :] = lab.utils.quat_mul
(rot, gq[:, character.bone_index('LeftUpLeg'), :])
22 local_anim.quats[:, character.bone_index('LeftUpLeg'), :] = lab.
utils.quat_ik(gq, gp, local_anim.parents)[0][:, character.bone_index
('LeftUpLeg'), :]
23
24 # recompute the global position
25 local_anim.quats = lab.utils.quat_normalize(local_anim.quats)
26 gq, gp = lab.utils.quat_fk(local_anim.quats, local_anim.pos, loc
al_anim.parents)
27
28 # force the foot on the constraint in global and convert in loca
l
29 gq[:, character.bone_index('LeftFoot'), :] = lab.utils.quat_mul(
ankle_rotation_buffer[:, 0, :], agquats[:, character.bone_index('Lef

```

```

30     tFoot'), :])
    gp[:, character.bone_index('LeftFoot'), :] = agpos[:, character.
31     bone_index('LeftFoot'), :] + ankle_position_buffer[:, 0, :]
    lq, lp = lab.utils.quat_ik(gq, gp, local_anim.parents)
32     local_anim.quats[:, character.bone_index('LeftFoot'), :] = lq[:,
    character.bone_index('LeftFoot'), :]
33     local_anim.pos[:, character.bone_index('LeftFoot'), :] = lp[:, c
    haracter.bone_index('LeftFoot'), :]

```

它其实可以再拆成 3 个小步骤：

1. 先算膝盖该弯多少
2. 再把整条腿转向目标
3. 最后把脚踝强制放回目标位姿

这三步和 Jerome 语音稿的描述是一一对应的：

“先算角度 → rotate the knee → recompute → rotate the entire leg → put back my ankle。”

D1：先算左腿的目标几何量

第 1 行

```

1  ankle_vector = agpos[:, character.bone_index('LeftFoot'), :] + ankle_
    position_buffer[:, 0, :] - gp[:, character.bone_index('LeftUpLeg'), :
    ]

```

这行在干什么

构造从左 hip 指向左目标 ankle 的向量。

分三部分看：

- `agpos[:, LeftFoot, :]`：原始左 ankle 位置
- `+ ankle_position_buffer[:, 0, :]`：加上第二阶段算出来的 ankle 目标平移 offset
- `- gp[:, LeftUpLeg, :]`：减去当前 root 修正后左 hip 的全局位置

所以它得到的是：

“当前 root 位置下，左 hip 到左目标 ankle 的目标向量。”

这正是论文里 single-limb IK 的目标向量 `pT - pH` 的 notebook 版本。

第 2 行

```
Python |  
1 ankle_dist = np.sum(ankle_vector*ankle_vector, axis=-1)
```

作用

求这个向量的平方长度，也就是 hip 到目标 ankle 的距离平方。

为什么不用 sqrt

因为后面代入余弦定理时，公式本来就可以直接用平方长度，更省一步开方。

第 3 行

```
Python |  
1 ankle_arccos = (l1_l2 - ankle_dist)/l1l2_2
```

作用

这一步是在构造余弦定理里的 `cos(theta)` 项。

对于两段长度分别为 `len1`，`len2` 的腿，若 hip 到目标 ankle 的距离为 `d`，则有：

$$\cos \theta = \frac{\text{len1}^2 + \text{len2}^2 - d^2}{2 \text{len1} \text{len2}}$$

这正是这行代码。

Jerome 在语音稿里说“prepare all the different length compared to the distance,

computing the vector, computing the arccos”，讲的就是这一段。

论文里也明确说：

- 先把腿看成 triangle
- 再用几何关系求 knee angle。

D2：对这个 arccos 输入做 knee damping

第 4 行

```
1 ankle_arccos = -.037037*ankle_arccos**7 + .0314815*ankle_arccos**6 +  
  .0374074*ankle_arccos**5 - .0260185*ankle_arccos**4 - .0114815*ankle_  
  arccos**3 + .00453704*ankle_arccos**2 + 1.00111*ankle_arccos
```

作用

把原始的 `cos(theta)` 输入通过一个多项式压一遍，得到“更保守的膝角输入”。

为什么这么做

因为当腿接近打直时，直接按普通解析 IK 去算，会出现 knee pop。论文给的标准方案是：

- 用 knee damping 限制靠近完全伸直时的膝盖旋转量
- 如果还差一点到不了目标，就再允许腿长轻微变化。

Jerome 的 notebook 没有按论文去“真的拉长 thigh/shin bone”，而是自己做了一个工程改写：

- 先把 arccos 输入送进多项式
- 保证永远别真的打成完全 straight
- 让残余误差更多由 ankle 附近吸收

他在语音稿里亲口说了：

“not exactly how they were doing it”，他不是去 stretch actual leg，而是锁住 knee angle，让 deformation 更多吸收到 ankle/foot relative to leg。

所以这里你一定要分清：

论文

- knee damping
- 然后必要时 stretch leg length。

Jerome notebook

- 用 polynomial 近似 knee damping
- 再把剩余误差吸收到 ankle/foot 目标那边。

D3：把这个膝角真正写进左小腿 / 左膝关节

第 5 行

```
1 _angle = (np.pi - np.arccos(np.maximum(np.array(-1, dtype=np.float32)
    .repeat(frame_count), ankle_arccos)))/2.0
```

这行在干什么

它先把前面那个经过 damping 的 `ankle_arccos` 取 `arccos`，得到角度，再变形成为当前骨架需要的“膝关节旋转半角”。

分开看：

```
np.maximum(..., ankle_arccos)
```

只做下界钳制到 `-1`，避免数值误差导致 `arccos` 输入掉到非法范围以下。

```
np.arccos(...)
```

把 `cos(theta)` 变回角度。

```
np.pi - ...
```

把三角几何角转换成这个骨架链实际需要的弯曲角定义。

因为后面写 quaternion 时，`cos` 和 `sin` 用的是半角。

为什么是半角

因为单位四元数表示旋转时，本来就写成：

$$q = [\cos(\phi/2), \sin(\phi/2)\hat{n}]$$

$$q = [\cos(\phi/2), \sin(\phi/2)\hat{n}]$$

所以这里 `_angle` 是拿来直接塞进 quaternion 的半角变量。

第 6、7 行

```
Python |
1 local_anim.quats[:, character.bone_index('LeftLeg'), 0] = np.cos(-angle)
2 local_anim.quats[:, character.bone_index('LeftLeg'), 3] = np.sin(-angle)
```

这两行在干什么

直接重写左膝关节（这里骨架里叫 `LeftLeg`）的局部 quaternion。

为什么只写 `[0]` 和 `[3]`

因为这个骨架里膝盖被当成单自由度铰链，主要绕固定轴转。

这里的写法就是：

- `w = cos(half_angle)`
- 某个固定轴分量 `= sin(half_angle)`

也就是只保留“绕一根轴的纯旋转”。

对应论文

论文明确说：

- knee can only rotate about a single axis
- 所以先调整 knee angle。

Jerome 语音稿也说了：

- 先 compute the angle
 - then rotate the knee。
-

D4：重算 FK，因为膝盖已经变了

第 8、9 行

```
Python |  
1 local_anim.quats = lab.utils.quat_normalize(local_anim.quats)  
2 gq, gp = lab.utils.quat_fk(local_anim.quats, local_anim.pos, local_an  
  im.parents)
```

作用

因为你刚刚直接改了局部 quaternion，所以要：

1. 先 normalize，避免数值误差
2. 再做 FK，得到“现在膝盖弯完之后”的全局姿态

为什么必须重算

不然你后面根本不知道：

- 当前 foot 被膝盖弯曲带到了哪里
- 还差多少才能朝向 target

Jerome 语音稿里也明确说了：

- rotate the knee
 - then recompute the position。
-

D5：把整条左腿再转向目标 ankle 方向

第 10 行

```
1 rot = lab.utils.quat_between(gp[:, character.bone_index('LeftFoot'), :]  
    - gp[:, character.bone_index('LeftUpLeg'), :], ankle_vector)
```

作用

计算一个 quaternion，把：

- 当前膝盖弯完以后，左 hip→左 foot 的方向
- 旋转到目标 `ankle_vector` 的方向

也就是“整条腿朝目标再对准一下”。

对应论文

论文原算法中，这一步就是：

- 先得到新 ankle 位置 $\hat{p}_A$
- 再用 $\text{Rot}(\hat{p}_A - p_H, p_T - p_H)$ 去旋转 hip，让腿真正 pointing toward target。

这行 `quat_between(...)` 就是 notebook 的实现版本。

第 11 行

```
1 gq[:, character.bone_index('LeftUpLeg'), :] = lab.utils.quat_mul(rot,  
    gq[:, character.bone_index('LeftUpLeg'), :])
```

作用

把这次“整条腿指向目标”的全局旋转，乘到左大腿 / 左 hip 关节的全局旋转上。

直观理解

刚才是改膝盖“腿长和弯曲量”，
现在是改 hip “整条腿朝哪指”。

第 12 行

```
Python |  
1 local_anim.quats[:, character.bone_index('LeftUpLeg'), :] = lab.utils  
  .quat_ik(gg, gp, local_anim.parents)[0][:, character.bone_index('Left  
  UpLeg'), :]
```

作用

前一行改的是全局旋转，但动画实际存的是局部 quaternion。

所以这里要用 `quat_ik(...)` 把全局姿态逆回局部，取出新的 `LeftUpLeg` 局部旋转写回去。

为什么这一行很关键

它说明 Jerome 这里的 IK 不是“黑盒数值迭代”，而是：

- 在全局空间做几何操作
- 再转换回局部动画参数

这和论文强调的“analytic IK”思路是一致的。

D6：再次重算 FK，因为 hip 也被改了

第 13、14 行

```
Python |  
1 local_anim.quats = lab.utils.quat_normalize(local_anim.quats)  
2 gq, gp = lab.utils.quat_fk(local_anim.quats, local_anim.pos, local_an  
  im.parents)
```

作用

现在膝盖和 hip 都改了，再次刷新全局姿态。

为什么要再来一次

因为最后一步要“强制脚踝精确贴回目标位姿”，
所以必须在最新骨架状态下操作。

D7：最后强制把左脚踝贴到目标位置和目标朝向

第 15 行

```
Python |  
1 gq[:, character.bone_index('LeftFoot'), :] = lab.utils.quat_mul(ankle  
  _rotation_buffer[:, 0, :], agquats[:, character.bone_index('LeftFoot'  
    ), :])
```

作用

把左脚踝的**目标朝向**直接设置成：

- 原始 ankle 朝向
- 再乘上第二阶段求出的 rotation offset

这正对应第二阶段的输出语义。

第 16 行

```
1 gp[:, character.bone_index('LeftFoot'), :] = agpos[:, character.bone_index('LeftFoot'), :] + ankle_position_buffer[:, 0, :]
```

作用

把左脚踝的**目标位置**直接设置成：

- 原始 ankle 位置
- 再加上第二阶段求出的 position offset

也就是“我要强制这个 foot 真的出现在那个 target ankle location 上”。

第 17 行

```
1 lq, lp = lab.utils.quat_ik(gq, gp, local_anim.parents)
```

作用

现在你已经在全局空间里明确指定了：

- 左 foot 的全局旋转
- 左 foot 的全局位置

于是再次做全局→局部逆变换，得到新的局部 quats / pos。

为什么这一步叫“force the foot on the constraint”

因为这一刻不是“尽量朝目标”，而是：

直接把 foot 的全局目标钉死。

Jerome 在注释里也写了 `force the foot on the constraint in global and convert in local`。

第 18、19 行

```
▼ Python |
1 local_anim.quats[:, character.bone_index('LeftFoot'), :] = lq[:, character.bone_index('LeftFoot'), :]
2 local_anim.pos[:, character.bone_index('LeftFoot'), :] = lp[:, character.bone_index('LeftFoot'), :]
```

作用

把计算出的左 foot 局部旋转和局部位置写回动画。

到这里左腿发生了什么

左腿整个链已经经历了：

1. 应用 root 修正
2. 算膝盖角度
3. 膝盖做 knee damping 后弯曲
4. 大腿整体对准目标
5. 脚踝最后精确贴到目标位置 / 朝向

这就是 Jerome 说的“all the step of the paper”。

八、Part E：右腿完全重复左腿流程

右腿代码和左腿是镜像的：

```

1  # compute right leg angle
2  #-----
3  ankle_vector = agpos[:, character.bone_index('RightFoot'), :] +
ankle_position_buffer[:, 1, :] - gp[:, character.bone_index('RightUp
Leg'), :]
4  ankle_dist = np.sum(ankle_vector*ankle_vector, axis=-1)
5  ankle_arccos = (l1_l2 - ankle_dist)/l1l2_2
6
7  # knee damping using a polynomial to ease out the over stretch
8  ankle_arccos = -.037037*ankle_arccos**7 + .0314815*ankle_arccos*
*6 + .0374074*ankle_arccos**5 - .0260185*ankle_arccos**4 - .0114815*
ankle_arccos**3 + .00453704*ankle_arccos**2 + 1.00111*ankle_arccos
9
10 # apply the rotation on the knee
11 _angle = (np.pi - np.arccos(np.maximum(np.array(-1, dtype=np.flo
at32).repeat(frame_count), ankle_arccos)))/2.0
12 local_anim.quats[:, character.bone_index('RightLeg'), 0] = np.co
s(-_angle)
13 local_anim.quats[:, character.bone_index('RightLeg'), 3] = np.si
n(-_angle)
14
15 # recompute the global position of the foot
16 local_anim.quats = lab.utils.quat_normalize(local_anim.quats)
17 gq, gp = lab.utils.quat_fk(local_anim.quats, local_anim.pos, loc
al_anim.parents)
18
19 # rotate the leg (must be done on global position, then convert
back on local)
20 rot = lab.utils.quat_between(gp[:, character.bone_index('RightFo
ot'), :] - gp[:, character.bone_index('RightUpLeg'), :], ankle_vecto
r)
21 gq[:, character.bone_index('RightUpLeg'), :] = lab.utils.quat_mu
l(rot, gq[:, character.bone_index('RightUpLeg'), :])
22 local_anim.quats[:, character.bone_index('RightUpLeg'), :] = lab
.utils.quat_ik(gq, gp, local_anim.parents)[0][:, character.bone_inde
x('RightUpLeg'), :]
23
24 # recompute the global position
25 local_anim.quats = lab.utils.quat_normalize(local_anim.quats)
26 gq, gp = lab.utils.quat_fk(local_anim.quats, local_anim.pos, loc
al_anim.parents)
27
28 # force the foot on the constraint in global and convert in loca
l
29

```

```

30     gq[:, character.bone_index('RightFoot'), :] = lab.utils.quat_mul
        (ankle_rotation_buffer[:, 1, :], agquats[:, character.bone_index('Ri
        ghtFoot'), :])
31     gp[:, character.bone_index('RightFoot'), :] = agpos[:, character
        .bone_index('RightFoot'), :] + ankle_position_buffer[:, 1, :]
32     lq, lp = lab.utils.quat_ik(gq, gp, local_anim.parents)
33     local_anim.quats[:, character.bone_index('RightFoot'), :] = lq[
        :, character.bone_index('RightFoot'), :]
        local_anim.pos[:, character.bone_index('RightFoot'), :] = lp[:,
        character.bone_index('RightFoot'), :]

```

这整段和左腿完全对称，唯一变化是：

- `Left*` 全换成 `Right*`
- `ankle_position_buffer[:, 0, :]` 换成 `[:, 1, :]`

为什么右腿不用重新单独准备腿长

因为默认左右骨长一致，所以 `len1 / len2 / l1_l2 / l1l2_2` 可以复用。

工程意义

这说明 notebook 的“单腿解析 IK 求解器”已经足够一般，只是左右各跑一遍。

九、Part F：返回最终求解结果

最后这两行：

```

1     return local_anim
2
3     solved_animation = compute_final_animation()

```

`return local_anim`

返回整段已经求解完的动画。

```
solved_animation = compute_final_animation  
( )
```

立即跑一遍，得到可视化用的结果缓存。

和论文的关系

到这里，第四阶段“Meeting the Target Ankle Configuration”已经做完了。

如果你严格按论文顺序，下一步其实应该进入 final processing，去处理 constraint on/off 时的整体 blend off。

Jerome 在语音稿里也明确说了：

- 到这里你会发现脚已经能 properly placed
 - 但在 constraint 开关时 still have pops
 - 那就是下一步 final processing 要解决的。
-

十、Part G: `render(...)` 这一整块在做什么

你还贴了一个 render：

```

1 def render(frame, solved=True):
2
3     anim = solved_animation if solved else animation
4     p = (anim.pos[frame,...])
5     q = (anim.quats[frame,...])
6
7     a = lab.utils.quat_to_mat(q, p)
8     viewer.set_shadow_poi(p[0])
9
10    viewer.begin_shadow()
11    viewer.draw(character, a)
12    viewer.end_shadow()
13
14    viewer.begin_display()
15    viewer.draw_ground()
16    viewer.draw(character, a)
17
18    contacts = np.eye(4, dtype=np.float32)[np.newaxis,...].repeat(4,
axis=0)
19    contacts[:, :3, 3] = constraint_positions_buffer[frame, :, :]
20
21    viewer.draw(target, contacts)
22    viewer.end_display()
23
24    viewer.disable(depth_test=True)
25
26    viewer.draw_lines(character.world_skeleton_lines(a))
27
28    gquat = lab.utils.quat_mul(ankle_rotation_buffer[frame, 0, :], a
gquats[frame, character.bone_index('LeftFoot'), :])
29    gpos = agpos[frame, character.bone_index('LeftFoot'), :] + ankle
_position_buffer[frame, 0, :]
30    a = lab.utils.quat_to_mat(gquat, gpos)
31    viewer.draw_axis(a[np.newaxis,...], 20)
32
33    gquat = lab.utils.quat_mul(ankle_rotation_buffer[frame, 1, :], a
gquats[frame, character.bone_index('RightFoot'), :])
34    gpos = agpos[frame, character.bone_index('RightFoot'), :] + ankl
e_position_buffer[frame, 1, :]
35    a = lab.utils.quat_to_mat(gquat, gpos)
36    viewer.draw_axis(a[np.newaxis,...], 20)
37
38    gquat = agquats[frame, character.bone_index('Hips'), :]
39    gpos = agpos[frame, character.bone_index('Hips'), :] + root_buff
er[frame, :]

```

```
40     a = lab.utils.quat_to_mat(gquat, gpos)
41     viewer.draw_axis(a[np.newaxis,...], 20)
42
43     viewer.execute_commands()
```

这块不是 IK 计算本体，而是一个验证器 / 调试显示器。

1) `anim = solved_animation if solved else animation`

允许你在 UI 里切换看：

- 求解前
- 求解后

这是最直接的 A/B。

2) 前半段 `viewer.draw(character, a)`

先画当前角色姿态。

3) `contacts[:, :3, 3] = constraint_positions_buffer[frame, :, :]`

把第一阶段算出来的四个脚底地面 target 画出来。

意义

你可以直接看脚底最终有没有贴回这些 target。

4) 左脚、右脚 ankle axis

```
Python |  
▼  
1  gquat = quat_mul(ankle_rotation_buffer, agquats[...])  
2  gpos  = agpos[...] + ankle_position_buffer[...]
```

这两段画的是第二阶段算出来的 ankle target axes。

意义

你可以看到：

- IK 解出来的腿
 - 和“本来该满足的 ankle target”
 - 是否真的对上
-

5) hips axis

```
Python |  
▼  
1  gpos = agpos[frame, Hips] + root_buffer[frame]
```

这段画的是第三阶段 root target。

意义

于是一个画面里你同时能看到：

- 第一阶段：constraint positions
- 第二阶段：ankle targets
- 第三阶段：root target
- 第四阶段：solved_animation 实际是否把整条腿摆到位

这就是为什么这个 render 很有价值：

它不是只看结果 mesh，而是把整条求解链全部可视化。

十一、这一步和论文的“同”与“不同”

这一段你一定要吃透。

相同点

Jerome 的 notebook 完整保留了论文第四阶段的主结构：

1. 先根据腿长和目标距离算膝角
 2. 再 rotate the leg toward target
 3. 再把 ankle 精确放到目标位姿
 4. 这样完成 target ankle configuration。
-

不同点

论文在 knee damping 后，如果 ankle 还差一点到不了 target，会：

- 轻微调整腿长
- 改 `oK` 和 `oA`
- 真正 stretch bones。

Jerome 则明确说：

- “I will not stretch the actual leg”
- “I will just lock the angle of the knee”
- “and stretch by moving the foot relative to the leg”
- 也就是把那点误差更多吸收到 ankle / foot 那边。

所以 notebook 是：

论文思路的教学改写版

而不是一份 100% 逐公式对等的实现。

十二、把这一步浓缩成一句话

你可以把 **Compute IK** 理解成：

先在已经应用 root 修正的姿态上，把每条腿看成一个两段骨的三角形；

先用余弦定理求膝盖该弯多少；

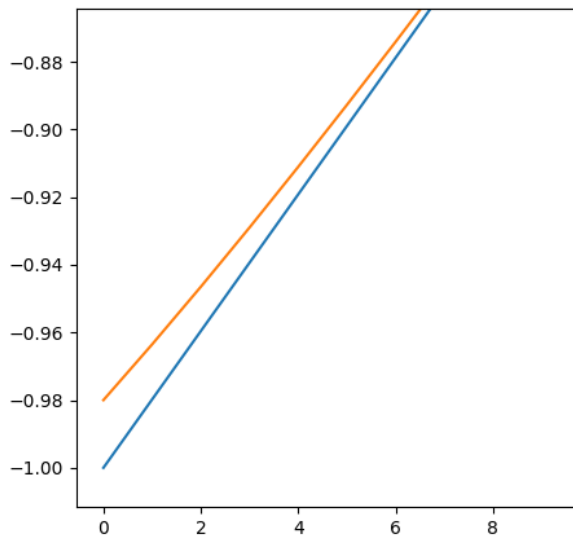
再对这个膝角做 knee damping，避免腿接近打直时的 knee pop；

然后把整条腿从 hip 处转向目标 ankle 方向；

最后把 foot/ankle 在全局空间里强制贴回前面算好的 target position 和 target orientation；

左腿做完，再对右腿重复一次。

knee damping polynomial



这张图的目的，是直观展示 knee damping polynomial 对 IK 膝盖解算输入的“限幅/缓冲”作用：蓝线代表原始的几何输入，橙线代表经过多项式修正后的输出，它说明在腿部接近完全伸直、最容易发生 knee pop 的危险区间里，系统不会直接使用原始解析解，而是把输入轻轻推回到更保守、更平滑的范围，从而避免膝盖突然打直或角度剧烈跳变；换句话说，这张图不是在展示角色动画本身，而是在展示“为防止膝盖弹跳而加入的非线性修正函数长什么样”。

▼ GPT

你这里“不明白橙线为什么比蓝线高”，核心原因其实有两层。

第一层，是负数坐标的直觉问题。

你这张图的纵轴全是负数，比如 `-1.00`、`-0.98`、`-0.96...`。

在这种坐标里，越高不代表数值越小，反而代表数值越大。所以：

```
▼ Plain Text |
1  -0.98 比 -1.00 高
```

因为 `-0.98 > -1.00`。

所以橙线比蓝线高，实际含义是：

多项式输出 `z` 比原始输入 `x` 更大了一点点。

不是“更猛地压下去”，而是“把它从 `-1` 那个危险极限值往上抬了一点”。

第二层，是这个 polynomial 本来就是故意这么设计的。

在这段 IK 里，`x`（也就是 `ankle_arccos`）接近 `-1` 时，代表腿快要完全打直了。

而代码后面会做：

```
▼ Python |
1  _angle = (np.pi - np.arccos(x)) / 2.0
```

这时候如果：

- `x = -1`
- 那么 `arccos(-1) = π`
- 所以 `_angle = 0`

这就对应**完全伸直**，也就是最容易出现 knee pop 的危险状态。

所以 Jerome 才故意让 polynomial 在 `-1` 附近把输入稍微往上抬一点：

```
▼ Plain Text |
1  原来: x = -1.00
2  修后: z ≈ -0.98
```

这样会发生什么？

- `arccos(-1.00) = 180°`

- $\arccos(-0.98) < 180^\circ$
- 所以 `_angle` 不再是 0，而会保留一个很小的弯曲角

也就是说：

橙线比蓝线高，不是在“出错”，而是在“故意把接近完全伸直的解拉回一点”，让膝盖永远别真的打成死直。

这正是 knee damping 的目的。

你可以把它想成这样：

```

▼ Plain Text |
1  蓝线：原始几何解
2      “理论上可以完全打直，那就真的打直到 180°”
3
4  橙线：修正后的工程解
5      “别真打直，留一点点弯曲余量，避免膝盖突然弹一下”

```

再用一个极简 ASCII 看：

```

▼ Plain Text |
1  危险区（原始）：
2  x = -1.00  ---> arccos = 180° ---> 膝盖完全打直 ---> 容易 pop
3
4  修正后：
5  z = -0.98  ---> arccos < 180° ---> 膝盖还留一点弯 ---> 更稳

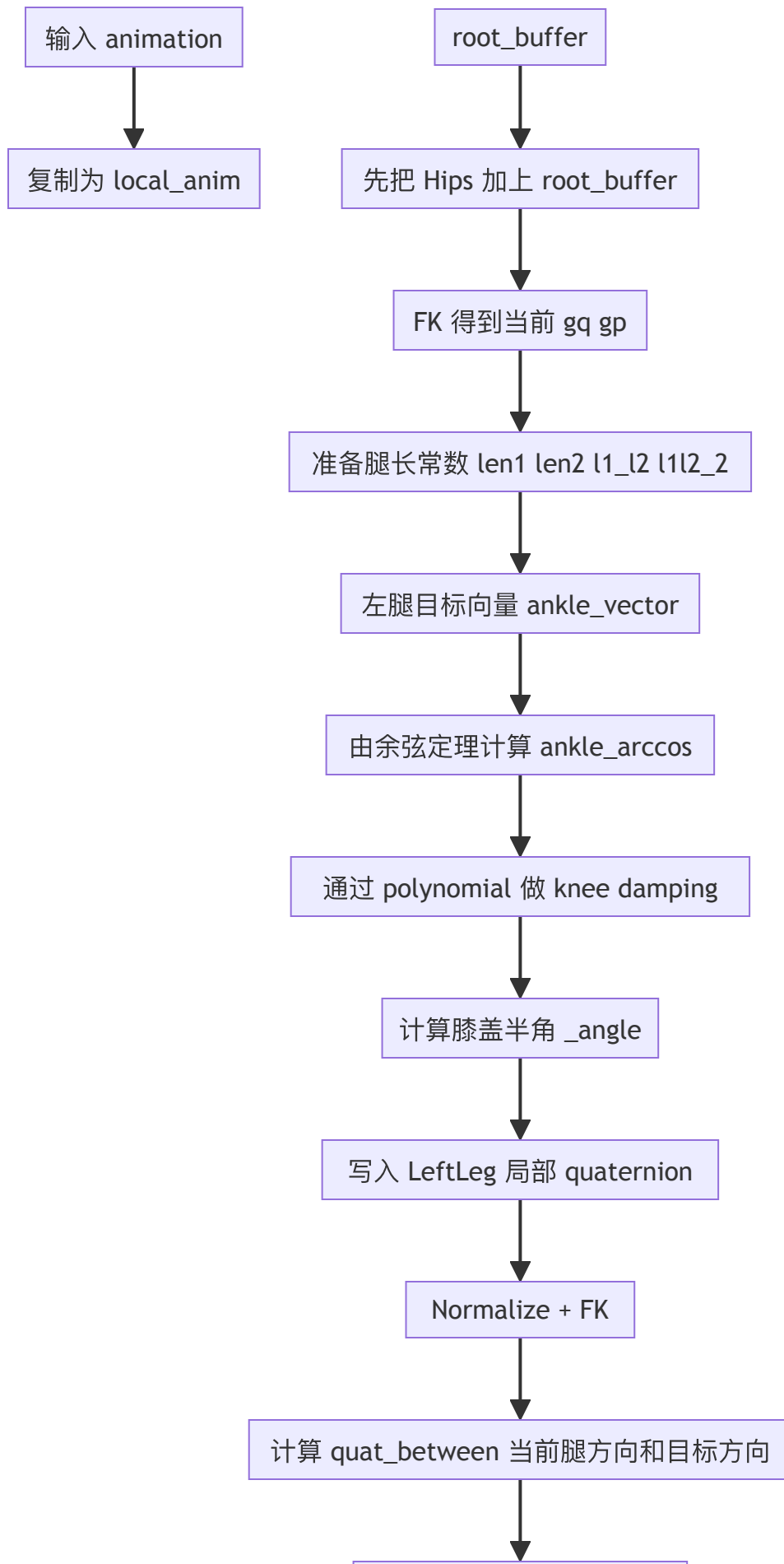
```

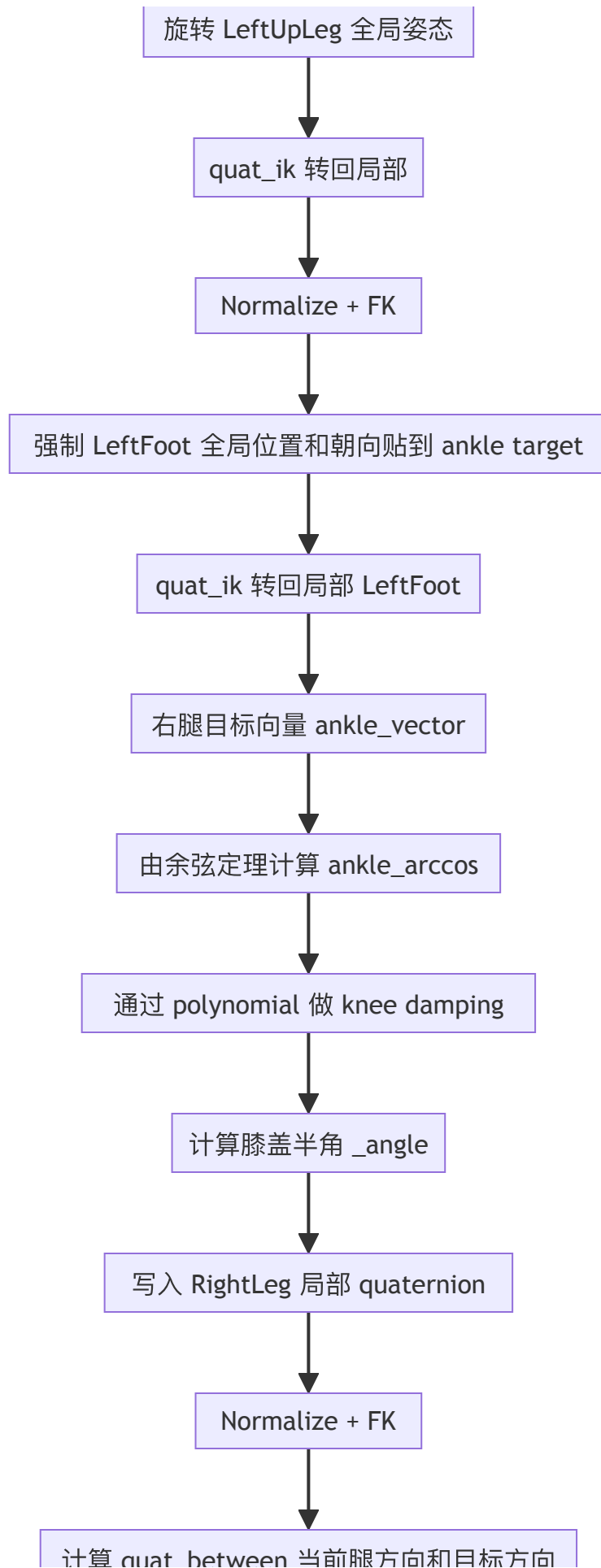
所以一句话总结就是：

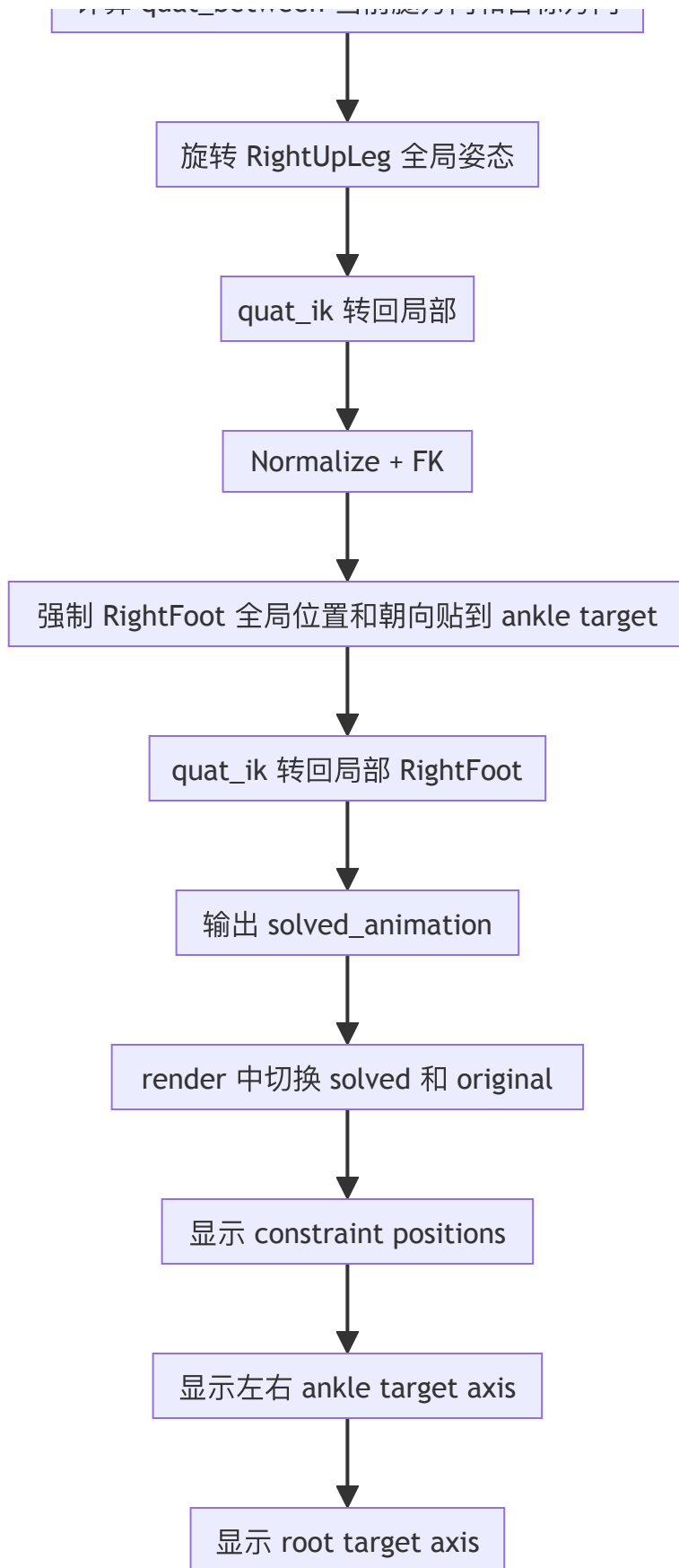
橙线比蓝线高，是因为这个 polynomial 在 `-1` 附近故意把输入值“抬高一点”，把腿从“完全伸直”的极限状态拉回来一点，从而减少 knee pop。

我可以下一条直接给你画一张真正的 `x-z` 输入输出示意，并把 `x=-1`、`z≈-0.98` 代进 `arccos` 和 `_angle`，让你一眼看懂它为什么能防膝盖弹跳。

数据流程图







Final Processing

<https://chatgpt.com/g/g-p-69affc08e9088191811d69bef0b44dc1/c/69c1466d-9b20-83aa-ac9a-9d82d0d81db8>

<https://chatgpt.com/share/69c23465-6a88-8009-863d-c0a9c69022ec>

总览

▼ GPT

可以，这一段我按你希望的方式来讲：

1. 先说 **Final Processing** 的目的
 2. 再说 它的总体输入 / 输出
 3. 然后把你贴的整块代码拆开，按合适位置逐段解释
 4. 最后给你一张 **Mermaid** 流程图
-

一、Final Processing 这一步到底是干什么的

这一步是整篇论文的 **第 5 阶段**。

前面 1~4 阶段已经做了这些事：

- 找到约束点应该在哪里
- 算出 ankle 该去哪里
- 算 root 应该怎么挪
- 让腿通过 IK / 长度调整去满足这些目标

但即使这些都做完了，**约束刚开启、刚关闭** 的地方还是很容易出现跳变，也就是 Jerome 在视频里反复说的那个 **pop**。论文对这个阶段的原话就是：当前面几步在“约束持续成立”的区间里不会引入跳变，但当约束 **active / inactive** 切换时，仍然可能出现 **discontinuity**，所以最后一步要把这些 adjustment 平滑地“blend off / dissipate”到周围无约束帧里。

Jerome 在语音稿里的解释也和论文完全对上：

前面算出来的 ankle / root 都是**相对于原动画的 offset**，所以最后一步不是重新发明一套 IK，而是：

- 检测“约束开 / 关”的边界
 - 把 offset 往前后传播一点
 - 再逐渐 blend in / blend out
 - 这样就不会在边界 frame 上突然啪一下跳过去。
-

二、这一步的总体输入和输出是什么

1. 输入

这一步真正吃进去的，不是原始 BVH，而是前几步已经算好的 buffer：

- `contacts`
 - 每帧哪些接触成立
 - 这里四列通常是：左 heel / 左 ball / 右 heel / 右 ball
- `root_buffer`
 - 第 3 阶段 root placement 算出来的 root 平移修正量
 - 不是完整 root 绝对位置，而是修正量 / offset
- `ankle_rotation_buffer`
 - 第 2 阶段和后续流程得到的 左脚 / 右脚 ankle 旋转 offset
- `ankle_position_buffer`
 - 左脚 / 右脚 ankle 位置 offset
- `frame_count`
 - 总帧数
- `L4`
 - Final Processing 的邻域窗口大小
 - 论文图 2 和 4.5 节里，Final Processing 专门对应 `L4` 这个窗口。

2. 输出

这一步输出的不是“最终角色姿态”本身，而是：

- 被平滑后的 `root_buffer`
- 被平滑后的 `ankle_rotation_buffer`
- 被平滑后的 `ankle_position_buffer`

然后这些平滑后的 buffer 再喂给：

```
Python |  
1 solved_animation = compute_final_animation()
```

得到最后可播放的 `solved_animation`。

所以你一定要分清：

Final Processing 的直接输出是“更平滑的 offset buffer”

而不是最终骨骼 pose 本体。

这也和论文的 buffer 化实现一致：论文 Figure 7 就是在讲，前面几步逐层生成 buffer，最后一层 buffer 做的就是“adjustments are blended off as described in Section 4.5”。

三、先给你一句最直观的话

这段 Final Processing 代码本质上做的是：

如果这一帧没有接触，那我就去前后找最近的“有接触并且算出过修正”的帧，把它们的 root / ankle 修正量衰减后借一点过来，让当前帧也带一点点修正，从而把“有约束”和“无约束”之间的边界磨平。

四、按代码拆开讲

Part A：窗口大小与主循环

```
Python |  
▼  
1  L4 = 10  
2  
3  for frame in range(frame_count):
```

这两行的作用

- `L4 = 10`
 - 表示 Final Processing 最多往前看 10 帧、往后看 10 帧
 - 也就是论文里 phase 5 使用的邻域长度 `L4`
- `for frame in range(frame_count):`
 - 对整段动画的每一帧都做一次“边界平滑检查”

工程理解

前面的阶段更多是在“约束区间内部”做事。

这一阶段是：

- 不再只盯着约束帧
 - 而是把整段动画每一帧都扫一遍
 - 看看哪些 **无约束帧** 其实正处在约束开关边缘附近
 - 如果是，就给它一点衰减后的修正量
-

Part B: root 的 Final Processing

先看这块：

```

1  # if there is no constraints we can check if we have to smooth the h
   ips
2  if not np.any(contacts[frame,:]):
3      pos = np.array([0,0,0], dtype=np.float32)
4
5      # backward
6      backhit = -1
7      for f in range(L4):
8          if frame-f >= 0 and np.any(contacts[frame,:]):
9              backhit = f
10             alpha = (f+1.0)/(L2+1.0)
11             alpha = 2*alpha*alpha*alpha - 3*alpha*alpha + 1
12             pos = root_buffer[frame-f]*(alpha)
13             break
14
15     # forward
16     for f in range(L4):
17         if frame+f < frame_count and np.any(contacts[frame,:]):
18             alpha = (f+1.0)/(L2+1.0)
19             alpha = 2*alpha*alpha*alpha - 3*alpha*alpha + 1
20             fpos = root_buffer[frame+f]*(alpha)
21             if backhit > -1:
22                 alpha = (backhit+1.0)/(backhit+f+1.0)
23                 pos = pos*(1.0-alpha) + fpos*(alpha)
24             else:
25                 pos = fpos
26             break
27
28     root_buffer[frame] = pos

```

B1. 这段在语义上想干什么

它想做的是：

- 只有当当前帧完全没有接触 时
- 才去决定：这一帧的 root offset 要不要从前后邻近的约束帧“借一点”过来

为什么？

因为如果当前帧本来就在接触区间里，前面的阶段已经算过 root correction 了。

Final Processing 主要补的是：

约束外侧的那些帧

这和论文 4.5 节是一致的：

对于当前帧里“没有因为满足约束而被直接调整”的参数，去前后邻域扫描，看有没有附近 frame 曾经被约束调整过；如果有，就把这些 adjustment blend off 过来。

B2. `if not np.any(contacts[frame,:]):`

这一行的意思是：

- 当前帧四个接触位（左 heel/ball、右 heel/ball）全都没开
- 说明这一帧是“无约束帧”
- 才需要做边界补偿

如果当前帧有接触，那 root_buffer 这帧本来就已经是约束求解阶段算出来的值，不需要这里改。

B3. `pos = np.array([0,0,0], dtype=np.float32)`

这是先给 root 的最终平滑结果一个默认值：

- 先假设“这一帧没有从邻居借到任何 root 修正”
- 那么它的 root offset 就是 0

也就是：

如果前后都找不到可借的修正量，那这帧就不加额外 root correction。

这也符合论文说法：如果前后都没有找到相邻的已调整帧，那参数就保持不变。

B4. backward 搜索

```

1 backhit = -1
2 for f in range(L4):
3     if frame-f >= 0 and np.any(contacts[frame,:]):
4         backhit = f
5         alpha = (f+1.0)/(L2+1.0)
6         alpha = 2*alpha*alpha*alpha - 3*alpha*alpha + 1
7         pos = root_buffer[frame-f]*(alpha)
8         break

```

这段逻辑上想做的是：

- 从当前帧往过去看，最多看 `L4` 帧
- 找到最近的一个“有接触 / 有 root 调整”的帧
- 记下它离当前帧的距离 `backhit = f`
- 然后把那个 frame 的 `root_buffer` 乘上一个衰减系数 `alpha`
- 作为当前帧从“后方邻居”继承来的修正

这里的 `alpha`

```

1 alpha = (f+1.0)/(L2+1.0)
2 alpha = 2*alpha*alpha*alpha - 3*alpha*alpha + 1

```

第二行这个三次多项式就是论文前面几节反复用的那个平滑函数：

```

[
\alpha(t)=2t^3 - 3t^2 + 1
]

```

它有个很重要的性质：

- 距离很近时，权重接近 1
- 距离变远时，权重平滑衰减到 0
- 而且首尾导数为 0，过渡更柔和，不是硬折线式的线性衰减。

直观理解

如果当前帧离最近的约束帧只差 1 帧，那它借到的 root offset 很多；
如果差了 8~10 帧，那只借一点点，甚至接近 0。

B5. forward 搜索

```
Python |
1 for f in range(L4):
2     if frame+f < frame_count and np.any(contacts[frame,:]):
3         alpha = (f+1.0)/(L2+1.0)
4         alpha = 2*alpha*alpha*alpha - 3*alpha*alpha + 1
5         fpos = root_buffer[frame+f]*(alpha)
6         if backhit > -1:
7             alpha = (backhit+1.0)/(backhit+f+1.0)
8             pos = pos*(1.0-alpha) + fpos*(alpha)
9         else:
10            pos = fpos
11         break
```

这部分和 backward 对称：

- 从当前帧往未来找
- 找到最近一个“有约束”的帧
- 算出一个前向衰减版本 `fpos`

然后分两种情况：

情况 1：前面找到了，后面没找到

那就直接用 `fpos`

情况 2：前后都找到了

那就把 backward 候选 `pos` 和 forward 候选 `fpos` 再做一次插值混合

```
Python |
1 alpha = (backhit+1.0)/(backhit+f+1.0)
2 pos = pos*(1.0-alpha) + fpos*(alpha)
```

意思是：

- 如果当前帧更靠近后面的 hit, `fpos` 权重大
- 如果更靠近前面的 hit, `pos` 权重大

这在论文里对应什么

对应论文 4.5 节那个核心思想：

- 对于无约束帧
- 前后各找最近一个曾经被调整过的帧
- 如果两边都有，就把两边 adjustment 混起来
- 从而平滑跨过“约束打开 / 关闭”的边界。

B6. 最后一行

```
Python |  
1 root_buffer[frame] = pos
```

这行非常关键：

- 它不是生成新数组
- 而是原地覆盖这一帧的 root correction

也就是说，Final Processing 结束后：

- `root_buffer` 不再只是“约束帧里的 root 修正”
- 而变成“整段动画上经过边界平滑后的 root 修正”

B7. 这一段代码里有两个你必须知道的问题

这很重要，我要很诚实地指出来：

问题 1：按你贴出来的字面代码，这两个判断写错了

```
Python |  
1 if frame-f >= 0 and np.any(contacts[frame,:]):
```

和

```
Python |  
1 if frame+f < frame_count and np.any(contacts[frame,:]):
```

但外层已经是：

```
Python |  
1 if not np.any(contacts[frame,:]):
```

所以按字面逻辑：

- 外层要求当前帧 没有接触
- 内层却又要求当前帧 有接触
- 那 inner `if` 永远进不去

也就是说：

如果这就是你 notebook 里的原始代码，那 root 的 backward / forward 搜索实际上不会命中。

从算法意图看，这里大概率应该写成：

```
Python |  
1 np.any(contacts[frame-f,:])
```

和

```
Python |  
1 np.any(contacts[frame+f,:])
```

也就是检查“邻居帧”是否有接触，而不是再检查当前帧。

问题 2：这里用了 `L2` 而不是 `L4`

```
Python |  
1 alpha = (f+1.0)/(L2+1.0)
```

但 Final Processing 理论上对应的是 `L4` 邻域，不是 `L2`。论文图 2 和 4.5 节都把最后一步单独对应到 `L4`。

所以这里更合理的写法应该是 `L4+1.0` 。

这很像 notebook 里的一个遗留变量笔误。

Part C: 定义 `_ankle(...)`，对单只脚做 Final Processing

下面这段是核心中的核心：

```

1 def _ankle(foot_contacts, rotation_offsets, position_offsets):
2     if not np.any(foot_contacts[frame,:]):
3         local_quat = np.array([1,0,0,0], dtype=np.float32)
4         local_pos = np.array([0,0,0], dtype=np.float32)
5
6         # backward
7         back_hit = -1
8         for f in range(L4):
9             if frame-f >= 0 and np.any(foot_contacts[frame-f, :]):
10                 xx = position_offsets[frame-f]
11                 back_hit = f
12                 alpha = (f+1.0)/(L4+1.0)
13                 alpha = 2*alpha*alpha*alpha - 3*alpha*alpha + 1
14                 local_quat = lab.utils.quat_slerp(np.array([[1,0,0,0]
15 ]], dtype=np.float32), rotation_offsets[frame-f][np.newaxis,:], alpha)[0]
16                 local_pos = position_offsets[frame-f]*(alpha)
17                 break
18
19         # forward
20         front_hit = -1
21         for f in range(L4):
22             if frame+f < frame_count and np.any(foot_contacts[frame+
23 f, :]):
24                 yy = position_offsets[frame+f]
25                 front_hit = f
26                 alpha = (f+1.0)/(L4+1.0)
27                 alpha = 2*alpha*alpha*alpha - 3*alpha*alpha + 1
28                 front_quat = lab.utils.quat_slerp(np.array([[1,0,0,0]
29 ]], dtype=np.float32), rotation_offsets[frame+f][np.newaxis,:], alpha)[0]
30                 front_pos = position_offsets[frame+f]*(alpha)
31
32                 if back_hit > -1:
33                     alpha = (back_hit+1.0)/(back_hit+front_hit+1.0)
34                     local_quat = lab.utils.quat_slerp(local_quat[np.
35 newaxis,:], front_quat[np.newaxis,:], alpha)[0]
36                     local_pos = local_pos*(1-alpha) + front_pos*(alp
37 ha)
38                 else:
39                     local_quat = front_quat
40                     local_pos = front_pos
41                 break
42         return local_quat, local_pos
43     return rotation_offsets[frame], position_offsets[frame]

```

C1. 这个 helper 的职责是什么

它不是处理整个人，也不是处理 root。

它专门处理：

某一只脚在当前 frame 的 ankle offset，要不要从前后邻居借一点过来。

输入：

- `foot_contacts`
 - 只看某一只脚的接触子集
 - 例如左脚就看 `[left heel, left ball]`
- `rotation_offsets`
 - 这一只脚每帧的 ankle 旋转 offset
- `position_offsets`
 - 这一只脚每帧的 ankle 位置 offset

输出：

- 当前帧这只脚应该使用的
 - `local_quat`
 - `local_pos`

也就是说，它是在回答：

当前 frame 这只脚虽然没接触，但它是不是离接触区太近，应该带一点衰减后的 ankle 修正？

C2. `if not np.any(foot_contacts[frame,:]):`

这一句意思是：

- 当前帧这只脚没有任何接触
- 只有这种情况下才需要 blend off

如果当前帧这脚本来就在接触状态：

```
1 return rotation_offsets[frame], position_offsets[frame]
```

直接把已有 offset 原样返回，不做额外平滑。

这和论文 4.5 节一模一样：

只对“没有因为约束直接被调整的参数”做邻域扫描和平滑。

C3. 为什么初值是 identity quaternion 和 zero position

```
1 local_quat = np.array([1,0,0,0], dtype=np.float32)
2 local_pos = np.array([0,0,0], dtype=np.float32)
```

这两个值的含义是：

- 默认不做任何 ankle correction
- 旋转偏移为单位四元数
- 位置偏移为零向量

这跟 Jerome 在语音稿里说的“我们保存的是相对原动画的 offset，而不是绝对目标姿态”完全一致。

因为保存的是 offset，所以：

- “没有修正”对应的旋转 offset 就是 identity quaternion
- “没有修正”对应的位置 offset 就是零向量。

C4. backward 搜索：找最近的过去接触帧

```

Python |
1 for f in range(L4):
2     if frame-f >= 0 and np.any(foot_contacts[frame-f, :]):
3         back_hit = f
4         alpha = (f+1.0)/(L4+1.0)
5         alpha = 2*alpha*alpha*alpha - 3*alpha*alpha + 1
6         local_quat = quat_slerp(identity, rotation_offsets[frame-f],
    alpha)
7         local_pos = position_offsets[frame-f] * alpha
8         break

```

这一段意思是：

- 从当前帧向过去看
- 找最近一个这只脚“有接触”的 frame
- 如果找到，就把那个 frame 的 ankle offset 衰减后借过来

为什么旋转要用 `slerp(identity, offset, alpha)` ？

因为四元数旋转不能像向量一样直接线性乘 `alpha` 。

所以要做：

- 从“无修正姿态” `identity`
- 插值到“完整修正姿态” `rotation_offsets[frame-f]`

这正是论文里对 quaternion parameter 的处理方式：

四元数要用 `slerp` 。

为什么位置可以直接：

```

Python |
1 local_pos = position_offsets[frame-f] * alpha

```

因为位置 offset 是 3-vector，可以直接 lerp / scale。

论文也明确说了：对 3-vector 参数，用 `lerp` 。

C5. forward 搜索：找最近的未来接触帧

```

Python |
1 for f in range(L4):
2     if frame+f < frame_count and np.any(foot_contacts[frame+f, :]):
3         front_hit = f
4         alpha = (f+1.0)/(L4+1.0)
5         alpha = 2*alpha*alpha*alpha - 3*alpha*alpha + 1
6         front_quat = quat_slerp(identity, rotation_offsets[frame+f],
7             alpha)
8         front_pos = position_offsets[frame+f] * alpha

```

和 backward 完全对称。

它得到的是：

- 从未来那个接触 frame 衰减回来的候选修正 `front_quat/front_pos`

C6. 如果前后两边都找到了，就再合成一次

```

Python |
1 if back_hit > -1:
2     alpha = (back_hit+1.0)/(back_hit+front_hit+1.0)
3     local_quat = quat_slerp(local_quat[np.newaxis,:], front_quat[np.newaxis,:], alpha)[0]
4     local_pos = local_pos*(1-alpha) + front_pos*(alpha)
5 else:
6     local_quat = front_quat
7     local_pos = front_pos

```

语义上就是：

- 如果只有一边有 hit，就直接用那一边
- 如果两边都有 hit
 - 那么当前帧更靠近哪边，就更偏向哪边的修正
 - 旋转继续用 `slerp`
 - 位置继续用线性插值

这和论文 Figure 3 里的三种情况一一对应：

1. 只受前向影响
2. 只受后向影响

3. 前后都有影响，需要跨中间区域混合。

C7. 这一段在“思想上”比 root 那段更干净

你会注意到 `_ankle` 这段在逻辑上非常完整：

- 当前帧没接触才处理
- backward / forward 真正在检查邻居帧
- 衰减用 `L4`
- rotation 用 `slerp`
- position 用 `lerp/scaled vector`

所以如果你要理解 Final Processing，以 `_ankle` 这段为主线最清楚。

root 那段更像是同一思路在 root 上的平移版。

Part D: 把 `_ankle` 应用到左脚和右脚

```
Python |  
1  ankle_rotation_buffer[frame, 0, :], ankle_position_buffer[frame, 0, :]  
    = _ankle(contacts[:, :2], ankle_rotation_buffer[:,0,:], ankle_posit  
      ion_buffer[:,0,:])  
2  ankle_rotation_buffer[frame, 1, :], ankle_position_buffer[frame, 1, :]  
    = _ankle(contacts[:, 2:], ankle_rotation_buffer[:,1,:], ankle_posit  
      ion_buffer[:,1,:])
```

这两行就是把 helper 真正落地到左右脚：

- `contacts[:, :2]`
 - 左脚两路接触： `left heel`, `left ball`
- `contacts[:, 2:]`
 - 右脚两路接触： `right heel`, `right ball`
- `0`
 - 左 ankle
- `1`
 - 右 ankle

这一层的意义

前面 helper 只是“定义算法”，这两行才是：

把 phase 5 的 ankle final processing 应到左脚和右脚整段动画上。

执行完以后：

- 左脚 ankle offset buffer 被平滑更新
- 右脚 ankle offset buffer 被平滑更新

Part E: `compute_final_animation()` 才是把平滑后的 buffer 兑现成最终动画

```
Python |  
1 solved_animation = compute_final_animation()
```

这行特别容易被忽略，但它其实是整个 Final Processing 和最终结果之间的桥。

它做什么

它会拿着：

- 已经平滑好的 `root_buffer`
- 已经平滑好的 `ankle_rotation_buffer`
- 已经平滑好的 `ankle_position_buffer`

去真正重建最终动画。

Jerome 在语音稿里明确说：

做完 final processing 以后，再去对这个 solution 做 IK，而不是先做 IK 再做 final processing。

这说明 notebook 的实现顺序是：

1. 先把 root / ankle 的 target offset 平滑好
2. 再一次性生成 `solved_animation`

也就是说，这个 notebook 里：

- Final Processing 操作的是 中间表示 (buffers)

- `compute_final_animation()` 才把中间表示变成最终可播姿态

这是一种很好的工程组织方式，因为这样：

- 平滑逻辑和 IK/materialization 解耦
- 更容易调试 buffer 是否合理

Part F: 后面的 `render(...)` 不是算法本体，是调试可视化

这整块：

```
Python |
1 def render(frame, solved=True):
2     ...
```

严格说不属于 Final Processing 算法本体。

它的作用是把刚刚平滑完的东西画出来给你看。

我给你拆一下它在看什么。

F1. 选择显示原动画还是修复后动画

```
Python |
1 anim = solved_animation if solved else animation
```

- `solved=True`
 - 看修复后的动画
- `solved=False`
 - 看原始滑脚版本

这是最基础的 before / after 切换。

F2. 画角色和地面



Python |

```
1 p = (anim.pos[frame,...])
2 q = (anim.quats[frame,...])
3
4 a = lab.utils.quat_to_mat(q, p)
5 viewer.set_shadow_poi(p[0])
6
7 viewer.begin_shadow()
8 viewer.draw(character, a)
9 viewer.end_shadow()
10
11 viewer.begin_display()
12 viewer.draw_ground()
13 viewer.draw(character, a)
```

这部分只是：

- 把当前帧的骨骼姿态转成矩阵
- 画角色
- 画阴影
- 画地面

它不改算法，只是显示。

F3. 画 constraint target



Python |

```
1 contacts = np.eye(4, dtype=np.float32)[np.newaxis,...].repeat(4, axis=0)
2 contacts[:, :3, 3] = constraint_positions_buffer[frame, :, :]
3
4 viewer.draw(target, contacts)
```

这里是把当前 frame 的四个约束点画成小 target。

你能直观看到：

- left heel target
- left ball target
- right heel target

- right ball target

这是检查“脚底应该锁在哪里”的最好方式。

F4. 画调试线和三个关键坐标轴

```
1 viewer.draw_lines(character.world_skeleton_lines(a))
```

画骨架线框。

然后是三组 `draw_axis(...)` :

左脚 ankle axis

```
1 gquat = lab.utils.quat_mul(ankle_rotation_buffer[frame, 0, :], agquats[frame, character.bone_index('LeftFoot'), :])
2 gpos = agpos[frame, character.bone_index('LeftFoot'), :] + ankle_position_buffer[frame, 0, :]
3 a = lab.utils.quat_to_mat(gquat, gpos)
4 viewer.draw_axis(a[np.newaxis, ...], 20)
```

右脚 ankle axis

```
1 gquat = lab.utils.quat_mul(ankle_rotation_buffer[frame, 1, :], agquats[frame, character.bone_index('RightFoot'), :])
2 gpos = agpos[frame, character.bone_index('RightFoot'), :] + ankle_position_buffer[frame, 1, :]
3 a = lab.utils.quat_to_mat(gquat, gpos)
4 viewer.draw_axis(a[np.newaxis, ...], 20)
```

hips / root axis

```
1 gquat = agquats[frame, character.bone_index('Hips'), :]  
2 gpos = agpos[frame, character.bone_index('Hips'), :] + root_buffer[frame, :]  
3 a = lab.utils.quat_to_mat(gquat, gpos)  
4 viewer.draw_axis(a[np.newaxis,...], 20)
```

这三组轴分别在检查什么

- 左脚 axis: Final Processing 后左脚 ankle offset 有没有平滑
- 右脚 axis: 右脚有没有平滑
- hips axis: root_buffer 有没有在边界前后柔和过渡

所以 `render()` 的价值是：

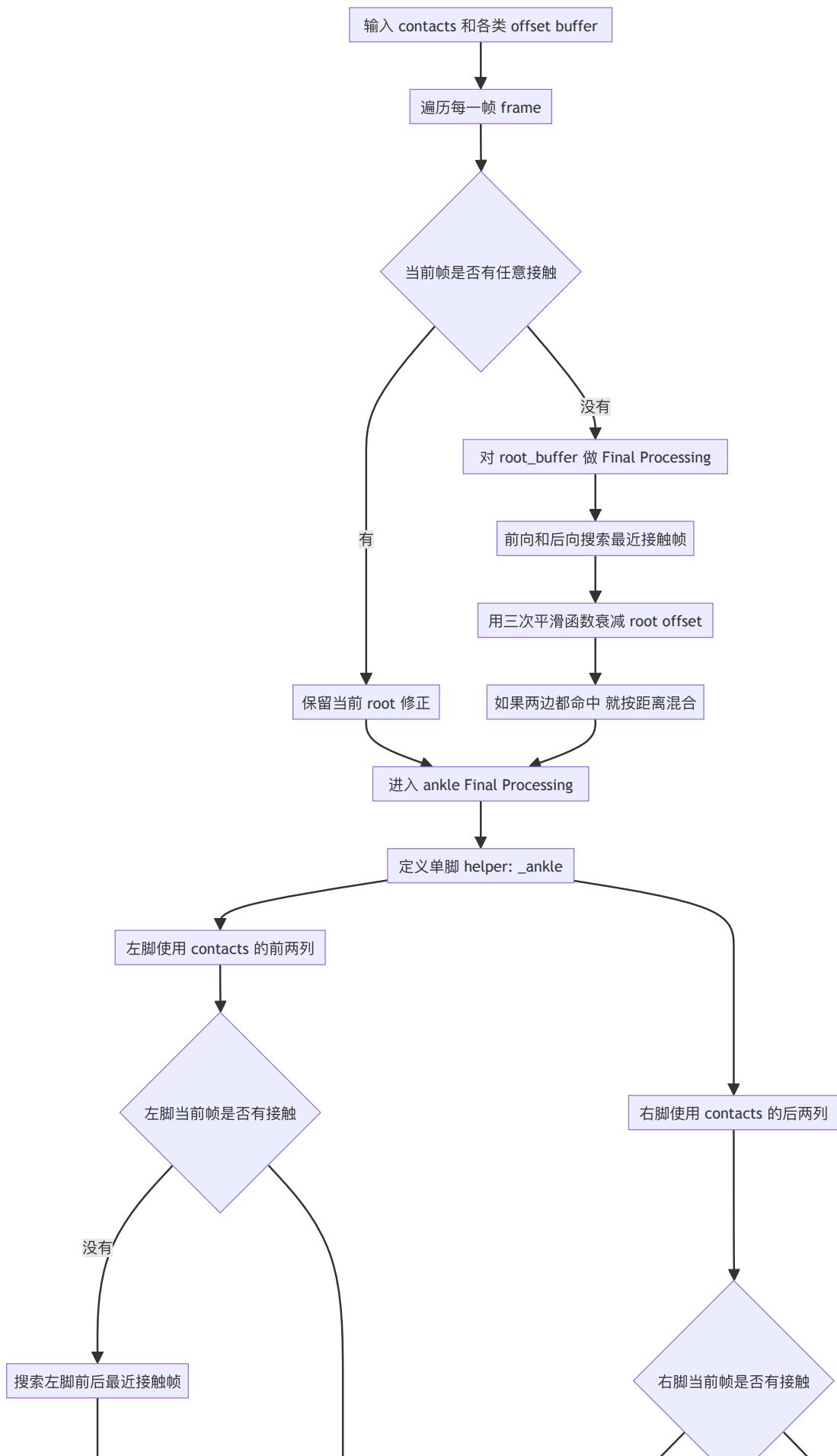
它把 Final Processing 这一步本来很抽象的“offset fade”变成了肉眼可检查的坐标轴变化。

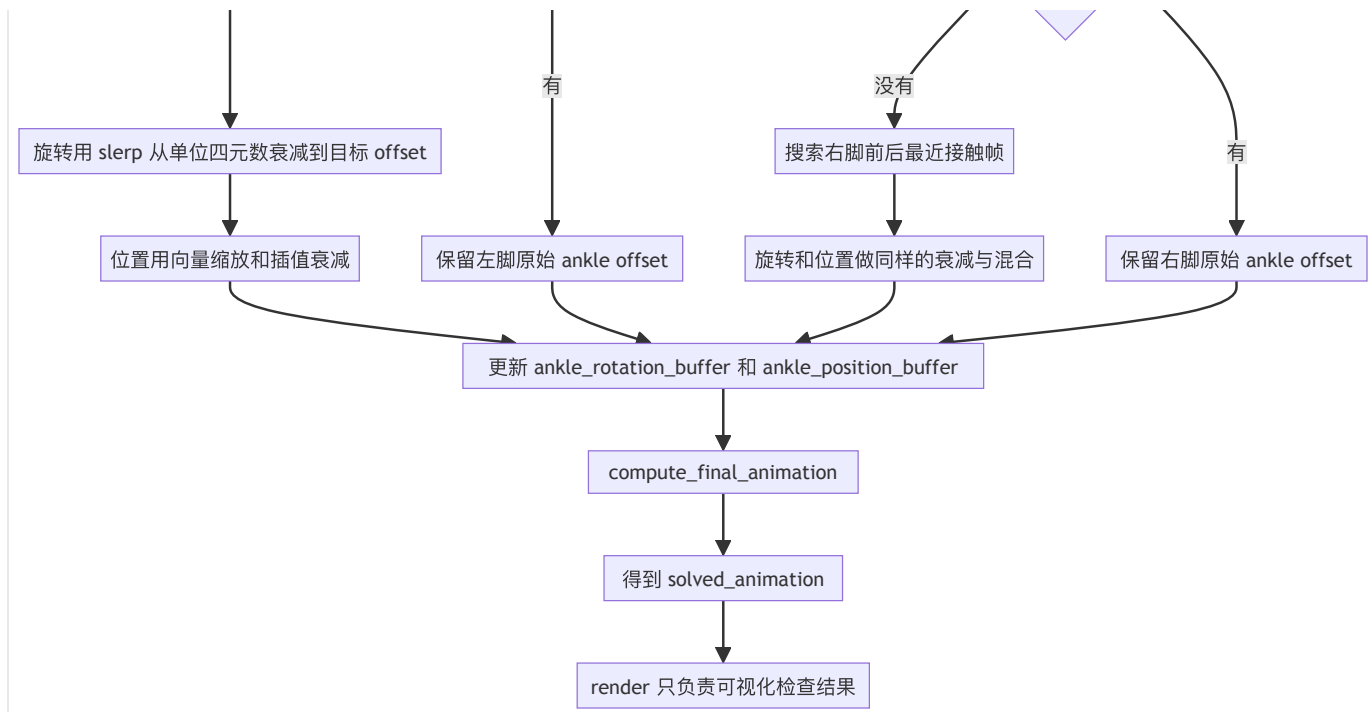
五、把整段 Final Processing 压成一条执行链

你可以把它记成下面这条链：

```
1  输入:
2  contacts
3  + root_buffer
4  + ankle_rotation_buffer
5  + ankle_position_buffer
6
7  对每一帧:
8      如果当前帧没有接触:
9          去前后 L4 帧里找最近的接触帧
10         把它们的 root / ankle offset 衰减后借过来
11         如果前后两边都找到了, 就按距离再混合一次
12
13  输出:
14  平滑后的 root_buffer
15  平滑后的 ankle_rotation_buffer
16  平滑后的 ankle_position_buffer
17
18  然后:
19  compute_final_animation()
20
21  最后:
22  得到 solved_animation 并可视化检查
```

数据流程图





效果对比

